MÔ HÌNH SOFTMAX REGRESSION

1. Từ Logistic Regression đến bài toán phân loại nhiều lớp

Hai bài trước đã đi một chặng đường rõ ràng. **Perceptron** biến điểm số tuyến tính $s = \mathbf{xw}$ thành một nhãn cứng bằng hàm dấu. **Logistic Regression** làm "mềm" hàm dấu thành hàm sigmoid, nhờ đó mô hình cho ra **xác suất** $p = \sigma(\mathbf{xw})$ và có một hàm mất mát trơn, lồi để huấn luyện. Cả hai mô hình đều giải **bài toán phân loại nhị phân**: chỉ có hai kết quả có thể xảy ra.

Thực tế lại thường có nhiều hơn hai kết quả. Một sinh viên có thể "rớt", "đạt" hoặc "giỏi". Một ảnh chữ số viết tay có thể là một trong 10 chữ số. Một email có thể là thư công việc, quảng cáo, mạng xã hội hay thư rác. Trong bài giảng này, ta xây dựng **Softmax Regression**, mô hình phân loại **nhiều lớp** bằng xác suất, bằng cách tiếp tục tận dụng những gì đã học. (Ở mục 6.3 của bài Logistic Regression, ta đã giới thiệu nhanh Softmax bằng công thức và một ví dụ trên dữ liệu Iris. Bài này đi sâu từ nền tảng: vì sao cần nó, nó được xây dựng như thế nào và hoạt động ra sao.)

Trong mục 1, ta trả lời ba câu hỏi:

- Logistic Regression còn hạn chế gì khi có **nhiều hơn hai lớp**?
- Bài toán phân loại nhiều lớp được mô tả toán học như thế nào, và **nhãn** nên được mã hóa ra sao?
- Có những **hướng giải** nào, và vì sao ta cần một mô hình xác suất chung thay vì ghép nhiều bộ phân loại nhị phân?

Quy ước ký hiệu

Ta giữ nguyên quy ước của hai bài trước: chữ thường in nghiêng là đại lượng vô hướng, **chữ thường in đậm là vector**, **chữ hoa in đậm là ma trận**. Vector đặc trưng là **vector hàng**, nhờ đó các vector đặc trưng của n mẫu xếp chồng thành các hàng của ma trận đặc trưng. Điểm mới là có K lớp, được đánh số $0, 1, \dots, K-1$, và mỗi lớp có **một vector tham số riêng**.

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Số mẫu, số đặc trưng, số lớp	n, d, K	vô hướng	n quan sát, d đặc trưng, K lớp
Vector đặc trưng (hàng)	$\mathbf{x}_i$	$1 \times (d+1)$	Đặc trưng của mẫu thứ i, phần tử đầu tiên bằng 1
Ma trận đặc trưng	$\mathbf{X}$	$n \times (d+1)$	Hàng thứ i của $\mathbf{X}$ là $\mathbf{x}_i$
Nhãn lớp	y_i, $\mathbf{y}$	vô hướng, $n \times 1$	$y_i \in \{0, 1, \dots, K-1\}$ là chỉ số lớp của mẫu i
Ma trận nhãn one-hot	$\mathbf{Y}$	$n \times K$	$y_{ik} = 1$ nếu mẫu i thuộc lớp k, ngược lại bằng 0

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Vector tham số của lớp k	$\mathbf{w}_k$	$(d+1)\times 1$	Cột thứ k của ma trận $\mathbf{W}$
Ma trận tham số	$\mathbf{W}$	$(d+1)\times K$	$\mathbf{W}=[\mathbf{w}_0 \quad \cdots \quad \mathbf{w}_{K-1}]$ (mục 2)
Ma trận điểm số	$\mathbf{S}=\mathbf{XW}$	$n\times K$	Phần tử $s_{ik}=\mathbf{x}_i\mathbf{w}_k$ là điểm số của mẫu i cho lớp k (mục 2)
Ma trận xác suất	$\mathbf{P}$	$n\times K$	p_{ik} là xác suất mô hình gán cho việc mẫu i thuộc lớp k (mục 2)
Nhãn dự đoán	$\hat{y}_i$, $\hat{\mathbf{y}}$	vô hướng, $n\times 1$	Chỉ số lớp mà mô hình dự đoán

Hai ma trận **Y** và **P** sẽ có **cùng kích thước** $n\times K$. Đây không phải ngẫu nhiên: ở mục 3, ta sẽ đặt chúng cạnh nhau để đo sai số, giống cách ta đặt **y** cạnh **p** ở bài Logistic Regression.

1.1. Hạn chế của Logistic Regression khi có nhiều hơn hai lớp

Chỉ có một xác suất

Logistic Regression cho ra **đúng một con số** $p=P(y=1\mid\mathbf{x})$, và xác suất của lớp còn lại tự động là $1-p$. Cấu trúc đó **vừa đủ cho hai kết quả** và không thể mở rộng trực tiếp ra ba kết quả. Với kết quả học tập ba mức (rớt, đạt, giỏi), ta cần **ba xác suất** p_0, p_1, p_2 không âm và cộng lại bằng 1. Một con số p duy nhất không đủ để biểu diễn điều đó.

Chữa cháy thử nghiệm: gán nhãn bằng số và hồi quy

Một ý tưởng tự nhiên là: ta đã có công cụ hồi quy rất mạnh, vậy hãy **gán cho ba lớp các số** 0,1,2 (rớt $=0$, đạt $=1$, giỏi $=2$), huấn luyện hồi quy tuyến tính để dự đoán số đó, rồi làm tròn kết quả về số nguyên gần nhất. Cách làm này nghe có vẻ hợp lý, nhưng nó **ngầm áp đặt hai điều** không có trong bản chất của bài toán:

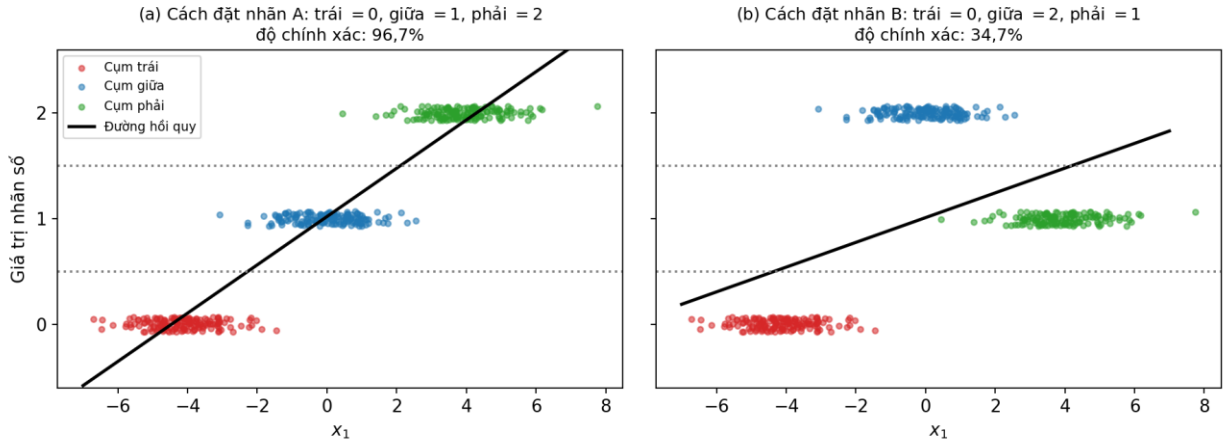
- **Áp đặt thứ tự.** Nhãn $0<1<2$ ngầm nói rằng "đạt" nằm giữa "rớt" và "giỏi". Với kết quả học tập, thứ tự này còn chấp nhận được, nhưng với các nhãn **danh nghĩa** (chuyên ngành, loài hoa, chữ số, loại email), việc "lớp 1 nằm giữa lớp 0 và lớp 2" hoàn toàn vô nghĩa. (Ở mục 1.2 của bài Perceptron, ta đã nhắc rằng việc gán nhãn bằng các số 0,1,2, ... vô tình áp đặt một thứ tự giữa các lớp.)
- **Áp đặt khoảng cách đều.** Hồi quy coi lỗi "đoán nhầm 0 thành 2" nặng gấp **hai** lần lỗi "đoán nhầm 0 thành 1". Với chữ số viết tay, nhầm chữ số 1 thành 9 không hề "nặng gấp tám lần" nhầm 1 thành 2. Chữ số ở đây chỉ là **tên gọi**, không phải số lượng.

Thí nghiệm: kết quả phụ thuộc vào cách đặt số cho lớp

Ta thấy rõ vấn đề bằng một thí nghiệm. Xét ba cụm dữ liệu nằm dọc theo trục x_1: cụm **trái** (quanh $x_1=-4$), cụm **giữa** (quanh $x_1=0$) và cụm **phải** (quanh $x_1=4$), mỗi cụm 150 điểm, tách biệt rõ. Ta thử **hai cách đặt số** cho ba cụm rồi dùng hồi quy tuyến tính với cùng hai đặc trưng, làm tròn kết quả:

Cách đặt nhãn	Trái	Giữa	Phải	Độ chính xác
A: trái = 0, giữa = 1, phải = 2	0	1	2	96,7%
B: trái = 0, giữa = 2, phải = 1	0	2	1	34,7%

Ở cách A, các nhãn xếp theo đúng thứ tự trong không gian nên một đường thẳng (qua 0, 1, 2) đi đúng qua ba cụm. Ở cách B, **dữ liệu hoàn toàn giống hệt**, chỉ khác tên gọi, nhưng cụm giữa mang nhãn 2 và cụm phải mang nhãn 1, nên **không có hàm tuyến tính nào** đi qua giá trị 0 ở bên trái, 2 ở giữa và 1 ở bên phải. Độ chính xác rơi xuống 34,7% (gần mức đoán mò là 33,3%), và **cụm giữa bị đoán sai** 100% (Hình 1).



Hình 1. Hồi quy tuyến tính trên nhãn dạng số cho cùng một dữ liệu ba cụm, chỉ khác cách đặt số cho lớp. (a) Cách A: đường hồi quy đi qua đúng ba cụm. (b) Cách B: không có đường thẳng nào làm được, cụm giữa (nhãn 2) bị đoán sai hoàn toàn.

Một mô hình phân loại tốt **không được phụ thuộc vào việc ta đánh số lớp thế nào**: đổi tên "lớp 1" thành "lớp 2" và ngược lại không được làm thay đổi chất lượng. Mô hình nhiều lớp dựa trên xác suất mà ta xây dựng từ mục 2 có tính chất này: với hai cách đặt nhãn A và B ở trên, nó đều đạt 96,9%.

Tóm tắt các hạn chế và điều ta cần

Hạn chế	Điều ta mong muốn
Logistic Regression chỉ có **một xác suất** p (cho hai kết quả)	K **xác suất** $p_0, \dots, p_{K-1}$, không âm và **cộng lại bằng** 1
Nhãn số 0,1,2 **áp đặt thứ tự và khoảng cách đều** giữa các lớp	Nhãn **danh nghĩa**: các lớp **bình đẳng**, không có "lớp nào ở giữa" (mã hóa one-hot, mục 1.2)
Kết quả **phụ thuộc cách đặt số** cho lớp	Mô hình **bất biến** khi đánh số lại các lớp

1.2. Mô tả bài toán phân loại nhiều lớp và mã hóa one-hot

Từ hai lớp sang K lớp

Giả sử ta có bộ dữ liệu huấn luyện gồm n quan sát và K lớp:

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}, \qquad y_i \in \{0,1, \dots, K-1\}$$

Giống như ở bài Logistic Regression, ta nhìn bài toán theo **xác suất**: với mỗi mẫu, hai sinh viên có cùng đặc trưng vẫn có thể có kết quả khác nhau, nên mô tả trung thực nhất là nói về **khả năng**. Mục tiêu bây giờ là ước lượng **một phân phối xác suất trên K lớp** cho mỗi mẫu:

$$p_{ik} = P(y = k \mid \mathbf{x}_i), \qquad p_{ik} \geq 0, \qquad \sum_{k=0}^{K-1} p_{ik} = 1$$

Nhãn dự đoán được lấy từ lớp có xác suất lớn nhất:

$$\hat{y}_i = \operatorname{argmax}_k \, p_{ik}$$

Với $K = 2$, công thức này trở về quy tắc quen thuộc: dự đoán lớp 1 nếu $p_{i1} \geq 0{,}5$ (khi đó $p_{i1} \geq p_{i0} = 1 - p_{i1}$).

Mã hóa one-hot

Vì các lớp không có thứ tự, ta không biểu diễn nhãn bằng một **số** mà bằng một **vector** có K phần tử, trong đó chỉ phần tử ứng với lớp đúng bằng 1 và các phần tử còn lại bằng 0. Cách này gọi là **mã hóa one-hot**. Với $K = 3$:

$$\text{rớt } (y = 0) \to [1 \quad 0 \quad 0], \qquad \text{đạt } (y = 1) \to [0 \quad 1 \quad 0], \qquad \text{giỏi } (y = 2) \to [0 \quad 0 \quad 1]$$

Xếp các vector one-hot của n mẫu thành các hàng, ta được **ma trận nhãn one-hot Y** kích thước $n \times K$:

$$y_{ik} = \mathbb{1}[\, y_i = k \,]$$

Có ba điểm đáng chú ý về cách mã hóa này:

- **Các lớp bình đẳng.** Mọi cặp vector one-hot cách nhau cùng một khoảng ($\sqrt{2}$), nên không có lớp nào "gần" lớp nào hơn, khác với nhãn số 0,1,2.
- **Mỗi hàng của Y là một phân phối xác suất.** Các phần tử không âm và cộng lại bằng 1, trong đó toàn bộ khối lượng xác suất dồn vào lớp đúng. Đây là **"xác suất mục tiêu"** mà ta muốn mô hình tiến tới.
- **Y và ma trận xác suất dự đoán P có cùng kích thước và cùng ý nghĩa** (mỗi hàng là một phân phối trên K lớp), nên so sánh chúng là việc tự nhiên. Ở mục 3, hàm mất mát và gradient sẽ được viết qua hiệu $\mathbf{P} - \mathbf{Y}$.

Quan hệ với nhãn nhị phân của Logistic Regression

Với $K = 2$, mỗi hàng của **Y** là $[1 - y_i \quad y_i]$ với $y_i \in \{0,1\}$. Như vậy **nhãn** $\{0,1\}$ **ở bài Logistic Regression chính là mã hóa one-hot rút gọn**: cột thứ hai của **Y** trùng với vector nhãn **y** của bài trước, còn cột thứ nhất chỉ là $1 - \mathbf{y}$ nên **thừa** (ta suy ra được từ cột kia). Chính điều này cho phép bài toán nhị phân chỉ cần **một** xác suất. Khi có K lớp, ta cần $K - 1$ xác suất độc lập (xác suất còn lại là phần bù), và đó là gợi ý đầu tiên cho điều ta sẽ gặp ở mục 2.5.

Ví dụ xuyên suốt: ba mức kết quả học tập

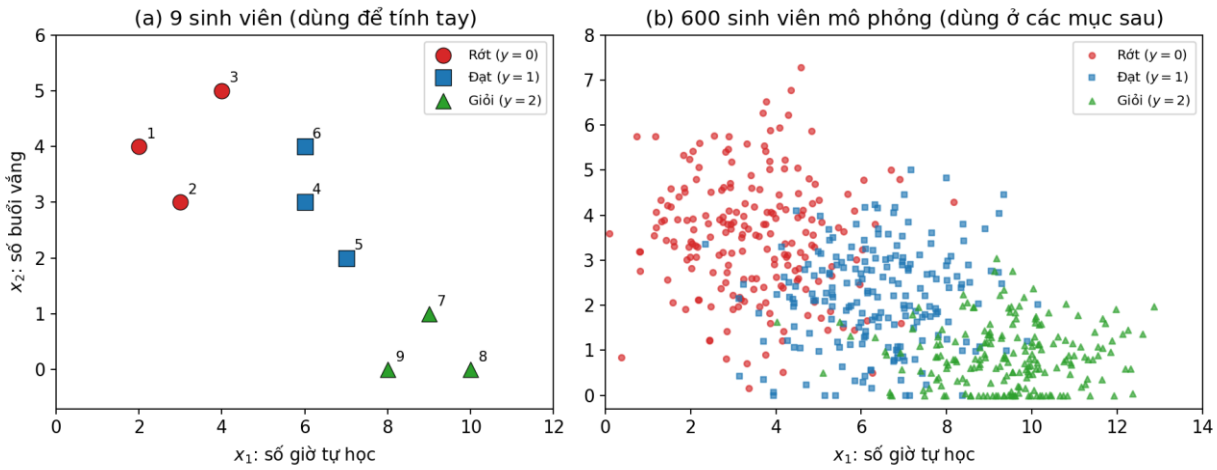
Ta mở rộng ví dụ sinh viên của hai bài trước từ hai kết quả (đậu, rớt) thành **ba mức**: rớt ($y = 0$), đạt ($y = 1$) và giỏi ($y = 2$), vẫn với hai đặc trưng x_1 là số giờ tự học mỗi tuần và x_2 là số buổi vắng. Để tính tay ở mục 3, ta dùng 9 sinh viên:

Sinh viên	x_1 (giờ tự học)	x_2 (buổi vắng)	y	Kết quả
1	2	4	0	rớt
2	3	3	0	rớt
3	4	5	0	rớt
4	6	3	1	đạt
5	7	2	1	đạt
6	6	4	1	đạt
7	9	1	2	giỏi
8	10	0	2	giỏi
9	8	0	2	giỏi

Ở đây $n = 9$, $d = 2$ và $K = 3$. Ma trận đặc trưng **X** (có cột 1), vector nhãn **y** và ma trận one-hot **Y** là

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 4 \\ 1 & 3 & 3 \\ 1 & 4 & 5 \\ 1 & 6 & 3 \\ 1 & 7 & 2 \\ 1 & 6 & 4 \\ 1 & 9 & 1 \\ 1 & 10 & 0 \\ 1 & 8 & 0 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 2 \\ 2 \\ 2 \end{bmatrix}, \quad \mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Ba sinh viên đầu thuộc lớp 0 (rớt), ba sinh viên tiếp theo thuộc lớp 1 (đạt), ba sinh viên cuối thuộc lớp 2 (giỏi). Cột thứ k của **Y** cho biết mẫu nào thuộc lớp k. Ở các mục sau, ta cũng dùng **bộ dữ liệu mô phỏng** 600 **sinh viên** (200 mỗi lớp, ba lớp chồng lấn nhau ở mức vừa phải, giống dữ liệu thật) để huấn luyện và đánh giá. Hình 2 vẽ cả hai bộ dữ liệu: sinh viên rớt tập trung ở vùng ít giờ học và nhiều buổi vắng, sinh viên giỏi ở vùng nhiều giờ học và ít buổi vắng, còn sinh viên đạt nằm **ở giữa**.



Hình 2. (a) 9 sinh viên dùng để tính tay: 3 rớt, 3 đạt, 3 giỏi. (b) Bộ dữ liệu mô phỏng 600 sinh viên (200 mỗi lớp) dùng ở các mục sau. Lớp "đạt" nằm giữa lớp "rớt" và lớp "giỏi" trong không gian đặc trưng.

1.3. Hai hướng giải: ghép nhiều bộ phân loại nhị phân và mô hình xác suất chung

Ta đã có Logistic Regression cho hai lớp. Một cách tự nhiên để xử lý K lớp là hỏi: **có thể tái sử dụng nó không?** Có hai hướng, và sự khác biệt giữa chúng chính là lý do ra đời của Softmax Regression.

Hướng 1: one-vs-rest (một lớp chống phần còn lại)

Ý tưởng của **one-vs-rest** (OvR) rất trực tiếp: huấn luyện K bộ phân loại Logistic Regression **độc lập**, trong đó bộ phân loại thứ k trả lời câu hỏi nhị phân *"mẫu này thuộc lớp k hay thuộc một lớp nào khác?"*. Nhãn nhị phân của bộ thứ k chính là **cột thứ k của Y**:

$$y_i^{(k)} = y_{ik} = \mathbb{1}[\, y_i = k \,]$$

Mỗi bộ có vector tham số $\mathbf{w}_k$ riêng và cho một xác suất $p_{ik}^{\text{OvR}} = \sigma(\mathbf{x}_i \mathbf{w}_k)$. Xếp các $\mathbf{w}_k$ thành các cột (hàng) ta được $\mathbf{W}$. Nhãn dự đoán là lớp có xác suất (hay tương đương, điểm số) lớn nhất, vì sigmoid là hàm tăng:

$$\hat{y}_i = \operatorname*{argmax}_k \sigma(\mathbf{x}_i \mathbf{w}_k) = \operatorname*{argmax}_k \mathbf{x}_i \mathbf{w}_k$$

Ưu điểm của OvR là **đơn giản** và tái sử dụng hoàn toàn mô hình nhị phân (có thể huấn luyện song song K mô hình). Nhưng khi thử trên dữ liệu thật, ta thấy nó có những nhược điểm đáng kể. Ta huấn luyện ba bộ phân loại OvR (Gradient Descent trên đặc trưng chuẩn hóa) trên 420 sinh viên huấn luyện của bộ dữ liệu 600 sinh viên, rồi xem kết quả trên 180 sinh viên kiểm tra.

Nhược điểm 1: lớp ở giữa bị che khuất. Hai bộ "rớt chống phần còn lại" và "giỏi chống phần còn lại" học tốt (giá trị hàm mất mát cuối cùng lần lượt là 0,172 và 0,192). Nhưng bộ "**đạt** chống phần còn lại" **gần như không học được gì**: các trọng số (w_1, w_2) gần 0, $w_0 \approx -0{,}694$,

nên nó trả về $p \approx \sigma(-0{,}694) = 0{,}333$ cho gần như mọi sinh viên, đúng bằng tỉ lệ lớp "đạt" trong dữ liệu. Hàm mất mát của nó là $0{,}636$, gần bằng mất mát $0{,}6365$ của một mô hình **chỉ biết tỉ lệ lớp**, không biết đặc trưng:

$$-[1/3 \ln 1/3 + 2/3 \ln 2/3] \approx 0{,}6365$$

Lý do: một đường thẳng không thể tách lớp "đạt" (nằm **giữa**) khỏi hai lớp ở hai phía. Đường thẳng nào cũng sẽ đặt lớp "đạt" cùng một phía với hoặc lớp "rớt" hoặc lớp "giỏi". Hiện tượng này gọi là lớp bị **che khuất (masking)**, và nó là điểm yếu kinh điển của OvR khi có lớp nằm giữa.

Nhược điểm 2: có vùng "không bộ nào nhận". Vì bộ "đạt" gần như không bao giờ trả lời "có" ($p \geq 0{,}5$), có những sinh viên mà **cả ba bộ đều trả lời "không"**. Trong 180 sinh viên kiểm tra, có 64 người ($35{,}6\%$) rơi vào tình huống này, và 116 người được đúng một bộ nhận. Trên mặt phẳng đặc trưng, vùng xám ở giữa trong Hình 3b chiếm khoảng 36% diện tích khung hình. Mô hình vẫn phải đưa ra quyết định bằng argmax, nhưng nó không còn mang ý nghĩa "bộ phân loại này tin chắc".

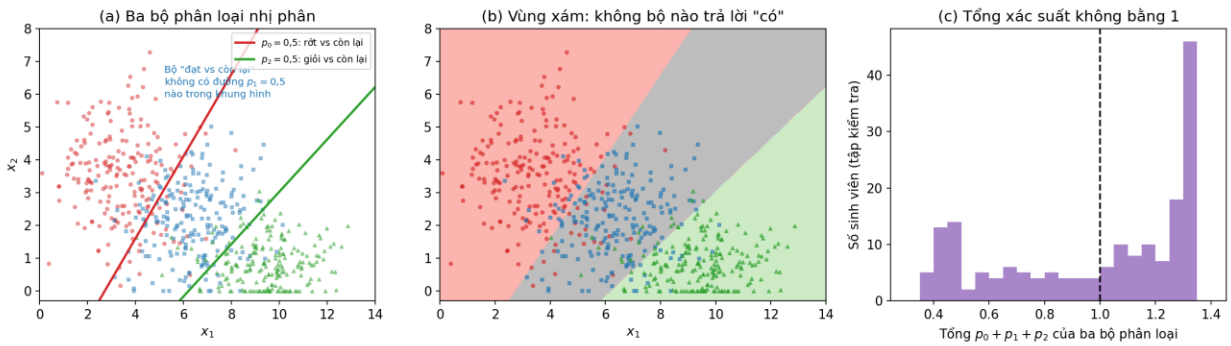
Nhược điểm 3: các xác suất không cộng lại bằng 1. Mỗi bộ chỉ trả lời về lớp của riêng mình, và chúng không biết gì về nhau. Trên 180 mẫu kiểm tra, tổng $p_0 + p_1 + p_2$ có trung bình $0{,}981$ nhưng dao động từ $0{,}357$ đến $1{,}349$, và $71{,}7\%$ số mẫu có tổng lệch khỏi 1 quá $0{,}2$ (Hình 3c). Ví dụ, sinh viên học 6 giờ và vắng 3 buổi có $(p_0, p_1, p_2) \approx (0{,}212;\ 0{,}324;\ 0{,}008)$, tổng chỉ $0{,}544$. Đây không phải là một phân phối xác suất, nên không thể diễn giải như "21% rớt, 32% đạt, 1% giỏi". Nếu chia mỗi p_k cho tổng để ép về 1 thì được con số đẹp hơn, nhưng đó chỉ là một thủ thuật **không có cơ sở xác suất**, và không giải quyết được hiện tượng che khuất.

Nhược điểm 4: mất cân bằng và chi phí. Mỗi bài toán nhị phân chỉ có $1/K$ số mẫu thuộc lớp dương (ở đây $1/3$, còn với chữ số viết tay $K = 10$ thì chỉ $1/10$ so với $9/10$). Ngoài ra, ta phải huấn luyện K mô hình riêng biệt, và các mô hình này **không "cạnh tranh"** với nhau trong lúc học.

Về chất lượng, OvR đạt độ chính xác $83{,}9\%$ trên tập kiểm tra, với ma trận nhầm lẫn (hàng là lớp thật $0{,}1{,}2$, cột là lớp dự đoán):

$$\begin{bmatrix} 53 & 7 & 0 \\ 15 & 40 & 5 \\ 0 & 2 & 58 \end{bmatrix}$$

Nhìn vào ma trận, lớp "rớt" được nhận ra đúng $53/60 \approx 88{,}3\%$ và lớp "giỏi" $58/60 \approx 96{,}7\%$, nhưng lớp "đạt" chỉ $40/60 \approx 66{,}7\%$: **lớp ở giữa chịu thiệt nhiều nhất**, đúng như phân tích về hiện tượng che khuất.



Hình 3. One-vs-rest trên 600 *sinh viên. (a) Đường* $p = 0{,}5$ *của hai bộ "rớt chống phần còn lại" và "giỏi chống phần còn lại"; bộ "đạt chống phần còn lại" không có đường nào đáng kể. (b) Vùng xám là nơi không bộ phân loại nào trả lời "có", nằm đúng ở khu vực của lớp "đạt". (c) Tổng ba xác suất của OvR dao động rất nhiều quanh* 1 *thay vì luôn bằng* 1.

Một biến thể khác là **one-vs-one**: huấn luyện một bộ phân loại cho **từng cặp** lớp ($K(K-1)/2$ bộ, tức 45 bộ với $K = 10$) rồi cho các bộ "bỏ phiếu". Nó khắc phục được hiện tượng che khuất nhưng tốn nhiều mô hình hơn và vẫn không cho ra một phân phối xác suất chung.

Hướng 2: một mô hình xác suất chung cho K lớp

Nguyên nhân gốc của các nhược điểm trên là OvR đặt ra K **câu hỏi "có hay không" độc lập**, trong khi bài toán thật chỉ có **một câu hỏi**: *"trong K lớp, mẫu này thuộc lớp nào?"* Hướng thứ hai tôn trọng điều đó: ta xây dựng **một mô hình duy nhất** cho ra **một phân phối xác suất trên** K **lớp**, ngay từ đầu đã thỏa mãn $p_{ik} \geq 0$ và $\sum_k p_{ik} = 1$.

Cách làm là tính K điểm số $s_{i0}, \ldots, s_{i,K-1}$ (mỗi lớp một điểm số tuyến tính $s_{ik} = \mathbf{x}_i \mathbf{w}_k$ như OvR) nhưng thay vì cho mỗi điểm số đi qua sigmoid **riêng lẻ**, ta đưa **cả** K **điểm số** qua một hàm chung để biến chúng thành K xác suất cộng lại bằng 1. Hàm đó là **softmax**, mở rộng của sigmoid cho nhiều lớp:

$$p_{ik} = \frac{e^{s_{ik}}}{\sum_{j=0}^{K-1} e^{s_{ij}}}$$

Ta sẽ xây dựng, phân tích và chứng minh các tính chất của nó ở mục 2 (đặc biệt là khi $K = 2$ nó **trở về đúng Logistic Regression**). Điểm khác biệt mấu chốt so với OvR là tính **cạnh tranh**: mẫu số $\sum_j e^{s_{ij}}$ chứa điểm số của mọi lớp, nên khi xác suất của một lớp tăng thì xác suất của các lớp còn lại **buộc phải giảm**. Các lớp cùng được huấn luyện **đồng thời**, và quan trọng hơn, ranh giới giữa hai lớp k và j là nơi $s_{ik} = s_{ij}$. Vùng của một lớp là phần mặt phẳng mà điểm số của lớp đó **thắng tất cả các lớp còn lại**, nên một lớp nằm giữa hai lớp khác có thể được "kẹp" bởi hai ranh giới, không cần tách tuyến tính khỏi toàn bộ phần còn lại như trong OvR (mục 2.5 sẽ phân tích hình học này).

Kết quả sơ bộ. Ta huấn luyện mô hình softmax (sẽ học ở mục 2 và 3) trên cùng dữ liệu, cùng cách chia và chuẩn hóa. Độ chính xác kiểm tra là 85,0%, nhỉnh hơn OvR (83,9%) một chút (chỉ cách nhau 2 sinh viên trên 180, nên đây chưa phải một khác biệt đáng kể). Khác biệt rõ nằm ở

lớp ở giữa: lớp "đạt" được nhận ra đúng $48/60 = 80{,}0\%$ so với $66{,}7\%$ của OvR, với ma trận nhầm lẫn

$$\begin{bmatrix} 49 & 11 & 0 \\ 9 & 48 & 3 \\ 0 & 4 & 56 \end{bmatrix}$$

Cái giá phải trả là lớp "rớt" và lớp "giỏi" nhận ra kém đi một chút ($49/60$ và $56/60$), vì mô hình chung phải **cân bằng** cả ba lớp thay vì để lớp giữa bị bỏ rơi.

Bảng sau tóm tắt hai hướng:

	One-vs-rest	Mô hình xác suất chung (Softmax)
Số mô hình	K mô hình nhị phân độc lập	1 **mô hình** với K cột tham số
Xác suất đầu ra	K con số độc lập, **không** cộng lại bằng 1	**Phân phối xác suất** thật, cộng lại bằng 1
Huấn luyện	Từng bộ riêng, không cạnh tranh	**Đồng thời**, các lớp cạnh tranh
Lớp nằm giữa	Bị che khuất	Được "kẹp" bởi hai ranh giới
Vùng "không ai nhận"	Có	Không (luôn có lớp thắng)
Khi $K = 2$	Logistic Regression	Logistic Regression

Mối liên hệ với các bài trước

Chuỗi ba bài giảng đi theo một mạch mà mỗi bài chỉ thay đổi **một thành phần** của khung "mô hình, hàm mất mát, thuật toán tối ưu":

$$\text{Perceptron} \xrightarrow{\text{làm mềm hàm dấu}} \text{Logistic Regression} \xrightarrow{\text{từ một điểm số sang } K \text{ điểm số}} \text{Softmax Regression}$$

Ở các bài trước, ta đã thấy ý tưởng "mở rộng sang nhiều lớp" xuất hiện hai lần. Ở bài Perceptron (mục 6.1), với K lớp ta dùng ma trận $\mathbf{W}$ và quy tắc cập nhật "**kéo** cột của lớp đúng lại gần mẫu, **đẩy** cột của lớp bị chọn nhầm ra xa". Ở bài Logistic Regression (mục 6.3), ta đã giới thiệu softmax và cho thấy gradient của nó có cùng dạng $\frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$. Bài này nối hai mảnh đó lại thành một mô hình hoàn chỉnh.

Tóm tắt mục 1

- Logistic Regression cho **một** xác suất và chỉ phù hợp với **hai** lớp. Với $K > 2$ lớp, ta cần K xác suất không âm cộng lại bằng 1.
- Việc **gán nhãn bằng số** 0,1,2, ... rồi hồi quy ngầm áp đặt **thứ tự và khoảng cách đều**, và kết quả **phụ thuộc cách đặt số**: với cùng dữ liệu, chỉ đổi tên hai lớp mà độ chính xác rơi từ $96{,}7\%$ xuống $34{,}7\%$.
- Bài toán được mô tả bằng $y_i \in \{0, \dots, K-1\}$ và mục tiêu ước lượng $p_{ik} = P(y = k \mid \mathbf{x}_i)$ với $\sum_k p_{ik} = 1$, nhãn dự đoán $\hat{y}_i = \text{argmax}_k p_{ik}$.

- **Mã hóa one-hot Y** ($n \times K$, $y_{ik} = \mathbb{1}[y_i = k]$) biểu diễn các lớp một cách bình đẳng và có cùng kích thước với ma trận xác suất **P**. Với $K = 2$, nhãn $\{0,1\}$ ở bài Logistic Regression chính là one-hot rút gọn.
- **One-vs-rest** huấn luyện K bộ nhị phân độc lập (nhãn là cột k của **Y**). Nó đơn giản nhưng lớp **ở giữa bị che khuất** (bộ "đạt chống phần còn lại" gần như không học được gì), có **vùng không bộ nào nhận** (35,6% số mẫu kiểm tra) và các xác suất **không cộng lại bằng** 1 (tổng từ 0,357 đến 1,349).
- **Mô hình xác suất chung** dùng K điểm số $s_{ik} = \mathbf{x}_i \mathbf{w}_k$ đưa qua **softmax**, cho phân phối xác suất thật và các lớp **cạnh tranh**. Khi $K = 2$ nó trở về Logistic Regression. Mục 2 sẽ xây dựng mô hình này.

Câu hỏi kiểm tra nhanh

1. Với $K = 4$ lớp, chuyển vector nhãn $\mathbf{y} = [2 \quad 0 \quad 3 \quad 1]^T$ sang ma trận one-hot **Y**. Tổng mỗi hàng và mỗi cột của **Y** bằng bao nhiêu?
2. Chuyển nhãn nhị phân $\mathbf{y} = [0 \quad 1 \quad 1 \quad 0]^T$ của bài Logistic Regression sang one-hot với $K = 2$. Cột nào của **Y** trùng với **y**?
3. Vì sao việc gán nhãn chữ số viết tay bằng chính giá trị chữ số rồi dùng hồi quy tuyến tính là không hợp lý? Nêu hai lý do.
4. Với $K = 10$ lớp cân bằng, tỉ lệ lớp dương của mỗi bài toán nhị phân trong one-vs-rest là bao nhiêu? Cần huấn luyện bao nhiêu mô hình nếu dùng one-vs-rest, và nếu dùng one-vs-one?
5. Ba bộ phân loại one-vs-rest cho một mẫu các xác suất $(p_0, p_1, p_2) = (0{,}62;\ 0{,}35;\ 0{,}41)$. Tính tổng, lớp dự đoán và các xác suất sau khi chia cho tổng.
6. Một bộ phân loại nhị phân cho lớp chiếm $1/3$ dữ liệu, chỉ biết tỉ lệ lớp (không dùng đặc trưng). Tính hàm mất mát cross-entropy của nó. Vì sao con số này là một "mốc" để đánh giá bộ "đạt chống phần còn lại" ở mục 1.3?
7. Cho $K = 3$ và các xác suất của one-vs-rest $(0{,}30;\ 0{,}20;\ 0{,}10)$. Mô hình dự đoán lớp nào? Có bộ nào trả lời "có" không? Tổng các xác suất là bao nhiêu?

Gợi ý đáp án

1. $\mathbf{Y} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}$. Mỗi **hàng** cộng lại bằng 1 (mỗi mẫu thuộc đúng một lớp). Mỗi **cột** cộng lại bằng số mẫu thuộc lớp đó, ở đây bằng 1 vì mỗi lớp xuất hiện đúng một lần.
2. $\mathbf{Y} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$. **Cột thứ hai** của **Y** trùng với $\mathbf{y} = [0 \quad 1 \quad 1 \quad 0]^T$, còn cột đầu là $1 - \mathbf{y}$ nên thừa.
3. Thứ nhất, chữ số chỉ là **tên gọi** của lớp, không phải số lượng: việc "chữ số 4 nằm giữa 3 và 5" hay "0 cách 9 xa nhất" không có ý nghĩa nhận diện. Thứ hai, hồi quy coi lỗi nhầm 1 thành 9 **nặng gấp** 8 **lần** nhầm 1 thành 2, và dự đoán những giá trị như 4,5 (nằm giữa

hai chữ số). Ngoài ra, kết quả còn **phụ thuộc cách đặt số** cho lớp, điều vô lý với nhãn danh nghĩa (Hình 1).

4. Tỉ lệ lớp dương là $1/10 = 10\%$ (so với 90% lớp âm). One-vs-rest cần $K = 10$ mô hình. One-vs-one cần $\frac{K(K-1)}{2} = \frac{10 \cdot 9}{2} = 45$ mô hình.
5. Tổng $= 0{,}62 + 0{,}35 + 0{,}41 = 1{,}38$, lớn hơn 1 nên không phải phân phối xác suất. Lớp dự đoán là lớp 0 (xác suất lớn nhất $0{,}62$). Chia cho tổng: $[0{,}449 \quad 0{,}254 \quad 0{,}297]$ (cộng lại bằng 1), nhưng đây chỉ là một thủ thuật chuẩn hóa không có cơ sở xác suất.
6. Mô hình chỉ biết tỉ lệ lớp dự đoán $p = 1/3$ cho mọi mẫu, nên mất mát trung bình là $-\left[\frac{1}{3}\ln\frac{1}{3} + \frac{2}{3}\ln\frac{2}{3}\right] = \frac{1}{3} \cdot 1{,}0986 + \frac{2}{3} \cdot 0{,}4055 \approx 0{,}3662 + 0{,}2703 = 0{,}6365$. Đây là mức mất mát "khi không dùng đặc trưng". Bộ "đạt chống phần còn lại" có mất mát $0{,}636$, **gần như bằng** mốc này, nghĩa là nó **gần như không khai thác được thông tin nào** từ đặc trưng.
7. Dự đoán là lớp 0 (xác suất lớn nhất $0{,}30$). **Không** bộ nào trả lời "có" vì cả ba xác suất đều nhỏ hơn $0{,}5$, nghĩa là mẫu rơi vào vùng "không bộ nào nhận". Tổng các xác suất là $0{,}30 + 0{,}20 + 0{,}10 = 0{,}60 \neq 1$.

2. Mô hình Softmax Regression

Ở mục 1, ta đã thấy Logistic Regression chỉ có **một** xác suất và không đủ cho $K > 2$ lớp, và ghép K bộ phân loại nhị phân kiểu one-vs-rest thì cho ra các xác suất không cộng lại bằng 1, lại để lớp ở giữa bị che khuất. Ta đã dừng ở một ý tưởng: **tính K điểm số tuyến tính rồi đưa cả K điểm số qua một hàm chung để thu được một phân phối xác suất thật**. Mục này biến ý tưởng đó thành một mô hình hoàn chỉnh, gọi là **Softmax Regression** (hồi quy softmax, còn gọi là hồi quy logistic đa lớp).

Trong mục này, ta trả lời ba câu hỏi:

- Softmax Regression nhận đầu vào, xử lý và đưa ra K xác suất như thế nào?
- Mô hình được viết gọn bằng vector và ma trận ra sao, và hàm softmax có những tính chất gì?
- Các vùng quyết định trông như thế nào, và các trọng số có ý nghĩa gì khi mô hình có **nhiều cột** tham số?

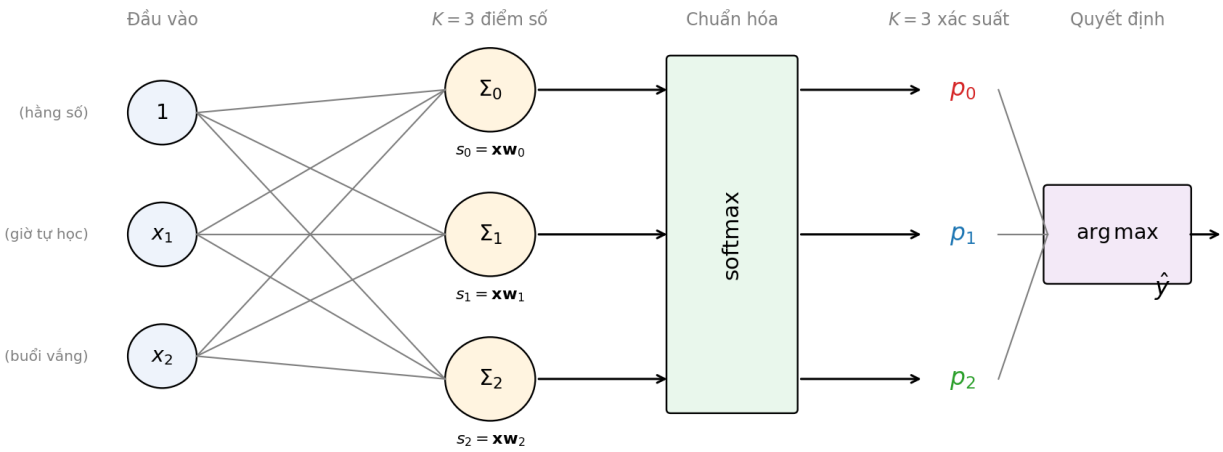
Ta tiếp tục dùng quy ước ký hiệu ở mục 1: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng** với phần tử đầu tiên bằng 1, các lớp được đánh số $0, \dots, K-1$ với nhãn $y_i \in \{0, \dots, K-1\}$, $\mathbf{Y}$ là ma trận one-hot $n \times K$, và $\mathbf{W} = [\mathbf{w}_0 \quad \cdots \quad \mathbf{w}_{K-1}]$ là ma trận tham số $(d+1) \times K$ với mỗi **cột** là vector tham số của một lớp.

2.1. Ý tưởng hoạt động của Softmax Regression

Có thể hình dung Softmax Regression như một **hội đồng K giám khảo**. Mỗi lớp có một giám khảo chuyên trách: giám khảo "rớt" chấm điểm theo tiêu chí của lớp rớt, giám khảo "đạt" chấm theo tiêu chí của lớp đạt, giám khảo "giỏi" chấm theo tiêu chí của lớp giỏi. Quy trình gồm bốn bước:

1. **Nhận đặc trưng.** Mô hình nhận vào $x_1, \dots, x_d$ của một mẫu.
2. **Chấm điểm.** Mỗi giám khảo k tính một **điểm số tuyến tính** $s_k = \mathbf{x}\mathbf{w}_k$, giống hệt điểm số của Logistic Regression nhưng với bộ trọng số $\mathbf{w}_k$ riêng. Ta được K điểm số $s_0, \dots, s_{K-1}$.
3. **Quy đổi thành xác suất.** Hàm **softmax** biến K điểm số thành K xác suất không âm **cộng lại bằng** 1, như chia 100% số phiếu cho các giám khảo theo điểm số của họ. Điểm khác quyết định so với one-vs-rest nằm ở bước này: các giám khảo **không chấm độc lập**, vì điểm của giám khảo này làm giảm phần phiếu của các giám khảo còn lại.
4. **Ra quyết định.** Lớp có xác suất lớn nhất là lớp được dự đoán, $\hat{y} = \text{argmax}_k p_k$.

Hình 4 minh họa quy trình này với ví dụ ba mức kết quả học tập ($d = 2, K = 3$).



Hình 4. Cấu trúc của Softmax Regression với hai đặc trưng và ba lớp. Mỗi lớp có một điểm số tuyến tính riêng, mỗi đầu vào nối tới cả ba điểm số. Hàm softmax chuyển ba điểm số thành ba xác suất cộng lại bằng 1*, và* $argmax$ *chọn lớp dự đoán.*

So với Logistic Regression, quy trình chỉ khác ở số lượng:

	Logistic Regression	Softmax Regression
Số điểm số	1 điểm số $s = \mathbf{xw}$	K điểm số $s_k = \mathbf{xw}_k$
Hàm chuẩn hóa	Sigmoid $\sigma(s)$	Softmax trên cả K điểm số
Đầu ra	1 xác suất p (xác suất lớp còn lại là $1 - p$)	K xác suất cộng lại bằng 1
Số tham số	$d + 1$	$K(d + 1)$
Quyết định	$\hat{y} = 1$ nếu $p \geq 0{,}5$	$\hat{y} = \text{argmax}_k p_k$

2.2. Ma trận tham số $\mathbf{W}$ và điểm số $\mathbf{S} = \mathbf{XW}$

Điểm số của một lớp

Với một mẫu có vector đặc trưng $\mathbf{x} = [1 \quad x_1 \quad \cdots \quad x_d]$ và lớp k, điểm số là tổng có trọng số của các đặc trưng cộng với hệ số chệch, hoàn toàn giống Perceptron và Logistic Regression:

$$s_k = w_{0k} + w_{1k}x_1 + \cdots + w_{dk}x_d = \mathbf{xw}_k, \qquad \mathbf{w}_k = [w_{0k} \quad w_{1k} \quad \cdots \quad w_{dk}]^T$$

Ý nghĩa của từng thành phần cũng quen thuộc, nhưng bây giờ **theo từng lớp**:

- w_{0k} là **hệ số chệch của lớp** k: điểm số "nền" mà lớp k nhận được khi mọi đặc trưng bằng 0.
- w_{jk} cho biết đặc trưng x_j **ủng hộ hay phản đối** lớp k: nếu $w_{jk} > 0$ thì x_j tăng làm điểm số của lớp k tăng, nếu $w_{jk} < 0$ thì làm giảm.

Gộp K điểm số bằng một ma trận

Xếp K vector $\mathbf{w}_0, \ldots, \mathbf{w}_{K-1}$ thành các **cột** của ma trận tham số $\mathbf{W}$ kích thước $(d + 1) \times K$:

$$\mathbf{W} = [\mathbf{w}_0 \quad \mathbf{w}_1 \quad \cdots \quad \mathbf{w}_{K-1}] = \begin{bmatrix} w_{00} & w_{01} & \cdots & w_{0,K-1} \\ w_{10} & w_{11} & \cdots & w_{1,K-1} \\ \vdots & \vdots & & \vdots \\ w_{d0} & w_{d1} & \cdots & w_{d,K-1} \end{bmatrix}$$

Khi đó vector hàng $\mathbf{x}_i$ nhân với $\mathbf{W}$ cho ra **một hàng** gồm K điểm số của mẫu i, và với toàn bộ n mẫu, một phép nhân ma trận cho ma trận điểm số $\mathbf{S}$ kích thước $n \times K$:

$$\mathbf{S} = \mathbf{XW} = [\mathbf{Xw}_0 \quad \mathbf{Xw}_1 \quad \cdots \quad \mathbf{Xw}_{K-1}], \qquad s_{ik} = \mathbf{x}_i \mathbf{w}_k$$

Cột thứ k của $\mathbf{S}$ chính là vector điểm số $\mathbf{Xw}_k$ của "bài toán thứ k" mà ta đã quen ở bài Logistic Regression. Nói cách khác, $\mathbf{S} = \mathbf{XW}$ là K **bộ chấm điểm tuyến tính chạy song song**, và việc tính cả K điểm số cho cả n mẫu chỉ là một phép nhân ma trận. Kiểm tra kích thước: $\mathbf{X}$ là $n \times (d+1)$ và $\mathbf{W}$ là $(d+1) \times K$ nên $\mathbf{XW}$ là $n \times K$.

Mô hình có tổng cộng $K(d+1)$ tham số. Với ví dụ ba mức kết quả học tập ($d = 2, K = 3$), đó là $3 \times 3 = 9$ tham số, gấp ba so với $d + 1 = 3$ tham số của Logistic Regression.

> **Lưu ý.** Hình thức của **W giống hệt** ma trận tham số $\mathbf{W}$ của one-vs-rest ở mục 1.3. Sự khác biệt giữa hai mô hình **không nằm ở cách tính điểm số** mà ở hai điều đi sau: điểm số được chuyển thành xác suất như thế nào (sigmoid độc lập hay softmax chung), và $\mathbf{W}$ được huấn luyện ra sao (từng cột riêng lẻ hay cả ma trận đồng thời, mục 3).

Ví dụ: một bộ trọng số cho ba mức kết quả

Để có số cụ thể, ta chọn tay một ma trận tham số cho 9 sinh viên ở mục 1.2 (mục 3 sẽ để máy tự tìm bộ tham số):

$$\mathbf{W} = \begin{bmatrix} 1{,}5 & 0 & -6{,}5 \\ -1 & 0 & 1 \\ 1 & 0 & -1 \end{bmatrix}, \quad \mathbf{w}_0 = \begin{bmatrix} 1{,}5 \\ -1 \\ 1 \end{bmatrix}, \quad \mathbf{w}_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{w}_2 = \begin{bmatrix} -6{,}5 \\ 1 \\ -1 \end{bmatrix}$$

Các điểm số tương ứng là

$$s_0 = 1{,}5 - x_1 + x_2, \qquad s_1 = 0, \qquad s_2 = -6{,}5 + x_1 - x_2$$

Có thể đọc như sau: điểm của lớp **rớt** (s_0) **giảm** theo số giờ học và **tăng** theo số buổi vắng. Điểm của lớp **giỏi** (s_2) làm ngược lại: tăng theo số giờ học và giảm theo số buổi vắng. Lớp **đạt** có điểm cố định bằng 0 và đóng vai trò "mốc": lớp nào có điểm cao hơn mốc này thì thắng lớp đạt. (Tại sao một cột có thể bằng 0 mà mô hình vẫn đầy đủ, ta sẽ giải thích ở mục 2.5.)

2.3. Hàm softmax, các tính chất và quan hệ với sigmoid khi $K = 2$

Định nghĩa

Với một mẫu có K điểm số $\mathbf{s} = [s_0 \quad \cdots \quad s_{K-1}]$, **hàm softmax** biến chúng thành K xác suất:

$$p_k = \text{softmax}(\mathbf{s})_k = \frac{e^{s_k}}{\sum_{j=0}^{K-1} e^{s_j}}, \qquad k = 0, 1, \ldots, K-1$$

Hàm làm hai việc nối tiếp:

1. **Lấy mũ** e^{s_k} để biến điểm số (có thể âm, dương bất kỳ) thành số **dương**. Hàm mũ cũng là hàm tăng.
2. **Chia cho tổng** $\sum_j e^{s_j}$ để các số này **cộng lại bằng** 1.

Ví dụ. Với $K = 3$ điểm số $\mathbf{s} = [0{,}5 \quad 1{,}5 \quad 2{,}5]$:

$$e^{0,5} = 1{,}6487, \quad e^{1,5} = 4{,}4817, \quad e^{2,5} = 12{,}1825 \;\Rightarrow\; \text{tổng} = 18{,}3129 \;\Rightarrow\; \mathbf{p} \approx [0{,}090 \quad 0{,}245 \quad 0{,}665]$$

Lớp có điểm số cao nhất (lớp 2) nhận phần xác suất lớn nhất, nhưng các lớp còn lại **không** bị gạt về 0: điểm số thấp hơn chỉ làm xác suất nhỏ đi một cách **mượt**. Tên "softmax" cũng từ đây: nếu "max cứng" chỉ chọn một lớp thắng và gán 1 cho nó, 0 cho các lớp còn lại, thì softmax là **phiên bản "mềm"** của phép chọn đó.

Giả thiết của Softmax Regression

Từ định nghĩa, tỉ số xác suất của hai lớp bất kỳ k và l là

$$\frac{p_k}{p_l} = \frac{e^{s_k}}{e^{s_l}} = e^{s_k - s_l} \quad \Rightarrow \quad \ln\frac{p_k}{p_l} = s_k - s_l = \mathbf{x}(\mathbf{w}_k - \mathbf{w}_l)$$

Đây chính là **giả thiết của Softmax Regression**: *log-odds giữa hai lớp bất kỳ là hàm tuyến tính của đặc trưng*. Nó là sự mở rộng trực tiếp của giả thiết ở bài Logistic Regression (mục 1.3), nơi ta đã giả thiết $\ln\frac{p}{1-p} = \mathbf{xw}$ cho hai lớp. Kiểm chứng với ví dụ: $\ln\frac{0{,}665}{0{,}245} \approx 1{,}0 = 2{,}5 - 1{,}5$.

Các tính chất

Tính chất	Biểu thức	Giải thích
Là phân phối xác suất	$0 < p_k < 1$ và $\sum_k p_k = 1$	Tử số dương và nhỏ hơn mẫu số; tổng các tử bằng mẫu số
Đơn điệu	$s_k > s_l \Leftrightarrow p_k > p_l$	e^s là hàm tăng, và mọi p có cùng mẫu số
Hệ quả cho dự đoán	$\text{argmax}_k p_k = \text{argmax}_k s_k$	Chỉ cần so điểm số, không cần tính mũ
Bất biến khi dịch	$\text{softmax}(\mathbf{s} + c) = \text{softmax}(\mathbf{s})$	Xem chứng minh bên dưới
Tỉ số xác suất	$p_k / p_l = e^{s_k - s_l}$	Chia hai xác suất: mẫu số triệt tiêu
Liên hệ với sigmoid	$K = 2$: $p_1 = \sigma(s_1 - s_0)$	Xem chứng minh bên dưới

Tính bất biến khi dịch. Cộng cùng một hằng số c vào mọi điểm số không làm đổi các xác suất:

$$\frac{e^{s_k + c}}{\sum_j e^{s_j + c}} = \frac{e^c\, e^{s_k}}{e^c \sum_j e^{s_j}} = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Điều này có nghĩa là softmax chỉ phụ thuộc vào **hiệu** giữa các điểm số, không phụ thuộc vào giá trị tuyệt đối của chúng. Tính chất này có hai hệ quả quan trọng: một là hệ quả **tính toán** (ngay bên dưới), hai là hệ quả về **tính dư thừa tham số** (mục 2.5).

Hệ quả tính toán. Khi điểm số rất lớn, e^s tràn số. Ví dụ với $\mathbf{s} = [1000 \quad 1001 \quad 1002]$, e^{1000} vượt quá giới hạn của số thực máy tính. Nhờ tính bất biến, ta **trừ điểm số lớn nhất của từng hàng** trước khi lấy mũ (khi đó số mũ lớn nhất bằng 0, nên không tràn). Hàm *softmax* sau (cũng là hàm đã dùng ở mục 6.3 bài Logistic Regression) làm đúng điều này:

```
def softmax(S):
    """Softmax theo từng hàng; trừ giá trị lớn nhất của hàng để không tràn số."""
    S = S - S.max(axis=1, keepdims=True)
    E = np.exp(S)
    return E / E.sum(axis=1, keepdims=True)

s = np.array([[1000., 1001., 1002.]])
with np.errstate(over="ignore", invalid="ignore"):
    print("naive :", np.exp(s) / np.exp(s).sum(axis=1, keepdims=True))
print("on dinh:", softmax(s))
```

```
naive : [[nan nan nan]]
on dinh: [[0.09   0.2447 0.6652]]
```

Cách tính trực tiếp cho kết quả *nan* (vì ∞/∞), còn cách trừ giá trị lớn nhất cho đúng kết quả $[0{,}090 \quad 0{,}245 \quad 0{,}665]$. Đây là kết quả **giống hệt** ví dụ trên với $\mathbf{s} = [0{,}5 \quad 1{,}5 \quad 2{,}5]$, vì $[1000 \quad 1001 \quad 1002]$ chỉ hơn nó một hằng số, đúng như tính bất biến khi dịch.

Tác động của độ lớn điểm số. Nhân tất cả điểm số với một hệ số $c > 0$ (không đổi thứ tự lớp) làm xác suất **quyết liệt hơn** nếu c lớn và **thoải hơn** nếu c nhỏ. Với $\mathbf{s} = [0{,}5 \quad 1{,}5 \quad 2{,}5]$:

Hệ số c	p_0	p_1	p_2
0,5	0,186	0,307	0,507
1	0,090	0,245	0,665
2	0,016	0,117	0,867
5	0,000	0,007	0,993

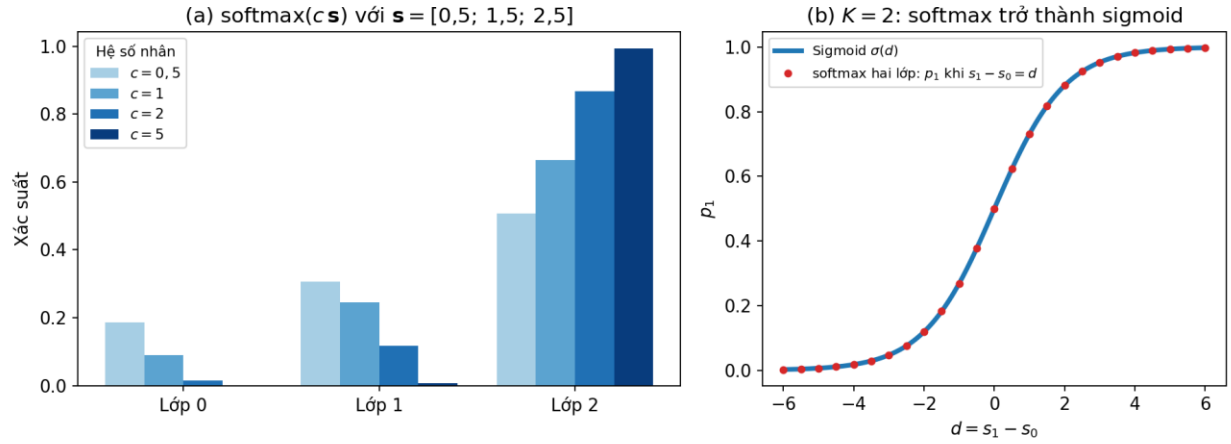
Khi $c \to \infty$, softmax tiến tới "max cứng": toàn bộ xác suất dồn vào lớp có điểm số cao nhất (Hình 5a). Đây là phiên bản nhiều lớp của hiện tượng ở bài Logistic Regression (mục 1.3), nơi $\sigma(c\,s)$ trở thành hàm bước khi $c \to \infty$, và cũng là lý do độ lớn của $\mathbf{W}$ điều khiển **độ tự tin** của mô hình.

Khi $K = 2$, softmax trở thành sigmoid

Với hai lớp có điểm số $s_0 = \mathbf{x}\mathbf{w}_0$ và $s_1 = \mathbf{x}\mathbf{w}_1$, xác suất của lớp 1 là

$$p_1 = \frac{e^{s_1}}{e^{s_0} + e^{s_1}} = \frac{1}{e^{s_0 - s_1} + 1} = \frac{1}{1 + e^{-(s_1 - s_0)}} = \sigma(s_1 - s_0) = \sigma\big(\mathbf{x}(\mathbf{w}_1 - \mathbf{w}_0)\big)$$

(chia cả tử và mẫu cho e^{s_1}), và $p_0 = 1 - p_1$. Đây **đúng là Logistic Regression** với vector tham số $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_0$. Vậy **Softmax Regression bao hàm Logistic Regression** như một trường hợp đặc biệt $K = 2$. Hình 5b kiểm chứng bằng số: các điểm đỏ (softmax hai lớp, tính theo $d = s_1 - s_0$) nằm khít trên đường sigmoid. Ví dụ, với $\mathbf{s} = [0{,}2 \quad 1{,}7]$, softmax cho $p_1 = 0{,}8176$, đúng bằng $\sigma(1{,}7 - 0{,}2) = \sigma(1{,}5) = 0{,}8176$.



Hình 5. (a) Softmax của ba điểm số $[0{,}5;\ 1{,}5;\ 2{,}5]$ *nhân với các hệ số khác nhau: hệ số càng lớn thì xác suất càng dồn về lớp có điểm số cao nhất. (b) Với* $K = 2$*, softmax chính là sigmoid của hiệu hai điểm số.*

Đạo hàm của softmax

Để chuẩn bị cho mục 3 (tính gradient), ta tính đạo hàm của p_k theo điểm số s_j. Đặt $Z = \sum_j e^{s_j}$ thì $p_k = e^{s_k}/Z$ và $\partial Z / \partial s_j = e^{s_j}$. Theo quy tắc đạo hàm của thương:

$$\frac{\partial p_k}{\partial s_j} = \frac{\mathbb{1}[k = j]\, e^{s_k}\, Z - e^{s_k}\, e^{s_j}}{Z^2} = p_k\big(\mathbb{1}[k = j] - p_j\big)$$

Khi $k = j$ ta có $p_k(1 - p_k)$, và khi $k \neq j$ ta có $-p_k p_j$. Với $K = 2$, công thức này trở về $\sigma'(s) = \sigma(s)\big(1 - \sigma(s)\big)$ ở bài Logistic Regression (mục 2.3). Viết dưới dạng ma trận (ma trận Jacobian) cho vector xác suất $\mathbf{p}$: $\frac{\partial \mathbf{p}}{\partial \mathbf{s}} = \mathrm{diag}(\mathbf{p}) - \mathbf{p}^T\mathbf{p}$ (với $\mathbf{p}$ là vector hàng). Với $\mathbf{p} \approx [0{,}090 \quad 0{,}245 \quad 0{,}665]$:

$$\mathrm{diag}(\mathbf{p}) - \mathbf{p}^T\mathbf{p} \approx \begin{bmatrix} 0{,}082 & -0{,}022 & -0{,}060 \\ -0{,}022 & 0{,}185 & -0{,}163 \\ -0{,}060 & -0{,}163 & 0{,}223 \end{bmatrix}$$

Mỗi hàng và mỗi cột cộng lại bằng 0. Điều đó hợp lý: vì $\sum_k p_k = 1$ luôn đúng nên khi một điểm số thay đổi, các xác suất tăng và giảm **bù trừ** nhau. Đây là biểu hiện của sự **cạnh tranh** giữa các lớp.

2.4. Biểu diễn vector và ma trận của mô hình

Một mẫu

Với một mẫu, mô hình gồm hai bước: nhân vector hàng $\mathbf{x}_i$ với ma trận $\mathbf{W}$ để được K điểm số, rồi đưa qua softmax:

$$\mathbf{s}_i = \mathbf{x}_i\mathbf{W} \in \mathbb{R}^{1\times K}, \qquad \mathbf{p}_i = \text{softmax}(\mathbf{s}_i), \qquad \hat{y}_i = \underset{k}{\text{argmax}}\ p_{ik}$$

Về kích thước, $\mathbf{x}_i$ là $1 \times (d+1)$ và $\mathbf{W}$ là $(d+1) \times K$ nên $\mathbf{x}_i\mathbf{W}$ là một vector hàng $1 \times K$.

Toàn bộ n mẫu cùng lúc

Với toàn bộ dữ liệu, ta tính ma trận điểm số rồi áp dụng softmax **theo từng hàng**:

$$\mathbf{S} = \mathbf{X}\mathbf{W}, \qquad \mathbf{P} = \text{softmax}(\mathbf{S}) = \begin{bmatrix} \text{softmax}(\mathbf{s}_1) \\ \text{softmax}(\mathbf{s}_2) \\ \vdots \\ \text{softmax}(\mathbf{s}_n) \end{bmatrix}, \qquad \hat{y}_i = \underset{k}{\text{argmax}}\ p_{ik}$$

Mỗi hàng của $\mathbf{P}$ là một phân phối xác suất trên K lớp. Cách viết này có các lợi ích quen thuộc: công thức ngắn gọn, thống nhất với Logistic Regression (chỉ thay vector bằng ma trận và sigmoid bằng softmax), và khi lập trình ta tính cho cả tập dữ liệu bằng một phép nhân ma trận. Bảng kích thước:

Đại lượng	Biểu thức	Kích thước
Ma trận đặc trưng	$\mathbf{X}$	$n \times (d+1)$
Ma trận tham số	$\mathbf{W}$	$(d+1) \times K$
Ma trận điểm số	$\mathbf{S} = \mathbf{X}\mathbf{W}$	$n \times K$
Ma trận xác suất	$\mathbf{P} = \text{softmax}(\mathbf{S})$	$n \times K$
Ma trận nhãn one-hot	$\mathbf{Y}$	$n \times K$
Vector nhãn dự đoán	$\hat{\mathbf{y}}$	$n \times 1$

Ví dụ từng bước cho một sinh viên

Xét sinh viên 6 ở mục 1.2: học 6 giờ, vắng 4 buổi, $\mathbf{x}_6 = [1 \quad 6 \quad 4]$. Với $\mathbf{W}$ ở mục 2.2:

$$\mathbf{s}_6 = \mathbf{x}_6\mathbf{W} = [1 \quad 6 \quad 4]\begin{bmatrix} 1{,}5 & 0 & -6{,}5 \\ -1 & 0 & 1 \\ 1 & 0 & -1 \end{bmatrix} = [1{,}5 - 6 + 4 \quad 0 \quad -6{,}5 + 6 - 4]$$
$$= [-0{,}5 \quad 0 \quad -4{,}5]$$

Lấy mũ được $[0{,}6065 \quad 1 \quad 0{,}0111]$ với tổng $1{,}6176$, nên

$$\mathbf{p}_6 = [0{,}375 \quad 0{,}618 \quad 0{,}007], \qquad \hat{y}_6 = 1 \text{ (đạt)}$$

Mô hình dự đoán đúng (nhãn thật là đạt), với độ tin cậy vừa phải 61,8%: sinh viên này khá gần ranh giới với lớp rớt (37,5%) vì học 6 giờ nhưng vắng tới 4 buổi.

Ví dụ cho toàn bộ 9 sinh viên

Với **X** (có cột 1) và **W** ở trên, ta tính ma trận điểm số và ma trận xác suất cho cả 9 sinh viên:

```
X_sv = np.array([[2, 4], [3, 3], [4, 5], [6, 3], [7, 2], [6, 4], [9, 1], [10,
0], [8, 0]], dtype=float)
y_sv = np.array([0, 0, 0, 1, 1, 1, 2, 2, 2])
Xb_sv = them_bias(X_sv)
W = np.array([[ 1.5, 0.0, -6.5],
              [-1.0, 0.0,  1.0],
              [ 1.0, 0.0, -1.0]])                    # cột k là w_k
S = Xb_sv @ W
P = softmax(S)
print("S ="); print(S)
print("P ="); print(P.round(3))
print("y_hat =", P.argmax(axis=1), "| y =", y_sv)
```

```
S =
[[ 3.5  0.  -8.5]
 [ 1.5  0.  -6.5]
 [ 2.5  0.  -7.5]
 [-1.5  0.  -3.5]
 [-3.5  0.  -1.5]
 [-0.5  0.  -4.5]
 [-6.5  0.   1.5]
 [-8.5  0.   3.5]
 [-6.5  0.   1.5]]
P =
[[0.971 0.029 0.   ]
 [0.817 0.182 0.   ]
 [0.924 0.076 0.   ]
 [0.178 0.798 0.024]
 [0.024 0.798 0.178]
 [0.375 0.618 0.007]
 [0.    0.182 0.817]
 [0.    0.029 0.971]
 [0.    0.182 0.817]]
y_hat = [0 0 0 1 1 1 2 2 2] | y = [0 0 0 1 1 1 2 2 2]
```

Theo đúng ký hiệu toán:

$$\mathbf{S} = \mathbf{XW} = \begin{bmatrix} 3{,}5 & 0 & -8{,}5 \\ 1{,}5 & 0 & -6{,}5 \\ 2{,}5 & 0 & -7{,}5 \\ -1{,}5 & 0 & -3{,}5 \\ -3{,}5 & 0 & -1{,}5 \\ -0{,}5 & 0 & -4{,}5 \\ -6{,}5 & 0 & 1{,}5 \\ -8{,}5 & 0 & 3{,}5 \\ -6{,}5 & 0 & 1{,}5 \end{bmatrix} \Rightarrow \mathbf{P} = \text{softmax}(\mathbf{S}) \approx \begin{bmatrix} 0{,}971 & 0{,}029 & 0{,}000 \\ 0{,}817 & 0{,}182 & 0{,}000 \\ 0{,}924 & 0{,}076 & 0{,}000 \\ 0{,}178 & 0{,}798 & 0{,}024 \\ 0{,}024 & 0{,}798 & 0{,}178 \\ 0{,}375 & 0{,}618 & 0{,}007 \\ 0{,}000 & 0{,}182 & 0{,}817 \\ 0{,}000 & 0{,}029 & 0{,}971 \\ 0{,}000 & 0{,}182 & 0{,}817 \end{bmatrix}$$

Mỗi hàng của **P** cộng lại bằng 1. Nhãn dự đoán $\hat{\mathbf{y}} = [0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 1 \quad 2 \quad 2 \quad 2]^T$ trùng với **y** ở cả 9 mẫu. Nhưng cũng như ở Logistic Regression, mô hình còn cho thêm **độ tin cậy**: sinh viên 1 và 8 rất chắc chắn (0,971), còn sinh viên 6 chỉ 0,618.

Xác suất mà mô hình gán cho lớp thật

Ở bài Logistic Regression (mục 2.4), ta đã dùng đại lượng q_i, xác suất mô hình gán cho **nhãn thật** của mẫu i. Với nhiều lớp, nhờ ma trận one-hot **Y**, đại lượng đó được lấy bằng cách "chọn" đúng phần tử của **P** ở vị trí lớp thật:

$$q_i = p_{i,y_i} = \sum_{k=0}^{K-1} y_{ik}\, p_{ik} \quad \Leftrightarrow \quad \mathbf{q} = (\mathbf{Y} \odot \mathbf{P})\,\mathbf{1}$$

trong đó $\odot$ là phép nhân từng phần tử và $\mathbf{1}$ là vector cột toàn số 1 (cộng theo từng hàng). Với ví dụ:

$$\mathbf{q} \approx [0{,}971 \quad 0{,}817 \quad 0{,}924 \quad 0{,}798 \quad 0{,}798 \quad 0{,}618 \quad 0{,}817 \quad 0{,}971 \quad 0{,}817]^T$$

Nếu coi các mẫu độc lập, xác suất mô hình gán cho **toàn bộ bộ nhãn quan sát được** là tích các q_i:

$$\prod_{i=1}^{9} q_i \approx 0{,}187, \qquad -\frac{1}{9}\sum_{i=1}^{9} \ln q_i \approx 0{,}186$$

Tích này càng gần 1 (hay trung bình $-\ln q_i$ càng gần 0) thì mô hình càng phù hợp với dữ liệu. Đây chính là điểm xuất phát của **nguyên lý hợp lý cực đại** ở mục 3, hoàn toàn song song với bài Logistic Regression, và con số 0,186 sẽ trở thành giá trị của hàm mất mát cross-entropy nhiều lớp của bộ **W** này.

2.5. Vùng quyết định, ý nghĩa của trọng số và tính dư thừa tham số

Vùng quyết định

Mô hình dự đoán lớp k khi s_k lớn nhất. Ranh giới giữa hai lớp k và l là nơi hai điểm số **bằng nhau** (cũng là nơi $p_k = p_l$):

$$s_k = s_l \quad \Leftrightarrow \quad \mathbf{x}(\mathbf{w}_k - \mathbf{w}_l) = 0$$

Đây là một **siêu phẳng** (đường thẳng khi $d = 2$) với vector pháp tuyến là phần của $\mathbf{w}_k - \mathbf{w}_l$ ứng với các đặc trưng (không tính bias). Vùng của lớp k là phần không gian mà lớp k **thắng mọi lớp còn lại**:

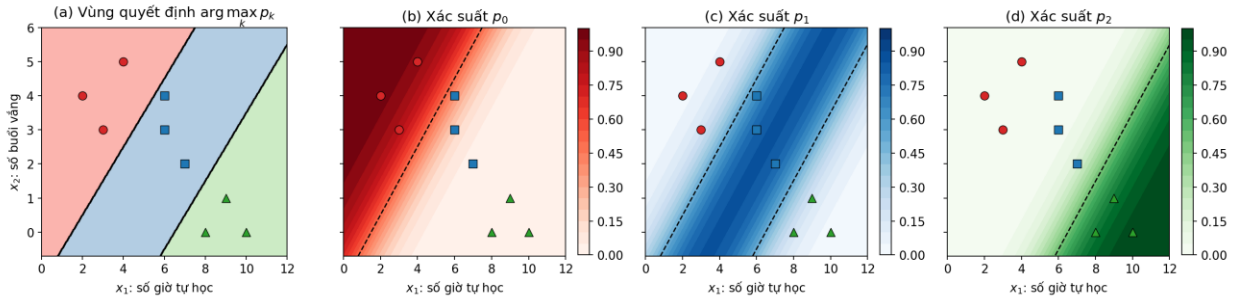
$$\mathcal{R}_k = \{\mathbf{x}: \mathbf{x}(\mathbf{w}_k - \mathbf{w}_l) \geq 0 \text{ với mọi } l \neq k\}$$

Đó là **giao của** $K-1$ **nửa không gian**, nên mỗi vùng là một miền **lồi** có biên là các đoạn thẳng. Với $K = 3$ và $d = 2$, ba vùng thường gặp nhau như ba "múi" quanh một điểm chung. Hình 6a vẽ các vùng quyết định của bộ **W** ở mục 2.2: các ranh giới là

- giữa lớp rớt và lớp đạt: $s_0 = s_1 \Leftrightarrow 1{,}5 - x_1 + x_2 = 0 \Leftrightarrow x_1 - x_2 = 1{,}5$;

- giữa lớp đạt và lớp giỏi: $s_1 = s_2 \Leftrightarrow -6{,}5 + x_1 - x_2 = 0 \Leftrightarrow x_1 - x_2 = 6{,}5$.

Ranh giới giữa lớp rớt và lớp giỏi ($x_1 - x_2 = 4$) nằm hoàn toàn bên trong vùng của lớp đạt nên không xuất hiện trên bản đồ. Vì bộ **W** này đặt ba điểm số cùng phụ thuộc vào $x_1 - x_2$, các ranh giới song song và ba vùng là ba dải song song; với **W** tổng quát, chúng gặp nhau ở một điểm.



*Hình 6. Mô hình Softmax Regression với ma trận **W** ở mục 2.2. (a) Ba vùng quyết định (rớt: đỏ, đạt: xanh dương, giỏi: xanh lá) ngăn cách bởi hai đường thẳng. (b), (c), (d) Bản đồ xác suất p_0, p_1, p_2 của từng lớp; đường đứt là $p_k = 0{,}5$. Các điểm là 9 sinh viên ở mục 1.2.*

Hình 6 cũng cho thấy **lý do Softmax không bị hiện tượng che khuất** của one-vs-rest (mục 1.3). Vùng của lớp đạt (ở giữa) là dải nằm giữa **hai** ranh giới, được tạo ra bởi hai phép so sánh riêng rẽ: đạt thắng rớt và đạt thắng giỏi. Lớp đạt **không cần** tách tuyến tính khỏi toàn bộ phần còn lại bằng **một** đường thẳng như ở one-vs-rest, vì điều đó là bất khả thi với một lớp nằm giữa. Chỉ cần so sánh theo từng cặp điểm số là đủ. Ở Hình 6c, xác suất p_1 của lớp đạt cao nhất đúng ở dải giữa và giảm về hai phía.

Ý nghĩa của trọng số: odds ratio giữa hai lớp

Ở Logistic Regression, e^{w_j} là hệ số nhân của odds khi x_j tăng một đơn vị (mục 2.5 bài Logistic). Với nhiều lớp, "odds" luôn được so giữa **hai lớp**. Từ $\ln \frac{p_k}{p_l} = \mathbf{x}(\mathbf{w}_k - \mathbf{w}_l)$, khi x_j tăng thêm 1 đơn vị (các đặc trưng khác giữ nguyên), tỉ số p_k/p_l được nhân với

$$e^{w_{jk} - w_{jl}}$$

Như vậy, điều có ý nghĩa là **hiệu** giữa các trọng số của hai lớp, không phải từng trọng số riêng lẻ. Với **W** ở mục 2.2:

- Mỗi giờ học thêm nhân tỉ số $p_{\text{giỏi}}/p_{\text{đạt}}$ với $e^{w_{12} - w_{11}} = e^{1-0} = e \approx 2{,}718$, và nhân tỉ số $p_{\text{đạt}}/p_{\text{rớt}}$ với $e^{w_{11} - w_{10}} = e^{0-(-1)} = e \approx 2{,}718$.
- Mỗi buổi vắng thêm nhân các tỉ số đó với $e^{-1} \approx 0{,}368$.

Kiểm chứng với sinh viên 6 ($\mathbf{s}_6 = [-0{,}5 \quad 0 \quad -4{,}5]$): $p_{\text{giỏi}}/p_{\text{đạt}} = e^{-4{,}5-0} \approx 0{,}0111$. Nếu học thêm một giờ ($x_1 = 7$) thì $\mathbf{s} = [-1{,}5 \quad 0 \quad -3{,}5]$ và $p_{\text{giỏi}}/p_{\text{đạt}} = e^{-3{,}5} \approx 0{,}0302 = 0{,}0111 \times 2{,}718$, đúng như dự đoán.

Tính dư thừa tham số

Có một điều lạ ở ví dụ mục 2.2: một cột của $\mathbf{W}$ ($\mathbf{w}_1$) bằng toàn bộ 0, mà mô hình vẫn phân loại đầy đủ ba lớp. Lý do là **tính bất biến khi dịch** của softmax (mục 2.3). Giả sử ta lấy một vector cột bất kỳ $\mathbf{v}$ kích thước $(d+1)\times 1$ và **cộng nó vào mọi cột** của $\mathbf{W}$:

$$\mathbf{W}' = \mathbf{W} + \mathbf{v}\mathbf{1}^T \quad \Leftrightarrow \quad \mathbf{w}_k{}' = \mathbf{w}_k + \mathbf{v} \quad \text{với mọi } k$$

Khi đó điểm số của mỗi mẫu bị dịch cùng một lượng $\mathbf{x}_i\mathbf{v}$ cho **tất cả** K lớp: $s_{ik}{}' = s_{ik} + \mathbf{x}_i\mathbf{v}$. Theo tính bất biến khi dịch, softmax cho cùng các xác suất:

$$\text{softmax}(\mathbf{X}\mathbf{W}') = \text{softmax}(\mathbf{X}\mathbf{W})$$

Hai ma trận $\mathbf{W}$ và $\mathbf{W}'$ khác nhau nhưng định nghĩa **cùng một mô hình**. Ta kiểm chứng với $\mathbf{v} = [-1{,}5 \quad 1 \quad -1]^T$:

```
v = np.array([[-1.5], [1.0], [-1.0]])
W2 = W + v                                          # cộng cùng một vector v
vào mọi cột
print("W' ="); print(W2)
print("max |P' - P| =", np.abs(softmax(Xb_sv @ W2) - P).max())
```

```
W' =
[[ 0.  -1.5 -8. ]
 [ 0.   1.   2. ]
 [ 0.  -1.  -2. ]]
max |P' - P| = 0.0
```

Bộ $\mathbf{W}'$ có cột đầu bằng $\mathbf{0}$ nhưng cho **đúng cùng ma trận xác suất P** (sai khác tối đa bằng 0). Điều này cho thấy:

- **Chỉ $K-1$ cột là độc lập thật sự.** Ta có thể luôn chọn $\mathbf{v}$ sao cho một **lớp tham chiếu** (giả sử lớp 0) có cột bằng $\mathbf{0}$, bằng cách lấy $\mathbf{v} = -\mathbf{w}_0$. Số tham số **độc lập** là $(K-1)(d+1)$, thay vì $K(d+1)$. Với ví dụ ($d=2$, $K=3$), đó là 6 chứ không phải 9. Với $K=2$ chỉ còn $d+1$ tham số, **đúng bằng** Logistic Regression.
- **Dạng "lớp tham chiếu".** Khi $\mathbf{w}_0 = \mathbf{0}$ (như $\mathbf{W}'$ ở trên), điểm số $s_0 = 0$ và mỗi cột còn lại có ý nghĩa rất sáng sủa: $\ln\frac{p_k}{p_0} = \mathbf{x}\mathbf{w}_k{}'$, tức **log-odds của lớp k so với lớp tham chiếu** là hàm tuyến tính của đặc trưng. Với $K=2$ đó chính là $\ln\frac{p_1}{1-p_1} = \mathbf{x}\mathbf{w}_1{}'$, giả thiết của Logistic Regression. Ví dụ, ở $\mathbf{W}'$, sinh viên 6 có $\ln\frac{p_{\text{đạt}}}{p_{\text{rớt}}} = \mathbf{x}_6\mathbf{w}_1{}' = -1{,}5 + 6 - 4 = 0{,}5$, và thật vậy $\ln\frac{0{,}618}{0{,}375} \approx 0{,}5$.
- **Không thể đọc riêng từng cột.** Vì $\mathbf{W}$ chỉ được xác định đến một phép dịch, giá trị của một cột $\mathbf{w}_k$ riêng lẻ **không có ý nghĩa tuyệt đối**. Chỉ có các **hiệu** $\mathbf{w}_k - \mathbf{w}_l$ mới có ý nghĩa (ta đã thấy ở trên, đó là thứ quyết định odds ratio và đường biên). Hiện tượng này gợi lại việc không thể diễn giải riêng lẻ hai trọng số của hai đặc trưng gần trùng nhau ở mục 6.1 bài Logistic Regression.
- **Hệ quả cho huấn luyện** (mục 3 và 4.3). Vì có cả một "họ" các ma trận $\mathbf{W}$ cho cùng một hàm mất mát, hàm mất mát **không có nghiệm duy nhất**: nếu $\mathbf{W}^*$ tối ưu thì mọi $\mathbf{W}^* +$

$\mathbf{v1}^T$ cũng tối ưu. Điều chuẩn hóa L2 (mục 4.4) giải quyết bằng cách chọn trong họ đó ma trận có độ lớn nhỏ nhất, tức là ma trận mà **trung bình của các cột bằng 0** (xét trên các hàng được phạt). Các phần mềm thống kê thường cố định luôn một cột bằng $\mathbf{0}$, còn các thư viện học máy thường giữ cả K cột và dựa vào điều chuẩn hóa.

Tóm tắt mục 2

- Softmax Regression tính K **điểm số tuyến tính** $s_k = \mathbf{x}\mathbf{w}_k$ (mỗi lớp một vector tham số riêng, xếp thành các cột của $\mathbf{W}$, kích thước $(d+1) \times K$), rồi dùng **softmax** biến chúng thành K xác suất cộng lại bằng 1: $p_k = \frac{e^{s_k}}{\sum_j e^{s_j}}$. Nhãn dự đoán là $\hat{y} = \text{argmax}_k p_k = \text{argmax}_k s_k$.
- Có thể nhớ gọn: **Softmax Regression** = K **hồi quy tuyến tính chạy song song** + **softmax**. Với toàn bộ dữ liệu: $\mathbf{S} = \mathbf{X}\mathbf{W}$ và $\mathbf{P} = \text{softmax}(\mathbf{S})$ theo từng hàng.
- Softmax **bất biến khi dịch** (chỉ phụ thuộc hiệu các điểm số, dùng để tính ổn định số học bằng cách trừ giá trị lớn nhất) và có $\frac{p_k}{p_l} = e^{s_k - s_l}$. Giả thiết của mô hình: log-odds giữa hai lớp bất kỳ là hàm tuyến tính của đặc trưng. Đạo hàm: $\frac{\partial p_k}{\partial s_j} = p_k\big(\mathbb{1}[k=j] - p_j\big)$.
- Khi $K = 2$, softmax chính là sigmoid: $p_1 = \sigma\big(\mathbf{x}(\mathbf{w}_1 - \mathbf{w}_0)\big)$. **Softmax Regression bao hàm Logistic Regression.**
- **Vùng quyết định** của mỗi lớp là giao của $K-1$ nửa không gian $\mathbf{x}(\mathbf{w}_k - \mathbf{w}_l) \geq 0$, nên là miền lồi, với ranh giới là các siêu phẳng. Lớp nằm giữa được "kẹp" bởi hai ranh giới, nên không bị che khuất như ở one-vs-rest.
- Ý nghĩa của trọng số nằm ở **hiệu** $w_{jk} - w_{jl}$: nhân tỉ số p_k/p_l với $e^{w_{jk}-w_{jl}}$ khi x_j tăng một đơn vị.
- **Dư thừa tham số:** cộng cùng một vector $\mathbf{v}$ vào mọi cột của $\mathbf{W}$ không đổi mô hình. Chỉ có $(K-1)(d+1)$ tham số độc lập; chọn một lớp tham chiếu có cột bằng $\mathbf{0}$ thì mỗi cột còn lại là log-odds so với lớp đó. Hệ quả: hàm mất mát sẽ không có nghiệm duy nhất (mục 4.3).

Câu hỏi kiểm tra nhanh

1. Với $\mathbf{s} = [2 \quad -1 \quad 0{,}5]$, tính softmax. Lớp nào được dự đoán? Kiểm tra rằng $\ln\frac{p_0}{p_2} = s_0 - s_2$.
2. Cộng 3 vào cả ba điểm số ở câu 1 thì các xác suất thay đổi thế nào? Giải thích.
3. Với $K = 2$ và $\mathbf{s} = [1 \quad 3]$, tính p_1 bằng softmax và bằng sigmoid. Nếu ta muốn dùng Logistic Regression thay cho softmax hai lớp, vector tham số $\mathbf{w}$ được tính từ $\mathbf{w}_0$ và $\mathbf{w}_1$ như thế nào?
4. Một mô hình có $K = 4$ lớp và $d = 5$ đặc trưng. Ma trận $\mathbf{W}$ có kích thước bao nhiêu? Có bao nhiêu tham số tổng cộng, và có bao nhiêu tham số độc lập?
5. Cho $d = 1$, $K = 3$ với $\mathbf{w}_0 = [0 \quad 0]^T$, $\mathbf{w}_1 = [-1 \quad 1]^T$ và $\mathbf{w}_2 = [-3 \quad 2]^T$ (phần tử đầu là hệ số chệch). Tính các xác suất tại $x = 2$. Điểm $x = 2$ nằm trên ranh giới của hai lớp nào?
6. Với $\mathbf{W}$ ở mục 2.2, khi số giờ học tăng thêm 2 giờ (các đặc trưng khác giữ nguyên), tỉ số $p_{\text{giỏi}}/p_{\text{đạt}}$ được nhân với bao nhiêu?

7. Chứng minh rằng nếu cộng cùng một vector $\mathbf{v}$ vào mọi cột của $\mathbf{W}$ thì ma trận xác suất $\mathbf{P}$ không đổi.
8. Vì sao one-vs-rest không "kẹp" được lớp ở giữa còn Softmax thì làm được? Trả lời bằng cách mô tả vùng của một lớp trong từng mô hình.

Gợi ý đáp án

1. $e^2 = 7{,}389$, $e^{-1} = 0{,}368$, $e^{0{,}5} = 1{,}649$; tổng $= 9{,}406$; $\mathbf{p} \approx [0{,}786 \quad 0{,}039 \quad 0{,}175]$. Dự đoán lớp 0 (xác suất lớn nhất, cũng là điểm số lớn nhất). Kiểm tra: $\ln\frac{0{,}786}{0{,}175} \approx 1{,}5 = 2 - 0{,}5$.
2. Các xác suất **không đổi**. Cộng cùng hằng số $c = 3$ vào mọi điểm số nhân cả tử và mẫu của softmax với e^3, hai thừa số này triệt tiêu: softmax chỉ phụ thuộc vào hiệu các điểm số.
3. Softmax: $p_1 = \frac{e^3}{e^1+e^3} = \frac{20{,}086}{2{,}718+20{,}086} \approx 0{,}881$. Sigmoid: $\sigma(3-1) = \sigma(2) \approx 0{,}881$. Hai kết quả trùng nhau. Khi dùng Logistic Regression, $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_0$.
4. $\mathbf{W}$ có kích thước $(d+1) \times K = 6 \times 4$. Tổng số tham số là 24. Số tham số độc lập là $(K-1)(d+1) = 3 \times 6 = 18$.
5. Tại $x = 2$ (vector $\mathbf{x} = [1 \quad 2]$): $s_0 = 0$, $s_1 = -1 + 2 = 1$, $s_2 = -3 + 4 = 1$. Lấy mũ được 1; 2,718; 2,718, tổng 6,437, nên $\mathbf{p} \approx [0{,}155 \quad 0{,}422 \quad 0{,}422]$. Hai lớp 1 và 2 có xác suất bằng nhau nên $x = 2$ nằm đúng trên ranh giới giữa lớp 1 và lớp 2 (kiểm tra: $\mathbf{w}_1 - \mathbf{w}_2 = [2 \quad -1]^T$ và $2 - 1 \cdot 2 = 0$).
6. Mỗi giờ học thêm nhân tỉ số với $e^{w_{12}-w_{11}} = e^1 = 2{,}718$. Hai giờ học thêm nhân với $e^2 \approx 7{,}389$ (bằng $2{,}718^2$, vì các tác động nhân lên nhau).
7. Điểm số mới là $s_{ik}{}' = \mathbf{x}_i(\mathbf{w}_k + \mathbf{v}) = s_{ik} + \mathbf{x}_i\mathbf{v}$, tức mọi điểm số của mẫu i bị cộng cùng hằng số $c_i = \mathbf{x}_i\mathbf{v}$. Do đó $p_{ik}{}' = \frac{e^{s_{ik}+c_i}}{\sum_j e^{s_{ij}+c_i}} = \frac{e^{c_i}e^{s_{ik}}}{e^{c_i}\sum_j e^{s_{ij}}} = p_{ik}$.
8. Với one-vs-rest, vùng của lớp k là vùng mà **một** đường thẳng $\mathbf{x}\mathbf{w}_k = 0$ cho là "có", tức một **nửa không gian**: lớp nằm giữa hai lớp khác không thể là một nửa không gian vì sẽ bị nằm cùng phía với một trong hai lớp ở hai bên. Với Softmax, vùng của lớp k là giao của $K-1$ nửa không gian $\mathbf{x}(\mathbf{w}_k - \mathbf{w}_l) \geq 0$, nên lớp giữa có thể là một **dải** nằm giữa hai ranh giới.

3. Huấn luyện mô hình Softmax Regression

Ở mục 2, ta đã có mô hình $\mathbf{P} = \text{softmax}(\mathbf{XW})$: mỗi lớp có một cột tham số, và mỗi mẫu nhận một phân phối xác suất trên K lớp. Nhưng ma trận $\mathbf{W}$ vẫn do ta chọn tay (ví dụ ma trận ở mục 2.2). Mục này để máy tự tìm $\mathbf{W}$ từ dữ liệu. Ta tiếp tục bám theo khung đã dùng ở hồi quy tuyến tính, Perceptron và Logistic Regression:

$$\text{Mô hình} + \text{Hàm mất mát} + \text{Thuật toán tối ưu} = \text{Huấn luyện}$$

Mô hình đã có (mục 2). Mục này làm hai thành phần còn lại và trả lời ba câu hỏi:

- Hàm mất mát nào đo tốt sự phù hợp giữa K xác suất dự đoán và nhãn thật, và nó có cơ sở xác suất ra sao?
- Gradient của hàm mất mát theo ma trận $\mathbf{W}$ là gì? Quy tắc cập nhật có còn gọn như ở Logistic Regression không?
- Thuật toán chạy ra sao, và các lựa chọn như tốc độ học, chuẩn hóa đặc trưng hay cách chia dữ liệu ảnh hưởng thế nào?

Ta dùng quy ước ký hiệu cũ: chữ thường in đậm là vector, chữ hoa in đậm là ma trận, vector đặc trưng $\mathbf{x}_i$ là vector hàng, $\mathbf{W} = [\mathbf{w}_0\ \mathbf{w}_1\ \cdots\ \mathbf{w}_{K-1}]$ có các cột là vector tham số của từng lớp, $\mathbf{Y}$ là ma trận one-hot, $\mathbf{S} = \mathbf{XW}$ và $\mathbf{P} = \text{softmax}(\mathbf{S})$ theo từng hàng. Với mẫu thứ i, ta ký hiệu thêm $\mathbf{s}_i = \mathbf{x}_i\mathbf{W}$, $\mathbf{p}_i$ và $\mathbf{y}_i$ là các **vector hàng** (hàng thứ i của $\mathbf{S}$, $\mathbf{P}$ và $\mathbf{Y}$), và $\mathbf{1}_K$ là vector cột gồm K số 1.

3.1. Từ cross-entropy nhị phân đến cross-entropy nhiều lớp

Vì sao không dùng lại MSE?

Ở Logistic Regression, ta đã thấy MSE kết hợp với sigmoid gây "gradient tắt" ở đúng những mẫu bị sai chắc chắn. Với softmax, vấn đề còn nguyên. Ta có thể thử đo sai số giữa vector xác suất $\mathbf{p}_i$ và vector one-hot $\mathbf{y}_i$ bằng bình phương khoảng cách:

$$\ell_i^{\text{MSE}} = \| \mathbf{p}_i - \mathbf{y}_i \|^2 = \sum_{k=0}^{K-1} (p_{ik} - y_{ik})^2$$

Theo quy tắc đạo hàm hàm hợp và Jacobian của softmax ở mục 2.3 ($\partial \mathbf{p} / \partial \mathbf{s} = \text{diag}(\mathbf{p}) - \mathbf{p}^T\mathbf{p}$), gradient theo vector điểm số là

$$\frac{\partial \ell_i^{\text{MSE}}}{\partial \mathbf{s}_i} = 2\,(\mathbf{p}_i - \mathbf{y}_i)(\text{diag}(\mathbf{p}_i) - \mathbf{p}_i^T\mathbf{p}_i)$$

Ma trận Jacobian chứa các tích $p_{ik}(1 - p_{ik})$ và $p_{ik}p_{ij}$, nên khi $\mathbf{p}_i$ gần một phân phối "chắc chắn" thì gradient lại gần 0. Xét một mẫu thuộc lớp 2 ($\mathbf{y} = [0\ 0\ 1]$) nhưng mô hình cho điểm số $\mathbf{s} = [4\ \ 0\ \ -2]$, tức mô hình tin chắc là lớp 0:

$$\mathbf{p} \approx [0{,}980 \;\; 0{,}018 \;\; 0{,}002]$$

Khi đó $\| \mathbf{p} - \mathbf{y} \| \approx 1{,}398$ (sai rất nặng), nhưng độ lớn gradient của MSE theo điểm số chỉ khoảng $0{,}056$. Thuật toán gần như không học được từ chính mẫu mà nó sai nặng nhất. Hình 7b so sánh với cross-entropy (sẽ xây dựng ngay bên dưới), có gradient $\mathbf{p} - \mathbf{y}$ lớn đúng ở vùng này.

Hàm mất mát ta cần vẫn là hàm đã đặt ra ở Logistic Regression: phạt nặng khi sai chắc chắn, gradient lớn khi sai nặng, trơn, lồi và có cơ sở xác suất. Hàm đó là cross-entropy, và ta mở rộng nó từ hai lớp lên K lớp.

Từ hai lớp lên K lớp

Với hai lớp, mất mát của một mẫu là $-[y_i \ln p_i + (1 - y_i)\ln(1 - p_i)]$. Dùng mã hóa one-hot ở mục 1.2, hàng của $\mathbf{Y}$ là $[y_{i0} \;\; y_{i1}] = [1 - y_i \;\; y_i]$, còn hàng của $\mathbf{P}$ là $[p_{i0} \;\; p_{i1}] = [1 - p_i \;\; p_i]$. Khi đó mất mát nhị phân viết được lại thành một tổng đối xứng giữa hai lớp:

$$\ell_i = -[y_{i1} \ln p_{i1} + y_{i0} \ln p_{i0}] = -\sum_{k=0}^{1} y_{ik} \ln p_{ik}$$

Cách viết này không còn phụ thuộc "lớp 1 là lớp đặc biệt", nên mở rộng ra K lớp chỉ là cộng thêm các số hạng:

$$\ell_i = -\sum_{k=0}^{K-1} y_{ik} \ln p_{ik}$$

Vì $\mathbf{y}_i$ là one-hot, chỉ có đúng một số hạng khác 0, ứng với lớp thật $c = y_i$. Vậy mất mát một mẫu rút gọn thành

$$\ell_i = -\ln p_{ic} = -\ln q_i$$

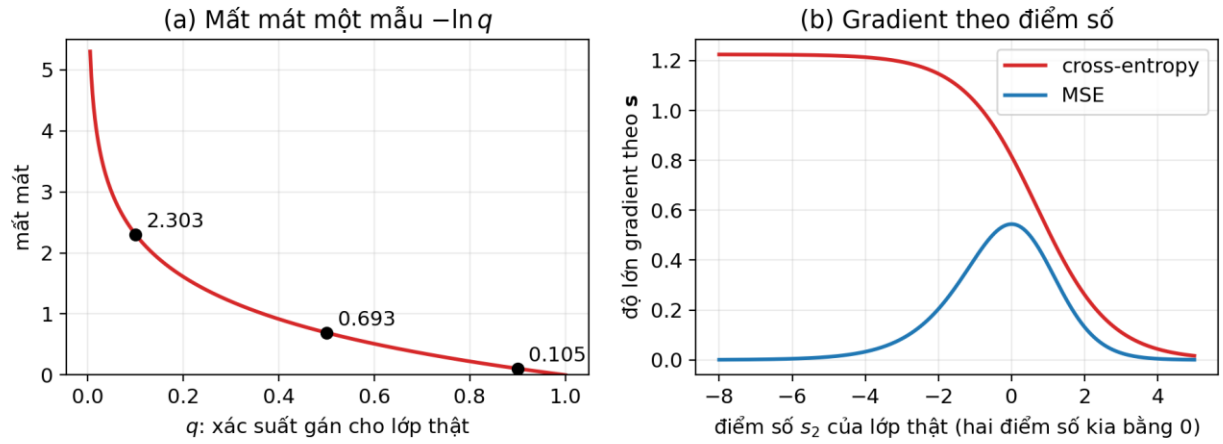
trong đó $q_i = p_{i,y_i}$ chính là xác suất mô hình gán cho **lớp thật** mà ta đã gặp ở mục 2.4. Giống hệt trường hợp nhị phân: $-\ln q$ là mức "ngạc nhiên" của mô hình khi lớp thật xảy ra. Bảng $q \to -\ln q$ ở bài Logistic Regression vẫn dùng được nguyên vẹn: $q = 0{,}99$ cho $0{,}010$; $q = 0{,}5$ cho $0{,}693$; $q = 0{,}01$ cho $4{,}605$. Khi $q \to 0$ mất mát tiến tới $+\infty$ (Hình 7a).

Lấy trung bình trên n mẫu, ta được **hàm mất mát cross-entropy nhiều lớp**:

$$J(\mathbf{W}) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{K-1} y_{ik} \ln p_{ik}, \qquad \mathbf{P} = \text{softmax}(\mathbf{XW})$$

Viết bằng ma trận, với $\ln\mathbf{P}$ lấy logarit từng phần tử và $\odot$ là phép nhân từng phần tử:

$$J(\mathbf{W}) = -\frac{1}{n} \, \mathbf{1}_n^T (\mathbf{Y} \odot \ln\mathbf{P}) \mathbf{1}_K$$



Hình 7. (a) Mất mát một mẫu $-\ln q$ theo xác suất q gán cho lớp thật: tiến tới vô cùng khi $q \to 0$. (b) Với mẫu lớp 2 có điểm số $[0\ 0\ s_2]$, độ lớn gradient theo điểm số: cross-entropy lớn khi mô hình sai nặng (s_2 rất âm), còn gradient của MSE tiến về 0 ở đúng vùng đó.

Ba kiểm tra quen thuộc

Thứ nhất, **với $K = 2$ ta trở về đúng cross-entropy nhị phân**, vì $y_{i1} = y_i$, $y_{i0} = 1 - y_i$ (chính là cách ta vừa xây dựng). Thứ hai, **giá trị khởi tạo**: khi $\mathbf{W} = \mathbf{0}$, mọi điểm số bằng 0 nên mọi $p_{ik} = 1/K$, và mỗi mẫu bị phạt đúng $-\ln(1/K) = \ln K$:

$$J(\mathbf{0}) = \ln K$$

Với $K = 3$ ta có $J(\mathbf{0}) = \ln 3 \approx 1{,}0986$ (tổng quát hóa của $\ln 2 \approx 0{,}693$ ở Logistic Regression), và đây là mốc để kiểm tra cài đặt ở mục 3.7. Thứ ba, ở mục 2.4 ta đã tính được với ma trận $\mathbf{W}$ chọn tay trên 9 sinh viên: trung bình $-\ln q_i \approx 0{,}186$. Đó chính là $J(\mathbf{W})$ của ma trận đó. Bảng sau cho mất mát từng sinh viên (nhắc lại q_i từ mục 2.4):

Sinh viên	1	2	3	4	5	6	7	8	9
q_i	0,971	0,817	0,924	0,798	0,798	0,618	0,817	0,971	0,817
$\ell_i = -\ln q_i$	0,030	0,202	0,079	0,226	0,226	0,481	0,202	0,030	0,202

Trung bình của dòng cuối là 0,186. Sinh viên 6 (học 6 giờ, vắng 4 buổi, gần ranh giới lớp rớt) bị phạt nặng nhất (0,481), còn sinh viên 1 và 8 (mô hình rất chắc chắn và đúng) gần như không bị phạt.

Dạng log-sum-exp (dùng khi lập trình)

Vì $\ln p_{ik} = s_{ik} - \ln \sum_j e^{s_{ij}}$, mất mát của mẫu i có lớp thật c cũng viết được là

$$\ell_i = -s_{ic} + \ln \sum_{j=0}^{K-1} e^{s_{ij}}$$

Cách viết này không cần tính **P** rồi lấy ln (dễ gặp ln0), và hàm $\ln\sum e^{s}$ có thể tính ổn định bằng cách trừ giá trị lớn nhất, giống mẹo ở mục 2.3. Ta sẽ dùng nó ở mục 3.7.

3.2. Hợp lý cực đại cho phân phối nhiều lớp

Mô hình xác suất của một nhãn

Cũng như ở Logistic Regression, ta chọn **W** sao cho dữ liệu quan sát được "có khả năng xảy ra nhất". Với hai lớp, nhãn là một biến Bernoulli. Với K lớp, nhãn là một biến **phân phối loại** (categorical): nhận giá trị k với xác suất p_{ik}. Dùng one-hot, xác suất mô hình gán cho nhãn quan sát được của mẫu i viết gọn là

$$q_i = P(y_i \mid \mathbf{x}_i) = \prod_{k=0}^{K-1} p_{ik}^{y_{ik}}$$

Vì $\mathbf{y}_i$ one-hot, tích này chỉ còn một thừa số khác 1, đó là p_{ic} của lớp thật. Với $K = 2$, nó trở về $p_i^{y_i}(1-p_i)^{1-y_i}$ ở bài Logistic Regression.

Hàm hợp lý và log-hợp lý

Coi các mẫu độc lập, xác suất của toàn bộ bộ nhãn là tích của các q_i, gọi là **hàm hợp lý**:

$$L(\mathbf{W}) = \prod_{i=1}^{n}\prod_{k=0}^{K-1} p_{ik}^{y_{ik}}, \qquad p_{ik} = \frac{e^{\mathbf{x}_i\mathbf{w}_k}}{\sum_j e^{\mathbf{x}_i\mathbf{w}_j}}$$

Lấy logarit (tích thành tổng, và ln là hàm tăng nên không đổi vị trí cực đại):

$$\ln L(\mathbf{W}) = \sum_{i=1}^{n}\sum_{k=0}^{K-1} y_{ik}\ln p_{ik}$$

Đổi dấu và lấy trung bình trên n mẫu, ta đúng bằng hàm cross-entropy ở mục 3.1:

$$\max_{\mathbf{W}} L(\mathbf{W}) \Leftrightarrow \max_{\mathbf{W}} \ln L(\mathbf{W}) \Leftrightarrow \min_{\mathbf{W}} J(\mathbf{W}), \qquad J(\mathbf{W}) = -\frac{1}{n}\ln L(\mathbf{W})$$

Như vậy cross-entropy nhiều lớp không phải lựa chọn tùy tiện: nó chính là **âm log-hợp lý trung bình** của mô hình phân phối loại.

So sánh vài bộ tham số trên 9 sinh viên

Ta tính L, $\ln L$ và J cho một vài ma trận **W** trên 9 sinh viên ở mục 1.2. Ký hiệu $\mathbf{W}_{\text{tay}}$ là ma trận chọn tay ở mục 2.2 (các điểm số $s_0 = 1{,}5 - x_1 + x_2$, $s_1 = 0$, $s_2 = -6{,}5 + x_1 - x_2$):

Bộ tham số	$L(\mathbf{W})$	$\ln L(\mathbf{W})$	$J(\mathbf{W})$
$\mathbf{W} = \mathbf{0}$ (mọi $p_{ik} = 1/3$)	$3^{-9} \approx 5{,}1 \cdot 10^{-5}$	$-9{,}888$	$1{,}099$
$\mathbf{W}_{\text{tay}}$ nhưng bỏ qua số buổi vắng (hàng của x_2 bằng $\mathbf{0}$)	$\approx 5{,}0 \cdot 10^{-4}$	$-7{,}607$	$0{,}845$

Bộ tham số	$L(\mathbf{W})$	$\ln L(\mathbf{W})$	$J(\mathbf{W})$
$\mathbf{W}_{\text{tay}}$ nhưng bỏ qua số giờ học (hàng của x_1 bằng $\mathbf{0}$)	$\approx 4{,}1 \cdot 10^{-18}$	$-40{,}04$	$4{,}449$
$\mathbf{W}_{\text{tay}}$ với hai cột $\mathbf{w}_0$, $\mathbf{w}_2$ đổi chỗ	$\approx 1{,}2 \cdot 10^{-26}$	$-59{,}68$	$6{,}631$
$\mathbf{W}_{\text{tay}}$	$\approx 0{,}187$	$-1{,}676$	$0{,}186$
$2\,\mathbf{W}_{\text{tay}}$	$\approx 0{,}567$	$-0{,}567$	$0{,}063$

Có ba nhận xét. Thứ nhất, $\mathbf{W}_{\text{tay}}$ "dễ xảy ra" hơn hẳn $\mathbf{W} = \mathbf{0}$ (hợp lý hơn khoảng 3700 lần), và bỏ bớt đặc trưng làm mất mát tăng, nhất là bỏ số giờ học. Thứ hai, đổi chỗ hai cột làm mô hình gán xác suất cao cho lớp sai nên J tăng vọt. Thứ ba, $2\,\mathbf{W}_{\text{tay}}$ có J **nhỏ hơn** $\mathbf{W}_{\text{tay}}$: do 9 sinh viên này phân loại đúng hoàn toàn, nhân $\mathbf{W}$ lên làm xác suất của lớp đúng càng gần 1. Đây là hiện tượng dữ liệu phân tách hoàn hảo mà ta sẽ phân tích ở mục 4.4.

Mất mát không phụ thuộc cách đặt tên lớp

Ở mục 1.1, hồi quy trên nhãn số cho độ chính xác 96,7% hoặc 34,7% tùy cách đặt số cho lớp. Với cross-entropy, điều đó không xảy ra. Giả sử ta đánh số lại các lớp theo một hoán vị π, tức hoán vị đồng thời các cột của $\mathbf{Y}$ và các cột của $\mathbf{W}$. Điểm số của mỗi lớp không đổi, chỉ đổi vị trí, nên các cột của $\mathbf{P}$ cũng hoán vị theo, và q_i không đổi. Do đó J không đổi: **hàm mất mát chỉ quan tâm đến xác suất gán cho lớp thật, không quan tâm lớp mang số nào**. (Ở bảng trên, ta chỉ hoán vị cột của $\mathbf{W}$ mà giữ nguyên $\mathbf{Y}$, tức là đổi mô hình chứ không đổi tên lớp, nên J tăng.)

Một tính chất nữa sẽ cần ở mục 4.3. Từ tính bất biến khi dịch của softmax (mục 2.5), cộng cùng một vector $\mathbf{v}$ vào mọi cột của $\mathbf{W}$ không đổi $\mathbf{P}$, nên

$$J(\mathbf{W} + \mathbf{v}\mathbf{1}_K^T) = J(\mathbf{W}) \quad \text{với mọi } \mathbf{v} \in \mathbb{R}^{d+1}$$

tức J có những hướng "phẳng": nếu $\mathbf{W}^*$ là điểm cực tiểu thì cả một họ ma trận $\mathbf{W}^* + \mathbf{v}\mathbf{1}_K^T$ cũng là cực tiểu.

3.3. Gradient của hàm mất mát và quy tắc cập nhật

Gradient của mất mát một mẫu

Ta tính gradient của $\ell_i = -\sum_k y_{ik} \ln p_{ik}$ theo $\mathbf{W}$ qua chuỗi $\mathbf{W} \to s_{ij} \to p_{ik} \to \ell_i$, với ba bước.

Bước 1: đạo hàm theo p_{ik}.

$$\frac{\partial \ell_i}{\partial p_{ik}} = -\frac{y_{ik}}{p_{ik}}$$

Bước 2: đạo hàm của softmax (đã chứng minh ở mục 2.3):

$$\frac{\partial p_{ik}}{\partial s_{ij}} = p_{ik}\big(\mathbb{1}[k = j] - p_{ij}\big)$$

Vì mỗi s_{ij} ảnh hưởng đến **mọi** p_{ik} (do mẫu số chung), ta phải cộng theo k. Thừa số p_{ik} ở bước 2 triệt tiêu mẫu số p_{ik} ở bước 1:

$$\frac{\partial \ell_i}{\partial s_{ij}} = \sum_{k=0}^{K-1} \left(-\frac{y_{ik}}{p_{ik}}\right) p_{ik}\left(\mathbb{1}[k=j] - p_{ij}\right) = -y_{ij} + p_{ij}\sum_{k=0}^{K-1} y_{ik} = p_{ij} - y_{ij}$$

(ta dùng $\sum_k y_{ik} = 1$). Đây đúng là kết quả đã báo trước ở mục 6.3 bài Logistic Regression. Gradient theo vector điểm số của mẫu i là

$$\frac{\partial \ell_i}{\partial \mathbf{s}_i} = \mathbf{p}_i - \mathbf{y}_i$$

Bước 3: đạo hàm của điểm số. Vì $s_{ij} = \mathbf{x}_i \mathbf{w}_j$, đạo hàm theo $\mathbf{w}_k$ là $\mathbf{x}_i^T$ khi $j = k$ và là $\mathbf{0}$ khi $j \neq k$. Nhân lại:

$$\nabla_{\mathbf{w}_k} \ell_i = (p_{ik} - y_{ik})\, \mathbf{x}_i^T, \qquad k = 0, \dots, K-1$$

Xếp K vector cột này cạnh nhau, ta được gradient theo cả ma trận $\mathbf{W}$ dưới dạng một tích ngoài:

$$\nabla_{\mathbf{W}} \ell_i = \mathbf{x}_i^T (\mathbf{p}_i - \mathbf{y}_i)$$

Kiểm tra kích thước: $\mathbf{x}_i^T$ là $(d+1) \times 1$ và $(\mathbf{p}_i - \mathbf{y}_i)$ là $1 \times K$, nên tích là $(d+1) \times K$, khớp với $\mathbf{W}$. Hệ quả giống Logistic Regression: thừa số $p(1-p)$ gây "gradient tắt" ở MSE đã bị triệt tiêu hoàn toàn, gradient theo điểm số chỉ còn là sai số $\mathbf{p}_i - \mathbf{y}_i$.

Ví dụ. Mẫu có điểm số $\mathbf{s} = [0{,}5 \;\; 1{,}5 \;\; 2{,}5]$ (mục 2.3), $\mathbf{p} \approx [0{,}090 \;\; 0{,}245 \;\; 0{,}665]$ và nhãn thật là lớp 1, tức $\mathbf{y} = [0 \;\; 1 \;\; 0]$. Mất mát là $-\ln 0{,}245 \approx 1{,}408$ và gradient theo điểm số là

$$\mathbf{p} - \mathbf{y} \approx [0{,}090 \;\; -0{,}755 \;\; 0{,}665]$$

Phần tử âm duy nhất ở lớp đúng: tăng điểm số lớp 1 sẽ làm J giảm. Hai phần tử dương ở hai lớp còn lại: tăng điểm số của chúng sẽ làm J tăng, và lớp 2 (đang bị gán $0{,}665$ một cách sai) có phần tử dương lớn nhất.

Gradient của hàm mất mát tổng

Cộng gradient của n mẫu và lấy trung bình:

$$\nabla_{\mathbf{W}} J = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i^T\, (\mathbf{p}_i - \mathbf{y}_i) = \frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$$

Kiểm tra kích thước: $\mathbf{X}^T$ là $(d+1) \times n$ và $(\mathbf{P} - \mathbf{Y})$ là $n \times K$ nên tích là $(d+1) \times K$. Với từng cột, ta có

$$\nabla_{\mathbf{w}_k} J = \frac{1}{n}\mathbf{X}^T\left(\mathbf{p}_{:,k} - \mathbf{y}_{:,k}\right)$$

trong đó $\mathbf{p}_{:,k}$ và $\mathbf{y}_{:,k}$ là cột thứ k của $\mathbf{P}$ và $\mathbf{Y}$. Đây **đúng là gradient của Logistic Regression** cho bài toán "lớp k hay không", chỉ khác ở chỗ xác suất $\mathbf{p}_{:,k}$ lấy từ softmax (có tính đến mọi lớp) thay vì sigmoid riêng lẻ. Một lần nữa ta thấy softmax là K bộ phân loại Logistic được gắn với nhau bằng mẫu số chung.

Quy tắc cập nhật

Áp dụng Gradient Descent $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} J$:

$$\mathbf{W} \leftarrow \mathbf{W} - \frac{\eta}{n} \mathbf{X}^T (\mathbf{P} - \mathbf{Y}) = \mathbf{W} + \frac{\eta}{n} \mathbf{X}^T (\mathbf{Y} - \mathbf{P})$$

Cách đọc quen thuộc: với mỗi mẫu i và mỗi lớp k, sai số $(y_{ik} - p_{ik})$ nhân với $\mathbf{x}_i^T$ rồi cộng vào cột $\mathbf{w}_k$.

- **Lớp đúng** ($y_{ik} = 1$): hệ số $(1 - p_{ik}) > 0$, nên cột $\mathbf{w}_k$ được **kéo** về phía $\mathbf{x}_i^T$, làm điểm số $\mathbf{x}_i \mathbf{w}_k$ tăng. Mô hình càng ít tin vào lớp đúng thì kéo càng mạnh.
- **Các lớp còn lại** ($y_{ik} = 0$): hệ số $-p_{ik} < 0$, nên cột $\mathbf{w}_k$ bị **đẩy** ra xa $\mathbf{x}_i^T$, và lớp nào đang bị gán xác suất cao một cách sai thì bị đẩy mạnh nhất.
- **Mẫu đã đúng và chắc chắn** ($\mathbf{p}_i \approx \mathbf{y}_i$): $\mathbf{p}_i - \mathbf{y}_i \approx \mathbf{0}$, gần như không đóng góp.

Đây là phiên bản "mềm" của quy tắc Perceptron nhiều lớp ở mục 6.1 bài Perceptron (kéo cột lớp đúng, đẩy cột lớp bị chọn nhầm): Perceptron chỉ đẩy **một** lớp bị chọn nhầm và chỉ khi sai, còn softmax đẩy **mọi** lớp sai theo trọng số p_{ik} và học cả từ mẫu đã đúng. Chi tiết "kéo và đẩy" sẽ được phân tích ở mục 4.1.

Một tính chất: tổng các cột của $\mathbf{W}$ không đổi

Mỗi hàng của $\mathbf{P}$ và của $\mathbf{Y}$ cộng lại bằng 1, nên $(\mathbf{P} - \mathbf{Y})\mathbf{1}_K = \mathbf{0}$. Suy ra

$$(\nabla_{\mathbf{W}} J)\mathbf{1}_K = \frac{1}{n} \mathbf{X}^T (\mathbf{P} - \mathbf{Y})\mathbf{1}_K = \mathbf{0}$$

Nghĩa là **mỗi hàng của gradient cộng lại bằng** 0: ở mỗi lần cập nhật, phần "kéo" của lớp đúng bằng đúng tổng phần "đẩy" của các lớp còn lại. Hệ quả: tổng $\mathbf{W}\mathbf{1}_K = \sum_k \mathbf{w}_k$ **không đổi** qua mọi bước. Nếu khởi tạo $\mathbf{W} = \mathbf{0}$ thì $\sum_k \mathbf{w}_k = \mathbf{0}$ ở mọi bước, và nghiệm tìm được là điểm trong họ $\mathbf{W}^* + \mathbf{v}\mathbf{1}_K^T$ (mục 3.2) có các cột cộng lại bằng $\mathbf{0}$. Đây là một cách giải quyết tính dư thừa tham số ở mục 2.5 mà không cần làm gì thêm. Ta sẽ kiểm tra bằng số ở mục 3.6 và 3.7.

So sánh các quy tắc cập nhật

Bốn mô hình ta đã học có quy tắc cập nhật cùng một "khung", chỉ khác phần "sai số":

Mô hình	Quy tắc cập nhật theo toàn bộ dữ liệu
Hồi quy tuyến tính	$\mathbf{w} \leftarrow \mathbf{w} - \frac{2\eta}{n} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$
Logistic Regression	$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{n} \mathbf{X}^T (\sigma(\mathbf{X}\mathbf{w}) - \mathbf{y})$
Softmax Regression	$\mathbf{W} \leftarrow \mathbf{W} - \frac{\eta}{n} \mathbf{X}^T (\text{softmax}(\mathbf{X}\mathbf{W}) - \mathbf{Y})$

(Hệ số 2 ở hồi quy có thể gộp vào η.) Cả ba đều có dạng $\mathbf{X}^T(\text{dự đoán} - \text{thực tế})$. Từ Logistic sang Softmax chỉ có hai thay đổi: vector $\mathbf{w}$ thành ma trận $\mathbf{W}$, và σ thành softmax. Với $K = 2$, quy tắc cập nhật cho $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_0$ trở về đúng dạng của Logistic Regression (với bước học hiệu dụng 2η, vì $\mathbf{w}_1$ và $\mathbf{w}_0$ di chuyển ngược chiều nhau).

3.4. Vai trò của learning rate và chuẩn hóa đặc trưng

Learning rate

Tốc độ học η quyết định mỗi bước cập nhật đi xa bao nhiêu. Giống Logistic Regression (và khác Perceptron), Softmax Regression rất nhạy với η, vì độ lớn của gradient phụ thuộc vào các xác suất hiện tại. Ta kiểm chứng trên bộ dữ liệu 600 sinh viên ở mục 1.2, dùng 420 mẫu huấn luyện và 180 mẫu kiểm tra (cách tạo dữ liệu ở mục 3.7). Với đặc trưng đã chuẩn hóa (xem bên dưới), ta chạy batch Gradient Descent từ $\mathbf{W}^{(0)} = \mathbf{0}$ với các giá trị η khác nhau. Bảng ghi J sau số epoch cho trước; giá trị nhỏ nhất có thể đạt được trên dữ liệu này là $J \approx 0{,}3730$ (tính bằng thuật toán L-BFGS chạy đến khi gradient gần bằng 0), và $J(\mathbf{0}) = \ln 3 \approx 1{,}0986$:

η	Sau 10 epoch	Sau 100 epoch	Sau 500 epoch	Sau 2000 epoch
0,01	1,061	0,846	0,608	0,464
0,1	0,843	0,528	0,406	0,375
0,5	0,600	0,405	0,374	**0,373**
1	0,520	0,383	0,373	**0,373**
2	0,455	0,375	0,373	**0,373**
5	0,394	0,373	0,373	**0,373**
10	0,408	0,405	0,405	0,405

Hình 8a vẽ các đường cong học tương ứng. Có thể rút ra:

- η quá nhỏ (0,01): J giảm rất chậm, sau 2000 epoch vẫn chưa tới mức tối ưu.
- η vừa phải (0,5 đến 5): J giảm nhanh và hội tụ về $\approx 0{,}373$.
- η quá lớn (10): bước nhảy quá dài làm $\mathbf{W}$ dao động quanh điểm tối ưu, J dừng ở khoảng 0,405 và không về được 0,373.

Cách nhận biết η chưa phù hợp giống Logistic Regression: vẽ J theo epoch. Nếu J tăng hoặc dao động mạnh thì giảm η; nếu J giảm đều nhưng quá chậm thì tăng η.

Vì sao cần chuẩn hóa đặc trưng

Giá trị η phù hợp phụ thuộc vào thang đo của các đặc trưng, vì gradient $\frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$ tỉ lệ với giá trị của các cột của $\mathbf{X}$. Trong bộ dữ liệu này, số giờ học x_1 có giá trị cỡ vài đơn vị đến hơn 10, còn số buổi vắng x_2 cỡ 0 đến 6, và cột 1 của hệ số chệch có giá trị 1. Các "thang đo" lệch nhau khiến hàm mất mát có dạng một thung lũng dài và hẹp, nên một η vừa với hướng này thì quá lớn với hướng kia. Có thể định lượng bằng giá trị riêng của $\frac{1}{n}\mathbf{X}^T\mathbf{X}$ (liên quan đến độ cong của J). Hessian

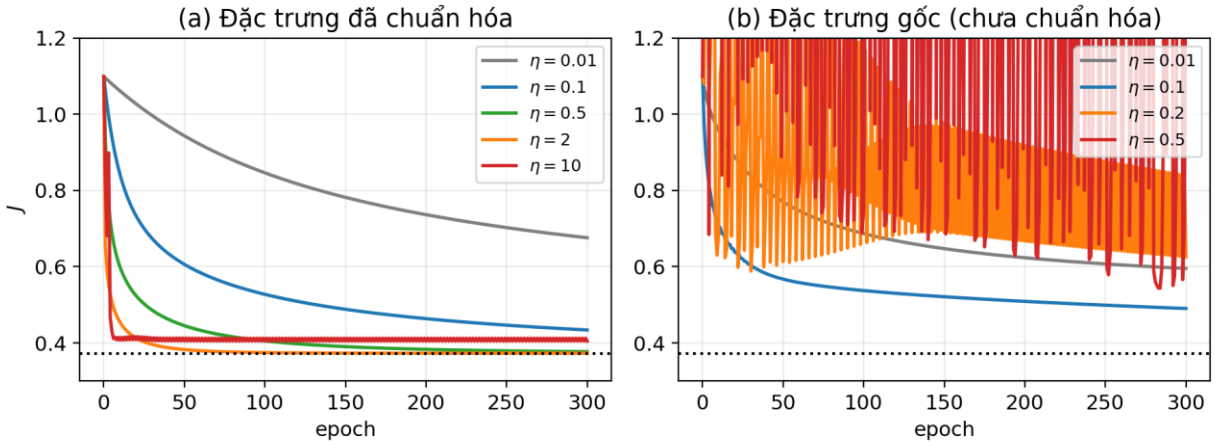
của J không vượt quá $\frac{1}{2}\lambda_{\max}\left(\frac{1}{n}\mathbf{X}^T\mathbf{X}\right)$ theo mọi hướng (mỗi mẫu có $\text{diag}(\mathbf{p}) - \mathbf{p}^T\mathbf{p} \preccurlyeq \frac{1}{2}\mathbf{I}$), nên $\eta < 4/\lambda_{\max}$ là một **điều kiện đủ** để J giảm qua mỗi bước:

Đặc trưng	Giá trị riêng của $\frac{1}{n}\mathbf{X}^T\mathbf{X}$	Tỉ số lớn/nhỏ	$4/\lambda_{\max}$
Gốc (chưa chuẩn hóa)	0,069; 5,24; 53,5	khoảng 770	khoảng 0,075
Đã chuẩn hóa	0,46; 1,00; 1,54	khoảng 3,3	khoảng 2,6

Với đặc trưng gốc, giá trị riêng lớn nhất lớn gấp 35 lần ở dữ liệu đã chuẩn hóa, nên η an toàn nhỏ hơn rất nhiều, trong khi giá trị riêng nhỏ nhất rất bé nên hướng đó học cực chậm. Thực nghiệm trên đặc trưng gốc (huấn luyện 2000 epoch từ $\mathbf{W} = \mathbf{0}$) cho thấy rõ:

η	Sau 10 epoch	Sau 100 epoch	Sau 500 epoch	Sau 2000 epoch
0,001	1,073	0,958	0,771	0,624
0,01	0,957	0,687	0,568	0,509
0,1	0,706	0,537	0,465	0,400
0,2	0,685	0,778	0,568	0,415
0,5	4,912	0,750	0,706	0,889

Ngay cả η tốt nhất (0,1) cũng mới đạt 0,400 sau 2000 epoch, trong khi đặc trưng chuẩn hóa với $\eta = 0{,}5$ đạt 0,374 chỉ sau 500 epoch. Với $\eta \geq 0{,}2$, J dao động mạnh (có lúc vọt lên 2,08 với $\eta = 0{,}2$ và 6,50 với $\eta = 0{,}5$, Hình 8b). (Điều kiện $\eta < 4/\lambda_{\max}$ chỉ là điều kiện đủ nên ngưỡng thực tế lớn hơn một chút: $\eta = 0{,}1$ vẫn chạy được dù lớn hơn 0,075.)



Hình 8. Hàm mất mát J theo epoch (batch Gradient Descent, $\mathbf{W}^{(0)} = \mathbf{0}$) với các tốc độ học khác nhau. (a) Đặc trưng đã chuẩn hóa: η từ 0,5 đến 2 đều hội tụ nhanh, $\eta = 0{,}01$ chậm, $\eta = 10$ dao động. (b) Đặc trưng gốc: η phải nhỏ, và hội tụ vẫn chậm hơn nhiều. Đường chấm là giá trị tối ưu $J \approx 0{,}373$.

Cách chuẩn hóa

Ta chuẩn hóa từng đặc trưng (không chuẩn hóa cột 1 của hệ số chệch) về trung bình 0 và độ lệch chuẩn 1:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \qquad \mu_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}\,, \qquad \sigma_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(x_{ij} - \mu_j\right)^2}$$

Quy tắc quan trọng: μ_j và σ_j chỉ tính **từ tập huấn luyện**, rồi dùng đúng các giá trị đó để biến đổi tập kiểm tra (nếu tính cả trên tập kiểm tra thì thông tin của tập kiểm tra đã lọt vào quá trình huấn luyện). Trên 420 mẫu huấn luyện, ta có $\mu = [6{,}295 \;\; 2{,}394]$ và $\sigma = [3{,}108 \;\; 1{,}685]$; ví dụ một sinh viên học 10 giờ có $z_1 = (10 - 6{,}295)/3{,}108 \approx 1{,}192$. Sau chuẩn hóa, các trọng số cũng dễ so sánh với nhau hơn, vì chúng cùng thang đo.

3.5. Thuật toán Gradient Descent qua từng bước

Các biến thể

Gradient $\nabla_{\mathbf{W}} J = \frac{1}{n}\sum_i \mathbf{x}_i^T\,(\mathbf{p}_i - \mathbf{y}_i)$ là trung bình của n gradient riêng lẻ. Tùy số mẫu dùng để ước lượng nó ở mỗi bước, ta có ba biến thể giống Logistic Regression:

Biến thể	Số mẫu mỗi lần cập nhật	Số lần cập nhật mỗi epoch	Quy tắc
Batch Gradient Descent	n (toàn bộ)	1	$\mathbf{W} \leftarrow \mathbf{W} - \frac{\eta}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$
Stochastic Gradient Descent (SGD)	1	n	$\mathbf{W} \leftarrow \mathbf{W} - \eta\,\mathbf{x}_i^T(\mathbf{p}_i - \mathbf{y}_i)$
Mini-batch Gradient Descent	b $(1 < b < n)$	$\lceil n/b \rceil$	$\mathbf{W} \leftarrow \mathbf{W} - \frac{\eta}{b}\mathbf{X}_B^T(\mathbf{P}_B - \mathbf{Y}_B)$

Trong dòng cuối, B là tập chỉ số của mini-batch hiện tại, $\mathbf{X}_B$ là ma trận gồm các hàng của $\mathbf{X}$ tương ứng, $\mathbf{P}_B$ và $\mathbf{Y}_B$ là các ma trận con tương ứng (mỗi ma trận có b hàng). Một epoch là một lượt duyệt qua toàn bộ n mẫu, bất kể biến thể nào. Với $n = 420$ và $b = 32$, mỗi epoch có $\lceil 420/32 \rceil = 14$ lần cập nhật (13 batch đủ 32 mẫu và một batch cuối có 4 mẫu).

Thuật toán

Đầu vào: ma trận đặc trưng $\mathbf{X}$ $(n \times (d+1)$, đã có cột 1), ma trận nhãn one-hot $\mathbf{Y}$ $(n \times K)$, tốc độ học $\eta > 0$, kích thước mini-batch b (với $b = n$ ta được batch, với $b = 1$ ta được SGD), số epoch tối đa $T_{\max}$, ngưỡng dừng ε.

Đầu ra: ma trận tham số $\mathbf{W}$.

3. Khởi tạo $\mathbf{W} = \mathbf{0}$ (kích thước $(d+1) \times K$).

4. Lặp cho $t = 1,2, \dots, T_{\max}$ (mỗi lần lặp là một epoch):
    1. Xáo trộn thứ tự các mẫu (với $b = n$ thì không cần).
    2. Chia các mẫu thành các mini-batch gồm b mẫu.
    3. Với mỗi mini-batch B:
        1. Tính ma trận điểm số và xác suất: $\mathbf{P}_B = \text{softmax}(\mathbf{X}_B\mathbf{W})$.
        2. Tính gradient $\mathbf{G} = \frac{1}{b}\mathbf{X}_B^T(\mathbf{P}_B - \mathbf{Y}_B)$.
        3. Cập nhật $\mathbf{W} \leftarrow \mathbf{W} - \eta\,\mathbf{G}$.
    4. Nếu điều kiện dừng thỏa (xem bên dưới): dừng.
5. Trả về $\mathbf{W}$.

Khởi tạo $\mathbf{W} = \mathbf{0}$ là hợp lệ: khác với mạng nơ-ron nhiều lớp (cần khởi tạo ngẫu nhiên để phá đối xứng), ở đây J lồi nên khởi tạo nào cũng được và các cột $\mathbf{w}_k$ vẫn phân hóa vì các cột của $\mathbf{Y}$ khác nhau (xem thêm mục 4.2). Mỗi bước tốn một phép nhân $\mathbf{X}_B\mathbf{W}$ (kích thước $b \times K$) và một phép nhân $\mathbf{X}_B^T(\cdot)$, nên chi phí tăng tuyến tính theo K.

Điều kiện dừng

Giống Logistic Regression, ta không đạt gradient bằng đúng 0 trong hữu hạn bước, nên dùng một trong các điều kiện:

- Độ lớn gradient đủ nhỏ: $\| \nabla_{\mathbf{W}} J \| < \varepsilon$ (với batch; $\|\cdot\|$ là chuẩn Frobenius của ma trận).
- Hàm mất mát gần như không giảm nữa: $|J_t - J_{t-1}| < \text{tol}$.
- Đạt số epoch tối đa $T_{\max}$ (luôn nên có, như một "van an toàn").

Với SGD và mini-batch, J dao động vì mỗi bước chỉ dùng một phần dữ liệu, nên điều kiện dừng thường dựa vào số epoch cố định hoặc theo dõi J trên toàn bộ dữ liệu sau mỗi epoch.

So sánh các biến thể

- **Batch:** hướng đi chính xác, đường cong học mượt, nhưng mỗi bước tính trên toàn bộ dữ liệu và chỉ có một bước mỗi epoch nên chậm khi n lớn.
- **SGD:** mỗi bước rất rẻ và có n bước mỗi epoch nên tiến bộ nhanh, nhưng hướng đi nhiễu, J dao động quanh điểm tối ưu và không hội tụ hẳn nếu η cố định.
- **Mini-batch:** là sự dung hòa: ít nhiễu hơn SGD, rẻ hơn batch, và tận dụng tốt phép nhân ma trận trên phần cứng hiện đại. Đây là lựa chọn phổ biến nhất trong thực tế.

Thực nghiệm ba biến thể trên 420 mẫu huấn luyện sẽ có ở mục 3.7 (Hình 9).

3.6. Ví dụ tính toán thủ công một vòng huấn luyện với $K = 3$

Ta chạy batch Gradient Descent trên 9 sinh viên ở mục 1.2, với khởi tạo $\mathbf{W}^{(0)} = \mathbf{0}$ và $\eta = 0{,}1$. Để dễ tính tay, ta dùng đặc trưng gốc (không chuẩn hóa). Nhắc lại

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 4 \\ 1 & 3 & 3 \\ 1 & 4 & 5 \\ 1 & 6 & 3 \\ 1 & 7 & 2 \\ 1 & 6 & 4 \\ 1 & 9 & 1 \\ 1 & 10 & 0 \\ 1 & 8 & 0 \end{bmatrix}, \qquad \mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Bước 1 (từ $\mathbf{W}^{(0)} = \mathbf{0}$)

Xác suất. Vì $\mathbf{W}^{(0)} = \mathbf{0}$ nên $\mathbf{S} = \mathbf{0}$ và $p_{ik} = 1/3$ cho mọi mẫu và mọi lớp, tức $\mathbf{P}^{(0)}$ là ma trận 9×3 toàn $1/3$.

Hàm mất mát. $J\left(\mathbf{W}^{(0)}\right) = \ln 3 \approx 1{,}0986$, đúng như đã nói ở mục 3.1.

Gradient. Ta tính $\mathbf{X}^T(\mathbf{P} - \mathbf{Y}) = \mathbf{X}^T\mathbf{P} - \mathbf{X}^T\mathbf{Y}$ theo từng phần. Vì mọi phần tử của $\mathbf{P}$ bằng $1/3$, mỗi cột của $\mathbf{X}^T\mathbf{P}$ bằng $\frac{1}{3}$ tổng các cột của $\mathbf{X}$, tức $\frac{1}{3}[9 \;\; 55 \;\; 22]^T = [3 \;\; 18{,}333 \;\; 7{,}333]^T$. Còn $\mathbf{X}^T\mathbf{Y}$ có cột k là tổng các hàng $\mathbf{x}_i^T$ của lớp k:

$$\mathbf{X}^T\mathbf{Y} = \begin{bmatrix} 3 & 3 & 3 \\ 9 & 19 & 27 \\ 12 & 9 & 1 \end{bmatrix} \quad \Rightarrow \quad \mathbf{X}^T(\mathbf{P} - \mathbf{Y}) = \begin{bmatrix} 0 & 0 & 0 \\ 9{,}333 & -0{,}667 & -8{,}667 \\ -4{,}667 & -1{,}667 & 6{,}333 \end{bmatrix}$$

$$\nabla_{\mathbf{W}} J\left(\mathbf{W}^{(0)}\right) = \frac{1}{9}\mathbf{X}^T(\mathbf{P} - \mathbf{Y}) \approx \begin{bmatrix} 0 & 0 & 0 \\ 1{,}0370 & -0{,}0741 & -0{,}9630 \\ -0{,}5185 & -0{,}1852 & 0{,}7037 \end{bmatrix}$$

Hàng đầu (hệ số chệch) bằng $\mathbf{0}$ vì ba lớp cân bằng (mỗi lớp 3 mẫu), nên chưa cần đổi hệ số chệch. Mỗi hàng cộng lại bằng 0, đúng như tính chất ở mục 3.3.

Cập nhật.

$$\mathbf{W}^{(1)} = \mathbf{W}^{(0)} - 0{,}1\,\nabla_{\mathbf{W}} J\left(\mathbf{W}^{(0)}\right) \approx \begin{bmatrix} 0 & 0 & 0 \\ -0{,}1037 & 0{,}0074 & 0{,}0963 \\ 0{,}0519 & 0{,}0185 & -0{,}0704 \end{bmatrix}$$

Dấu hợp lý: cột $\mathbf{w}_0$ (rớt) có hệ số của giờ học âm và của số buổi vắng dương; cột $\mathbf{w}_2$ (giỏi) ngược lại. Mô hình bắt đầu học đúng chiều mà ta đã chọn tay ở mục 2.2.

Bước 2 (từ $\mathbf{W}^{(1)}$)

Điểm số và xác suất. Ta tính $\mathbf{S} = \mathbf{X}\mathbf{W}^{(1)}$ rồi $\mathbf{P} = \text{softmax}(\mathbf{S})$ theo từng hàng. Ví dụ sinh viên 1 (học 2 giờ, vắng 4 buổi) có $\mathbf{s}_1 = \mathbf{x}_1\mathbf{W}^{(1)} \approx [0 \;\; 0{,}0889 \;\; -0{,}0889]$, và lấy mũ rồi chia tổng được $\mathbf{p}_1 \approx [0{,}3325 \;\; 0{,}3634 \;\; 0{,}3042]$. Với cả 9 sinh viên:

$$\mathbf{P}^{(1)} \approx \begin{bmatrix} 0{,}3325 & 0{,}3634 & 0{,}3042 \\ 0{,}2836 & 0{,}3582 & 0{,}3582 \\ 0{,}2834 & 0{,}3742 & 0{,}3424 \\ 0{,}1975 & 0{,}3481 & 0{,}4544 \\ 0{,}1610 & 0{,}3278 & 0{,}5112 \\ 0{,}2109 & 0{,}3596 & 0{,}4295 \\ 0{,}1113 & 0{,}2927 & 0{,}5960 \\ 0{,}0875 & 0{,}2658 & 0{,}6466 \\ 0{,}1193 & 0{,}2901 & 0{,}5907 \end{bmatrix}, \qquad \mathbf{P}^{(1)} - \mathbf{Y} \approx \begin{bmatrix} -0{,}6675 & 0{,}3634 & 0{,}3042 \\ -0{,}7164 & 0{,}3582 & 0{,}3582 \\ -0{,}7166 & 0{,}3742 & 0{,}3424 \\ 0{,}1975 & -0{,}6519 & 0{,}4544 \\ 0{,}1610 & -0{,}6722 & 0{,}5112 \\ 0{,}2109 & -0{,}6404 & 0{,}4295 \\ 0{,}1113 & 0{,}2927 & -0{,}4040 \\ 0{,}0875 & 0{,}2658 & -0{,}3534 \\ 0{,}1193 & 0{,}2901 & -0{,}4093 \end{bmatrix}$$

Hàm mất mát. $J\left(\mathbf{W}^{(1)}\right) \approx 0{,}9217$, giảm so với $1{,}0986$.

Gradient và cập nhật. Tính $\mathbf{X}^T\left(\mathbf{P}^{(1)} - \mathbf{Y}\right)$ rồi chia cho 9:

$$\nabla_{\mathbf{W}} J\left(\mathbf{W}^{(1)}\right) \approx \begin{bmatrix} -0{,}1348 & -0{,}0022 & 0{,}1370 \\ 0{,}0065 & -0{,}1720 & 0{,}1656 \\ -0{,}7258 & -0{,}1300 & 0{,}8559 \end{bmatrix}$$

$$\mathbf{W}^{(2)} = \mathbf{W}^{(1)} - 0{,}1\,\nabla_{\mathbf{W}} J\left(\mathbf{W}^{(1)}\right) \approx \begin{bmatrix} 0{,}0135 & 0{,}0002 & -0{,}0137 \\ -0{,}1043 & 0{,}0246 & 0{,}0797 \\ 0{,}1244 & 0{,}0315 & -0{,}1560 \end{bmatrix}, \qquad J\left(\mathbf{W}^{(2)}\right) \approx 0{,}8159$$

Các bước tiếp theo

Tiếp tục theo cách tương tự, ta thu được bảng:

Bước t	$J\left(\mathbf{W}^{(t)}\right)$	Số sinh viên dự đoán đúng
0	1,0986	(mọi lớp bằng nhau)
1	0,9217	3 / 9
2	0,8159	
3	0,7429	
4	0,6872	
5	0,6437	
6	0,6082	9 / 9

với $\mathbf{W}^{(6)} \approx \begin{bmatrix} 0{,}0417 & -0{,}0084 & -0{,}0333 \\ -0{,}1902 & 0{,}0351 & 0{,}1551 \\ 0{,}2999 & 0{,}0545 & -0{,}3544 \end{bmatrix}$.

Nhận xét

- J giảm đều ở mọi bước. Sau bước 1, nhãn dự đoán mới đúng $3/9$ (chỉ ba sinh viên giỏi được đoán đúng; ba sinh viên rớt bị đoán là đạt và ba sinh viên đạt bị đoán là giỏi). Sau 6 bước, $9/9$ mẫu đã được dự đoán đúng dù J còn $0{,}608$, vì các xác suất vẫn chưa đủ "chắc chắn". Giống Logistic Regression, thuật toán không dừng khi mọi mẫu đã đúng: ta tiếp tục đẩy xác suất của lớp đúng lên cao hơn.
- Cả 9 mẫu đóng góp vào gradient ở mọi bước, với trọng số $\mathbf{p}_i - \mathbf{y}_i$ (khác Perceptron chỉ dùng mẫu sai).
- Mỗi hàng của $\mathbf{W}^{(t)}$ cộng lại bằng 0 ở mọi bước (ví dụ hàng 2 của $\mathbf{W}^{(2)}$: $-0{,}1043 + 0{,}0246 + 0{,}0797 = 0$), đúng như tính chất ở mục 3.3.
- Dữ liệu 9 sinh viên này phân tách hoàn toàn được, nên J sẽ giảm mãi về 0 nếu cứ chạy tiếp ($\mathbf{W}$ tăng không giới hạn). Ta sẽ phân tích ở mục 4.4.

3.7. Mã hóa thuật toán Softmax Regression từ đầu bằng Python

Ta cài đặt thuật toán bằng NumPy. Mỗi công thức có một dòng mã tương ứng, giống hai bài trước; so với Logistic Regression, mã chỉ khác ở hai chỗ: vector thành ma trận và sigmoid thành softmax.

Ký hiệu toán	Trong mã Python
Ma trận đặc trưng $\mathbf{X}$ (có cột 1)	*Xb* (kết quả của *them_bias*)
Ma trận nhãn one-hot $\mathbf{Y}$	*Y* (kết quả của *one_hot*)
Ma trận tham số $\mathbf{W}$	*W*
Ma trận điểm số $\mathbf{S} = \mathbf{XW}$	*Xb @ W*
Ma trận xác suất $\mathbf{P} = \text{softmax}(\mathbf{S})$	*softmax(Xb @ W)*
Hàm mất mát $J(\mathbf{W})$	*mat_mat(Xb, Y, W)*
Gradient $\frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$	*Xb.T @ (P - Y) / n*
Cập nhật $\mathbf{W} \leftarrow \mathbf{W} - \eta\nabla J$	*W = W - eta * gradient(...)*

Các hàm cơ bản

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

def them_bias(X):
    """Thêm cột 1 vào đầu ma trận đặc trưng: (n, d) -> (n, d+1)."""
    return np.hstack([np.ones((X.shape[0], 1)), X])

def one_hot(y, K):
    """Vector nhãn (n,) -> ma trận one-hot Y (n, K)."""
    return np.eye(K)[y]
```

```
def softmax(S):
    """Softmax theo từng hàng; trừ giá trị lớn nhất của hàng để không tràn số
."""
    S = S - S.max(axis=1, keepdims=True)
    E = np.exp(S)
    return E / E.sum(axis=1, keepdims=True)

def mat_mat(Xb, Y, W):
    """Cross-entropy trung bình J(W), tính qua log-sum-exp:
    l_i = -s_ic + ln sum_j e^{s_ij}."""
    S = Xb @ W
    m = S.max(axis=1, keepdims=True)
    lse = m[:, 0] + np.log(np.exp(S - m).sum(axis=1))
    return np.mean(lse - np.sum(Y * S, axis=1))

def gradient(Xb, Y, W):
    """Gradient của J theo W: (1/n) * X^T (P - Y), kích thước (d+1, K)."""
    P = softmax(Xb @ W)
    return Xb.T @ (P - Y) / len(Xb)

def du_doan_xac_suat(Xb, W):
    """Ma trận xác suất P (n, K)."""
    return softmax(Xb @ W)

def du_doan(Xb, W):
    """Nhãn dự đoán: lớp có xác suất lớn nhất."""
    return du_doan_xac_suat(Xb, W).argmax(axis=1)
```

Có hai chi tiết về độ ổn định số học. Hàm *softmax* trừ giá trị lớn nhất của mỗi hàng trước khi lấy mũ (tính bất biến khi dịch, mục 2.3). Hàm *mat_mat* dùng dạng log-sum-exp ở mục 3.1 nên không bao giờ phải tính ln0, và ta không cần *np.clip* như ở Logistic Regression.

Hàm huấn luyện

Một hàm duy nhất xử lý cả ba biến thể: *kich_thuoc_batch=None* cho batch, bằng 1 cho SGD, và một số *b* khác cho mini-batch.

```
def huan_luyen(Xb, Y, eta=0.5, so_epoch=1000, kich_thuoc_batch=None, seed=0):
    """Gradient Descent: toàn bộ (batch), từng mẫu (SGD) hoặc mini-batch."""
    n, d1 = Xb.shape
    b = n if kich_thuoc_batch is None else kich_thuoc_batch
    rng = np.random.default_rng(seed)
    W = np.zeros((d1, Y.shape[1]))                    # khởi tạo W = 0
    lich_su = [mat_mat(Xb, Y, W)]                     # J sau mỗi epoch
    for epoch in range(so_epoch):
        idx = rng.permutation(n) if b < n else np.arange(n)
        for k in range(0, n, b):
            lo = idx[k:k + b]                         # chỉ số của mini-batc
h hiện tại
```

```
            W = W - eta * gradient(Xb[lo], Y[lo], W)
        lich_su.append(mat_mat(Xb, Y, W))
    return W, lich_su
```

Kiểm tra trên dữ liệu 9 sinh viên

Ta so với ví dụ tính tay ở mục 3.6 (batch, $\eta = 0{,}1$), và kiểm tra gradient giải tích bằng đạo hàm số (finite difference) tại một ma trận **W** ngẫu nhiên:

```
X_sv = np.array([[2, 4], [3, 3], [4, 5], [6, 3], [7, 2], [6, 4], [9, 1], [10,
0], [8, 0]], dtype=float)
y_sv = np.array([0, 0, 0, 1, 1, 1, 2, 2, 2])
Xb_sv, Y_sv = them_bias(X_sv), one_hot(y_sv, 3)

for k in (1, 2, 6):
    W_k, ls = huan_luyen(Xb_sv, Y_sv, eta=0.1, so_epoch=k)
    print(f"sau {k} buoc: J = {ls[-1]:.4f}")
    print(W_k)
print("J ban dau =", round(ls[0], 4), "= ln 3 =", round(np.log(3), 4))

rng = np.random.default_rng(0)
W_t = rng.normal(0, 0.3, size=(3, 3)); eps = 1e-6
g_so = np.zeros_like(W_t)
for i in range(3):
    for j in range(3):
        E = np.zeros_like(W_t); E[i, j] = eps
        g_so[i, j] = (mat_mat(Xb_sv, Y_sv, W_t + E) - mat_mat(Xb_sv, Y_sv, W_
t - E)) / (2 * eps)
g_gt = gradient(Xb_sv, Y_sv, W_t)
print("gradient giai tich:"); print(g_gt)
print("sai khac toi da   :", np.abs(g_gt - g_so).max())
```

```
sau 1 buoc: J = 0.9217
[[ 0.      0.      0.    ]
 [-0.1037  0.0074  0.0963]
 [ 0.0519  0.0185 -0.0704]]
sau 2 buoc: J = 0.8159
[[ 0.0135  0.0002 -0.0137]
 [-0.1043  0.0246  0.0797]
 [ 0.1244  0.0315 -0.156 ]]
sau 6 buoc: J = 0.6082
[[ 0.0417 -0.0084 -0.0333]
 [-0.1902  0.0351  0.1551]
 [ 0.2999  0.0545 -0.3544]]
J ban dau = 1.0986 = ln 3 = 1.0986
gradient giai tich:
[[ 0.2178 -0.1961 -0.0217]
 [ 2.0306 -1.4642 -0.5664]
 [ 0.2975 -0.5585  0.2609]]
sai khac toi da   : 1.744442645890132e-10
```

Các giá trị khớp hoàn toàn với ví dụ tính tay ($\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$, $\mathbf{W}^{(6)}$ cùng các giá trị J ở mục 3.6), và gradient giải tích khác đạo hàm số chỉ cỡ 10^{-10}, xác nhận công thức $\nabla_{\mathbf{W}} J = \frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$ là đúng.

Huấn luyện trên bộ dữ liệu 600 sinh viên

Ta tạo bộ dữ liệu mô phỏng 600 sinh viên (200 mỗi lớp, ba lớp chồng lấn vừa phải như ở mục 1.2), chia 70% huấn luyện và 30% kiểm tra (420 và 180 mẫu), rồi chuẩn hóa theo quy tắc ở mục 3.4:

```
def sinh_du_lieu(seed=42, n_lop=200):
    rng = np.random.default_rng(seed)
    def cum(m1, m2, s1, s2):
        return np.c_[rng.normal(m1, s1, n_lop), rng.normal(m2, s2, n_lop)]
    X = np.vstack([cum(3.0, 3.75, 1.55, 1.55),      # lớp 0: rớt
                   cum(6.25, 2.0, 1.55, 1.395),     # lớp 1: đạt
                   cum(9.75, 1.3, 1.55, 1.24)])     # lớp 2: giỏi
    return np.clip(X, 0, None), np.repeat([0, 1, 2], n_lop)

def chia_train_test(X, y, ti_le_test=0.3, seed=0):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(y))
    n_test = int(len(y) * ti_le_test)
    return X[idx[n_test:]], X[idx[:n_test]], y[idx[n_test:]], y[idx[:n_test]]

def chuan_hoa(X_tr, X_te):
    mu, s = X_tr.mean(axis=0), X_tr.std(axis=0)     # chỉ tính từ tập huấn lu
yện
    return (X_tr - mu) / s, (X_te - mu) / s, mu, s

X, y = sinh_du_lieu()
X_tr, X_te, y_tr, y_te = chia_train_test(X, y)
Z_tr, Z_te, mu, s = chuan_hoa(X_tr, X_te)
Zb_tr, Zb_te = them_bias(Z_tr), them_bias(Z_te)
Y_tr = one_hot(y_tr, 3)
print(Zb_tr.shape, Zb_te.shape, "| mu =", mu, "| s =", s)
```

```
(420, 3) (180, 3) | mu = [6.2951 2.3943] | s = [3.1081 1.685 ]
```

Ta chạy ba biến thể trong 200 epoch, mỗi biến thể dùng một η hợp lý (SGD cần η nhỏ hơn vì mỗi epoch có tới 420 bước):

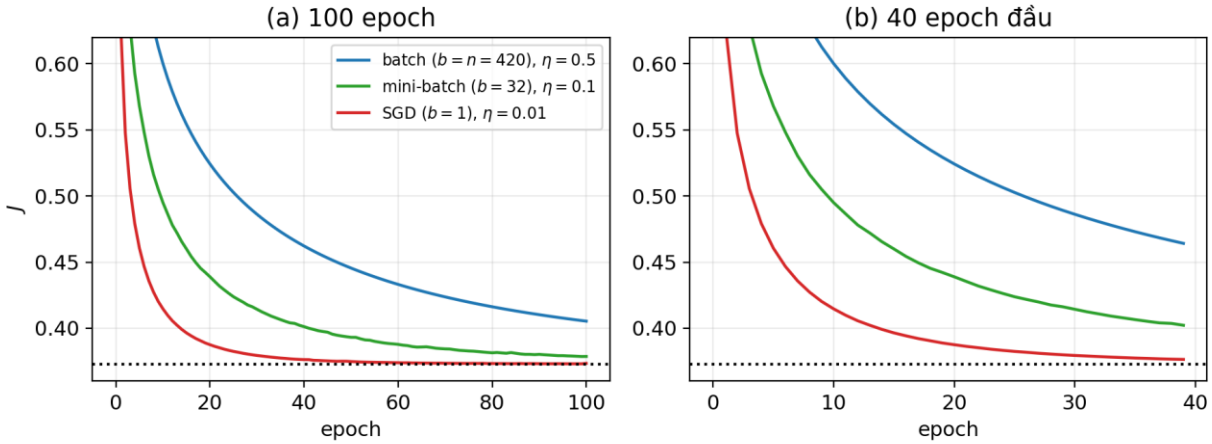
```
cau_hinh = {"batch (b = n = 420)": dict(eta=0.5, kich_thuoc_batch=None),
            "mini-batch (b = 32)": dict(eta=0.1, kich_thuoc_batch=32),
            "SGD (b = 1)":         dict(eta=0.01, kich_thuoc_batch=1)}
for ten, cfg in cau_hinh.items():
    W_k, ls = huan_luyen(Zb_tr, Y_tr, so_epoch=200, **cfg)
    acc_tr = np.mean(du_doan(Zb_tr, W_k) == y_tr)
    acc_te = np.mean(du_doan(Zb_te, W_k) == y_te)
    print(f"{ten:22s} J = {ls[-1]:.4f} | train {acc_tr:.4f} | test {acc_te:.4
```

```
f}")
print("J ban dau =", round(ls[0], 4), "= ln 3 =", round(np.log(3), 4))
```

```
batch (b = n = 420)    J = 0.3833 | train 0.8571 | test 0.8500
mini-batch (b = 32)    J = 0.3737 | train 0.8595 | test 0.8556
SGD (b = 1)            J = 0.3730 | train 0.8595 | test 0.8500
J ban dau = 1.0986 = ln 3 = 1.0986
```

Dòng cuối xác nhận $J(\mathbf{0}) = \ln 3$. Hình 9 vẽ J theo epoch cho ba cách. Có một số nhận xét:

- Theo số epoch, mini-batch và SGD giảm J nhanh hơn batch vì thực hiện nhiều lần cập nhật hơn trong mỗi epoch (tương ứng 14 lần và 420 lần so với 1 lần). Sau 200 epoch, batch với $\eta = 0{,}5$ mới đạt $J = 0{,}3833$ và chưa hội tụ hẳn (mức tối ưu là 0,3730, cần khoảng 500 epoch như bảng ở mục 3.4). Vì vậy độ chính xác của batch thấp hơn chút ít, không phải do batch "kém hơn" mà do nó chưa chạy đủ lâu. Chênh lệch độ chính xác kiểm tra giữa các cách (85,0% so với 85,6%) chỉ là một mẫu trên 180, chưa đáng kể.
- SGD và mini-batch dao động quanh điểm tối ưu vì mỗi bước chỉ dựa trên một phần dữ liệu; batch thì mượt.
- Cả ba cách đều tiến về cùng một ma trận tối ưu, chỉ khác tốc độ và độ nhiễu.



Hình 9. Hàm mất mát J theo epoch của ba biến thể Gradient Descent trên cùng dữ liệu huấn luyện (đặc trưng chuẩn hóa). (a) 100 epoch; (b) 40 epoch đầu. Mini-batch và SGD thực hiện nhiều bước hơn mỗi epoch nên giảm nhanh hơn và dao động nhẹ; batch mượt nhưng chậm hơn. Đường chấm là giá trị tối ưu $J \approx 0{,}373$.

Cuối cùng, ta chạy batch với $\eta = 0{,}5$ trong 2000 epoch để lấy ma trận $\mathbf{W}$ gần tối ưu, và kiểm tra các tính chất ở mục 3.3:

```
W, ls = huan_luyen(Zb_tr, Y_tr, eta=0.5, so_epoch=2000)
print("J cuoi =", round(ls[-1], 4)); print("W ="); print(W)
print("tong moi hang cua W:", W.sum(axis=1))
```

```
G = gradient(Zb_tr, Y_tr, W); print("||grad|| =", np.linalg.norm(G))
y_hat = du_doan(Zb_te, W)
print("acc train =", np.mean(du_doan(Zb_tr, W) == y_tr).round(4),
      "| acc test =", np.mean(y_hat == y_te).round(4))
C = np.zeros((3, 3), dtype=int)
for t, p in zip(y_te, y_hat): C[t, p] += 1
print(C)
```

```
J cuoi = 0.373
W =
[[-1.2902  1.8028 -0.5126]
 [-4.3761  0.2513  4.1248]
 [ 1.3687 -0.4102 -0.9584]]
tong moi hang cua W: [-0.  0.  0.]
||grad|| = 3.452176216415884e-05
acc train = 0.8619 | acc test = 0.85
[[49 11  0]
 [ 9 48  3]
 [ 0  4 56]]
```

Kết quả khớp với mô hình softmax ở mục 1.3: độ chính xác kiểm tra 85,0% và đúng ma trận nhầm lẫn 3×3 đã dùng ở đó, trong đó lớp "đạt" được nhận đúng $48/60 = 80{,}0\%$. Gradient đã gần bằng 0 ($3{,}5 \cdot 10^{-5}$), tổng mỗi hàng của $\mathbf{W}$ bằng 0 (hệ quả của $(\mathbf{P} - \mathbf{Y})\mathbf{1}_K = \mathbf{0}$ và khởi tạo $\mathbf{W} = \mathbf{0}$). Có thể đọc ma trận $\mathbf{W}$ như sau (đặc trưng đã chuẩn hóa): hàng của giờ học là $[-4{,}38 \;\; 0{,}25 \;\; 4{,}12]$, nên giờ học tăng thì điểm của lớp rớt giảm mạnh, của lớp giỏi tăng mạnh, và gần như không đổi với lớp đạt, đúng với trực giác (lớp đạt nằm ở giữa). Hiệu $\mathbf{w}_2 - \mathbf{w}_0$ mới là thứ có ý nghĩa (mục 2.5).

Tóm tắt mục 3

- Mất mát Perceptron không có ý nghĩa xác suất, còn MSE kết hợp với softmax gây "gradient tắt" ở những mẫu sai chắc chắn (ví dụ: gradient chỉ cỡ 0,056 so với sai số $\approx 1{,}4$). Ta cần hàm mất mát phạt nặng khi sai chắc chắn, có gradient lớn khi sai nặng, trơn, lồi.
- Với nhãn one-hot, xác suất mô hình gán cho nhãn quan sát là $q_i = \prod_k p_{ik}^{y_{ik}}$. Theo hợp lý cực đại, ta chọn $\mathbf{W}$ làm $L = \prod_i q_i$ lớn nhất, tương đương cực tiểu hóa **cross-entropy nhiều lớp**

$$J(\mathbf{W}) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=0}^{K-1} y_{ik} \ln p_{ik}$$

  Mất mát một mẫu là $-\ln q_i$. Với $K = 2$ nó là cross-entropy nhị phân; khi $\mathbf{W} = \mathbf{0}$ thì $J = \ln K$. Hàm này không phụ thuộc cách đặt tên lớp và không đổi khi cộng cùng một vector vào mọi cột của $\mathbf{W}$.

- Gradient có dạng rất gọn nhờ thừa số p_{ik} triệt tiêu: $\partial \ell_i / \partial \mathbf{s}_i = \mathbf{p}_i - \mathbf{y}_i$ và $\nabla_{\mathbf{W}} J = \frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$, kích thước $(d+1) \times K$. Quy tắc cập nhật $\mathbf{W} \leftarrow \mathbf{W} - \frac{\eta}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$ cùng dạng với hồi quy tuyến tính và Logistic Regression: kéo cột lớp đúng, đẩy các cột lớp còn lại theo trọng số p_{ik}. Mỗi hàng của gradient cộng lại bằng 0, nên tổng các cột của $\mathbf{W}$ không đổi qua các bước.
- Tốc độ học η rất quan trọng: quá nhỏ thì chậm, quá lớn thì dao động. Chuẩn hóa đặc trưng (chỉ dùng trung bình và độ lệch chuẩn của tập huấn luyện) làm J "tròn" hơn, cho phép dùng η lớn hơn nhiều: trên dữ liệu 600 sinh viên, đặc trưng chuẩn hóa với $\eta = 0{,}5$ gần mức tối ưu sau 500 epoch, trong khi đặc trưng gốc chưa đạt sau 2000 epoch.
- Batch, SGD và mini-batch khác nhau ở số mẫu dùng cho mỗi lần cập nhật; batch chính xác nhưng chậm, SGD nhanh nhưng nhiễu, mini-batch dung hòa. Một epoch là một lượt duyệt hết dữ liệu.
- Ví dụ tính tay $K = 3$ trên 9 sinh viên ($\eta = 0{,}1$) cho J: $1{,}0986 \rightarrow 0{,}9217 \rightarrow 0{,}8159$; cài đặt NumPy khớp với ví dụ này, gradient giải tích khớp đạo hàm số đến cỡ 10^{-10}, và trên 600 sinh viên mô hình đạt $85{,}0\%$ trên tập kiểm tra.

Câu hỏi kiểm tra nhanh

1. Cho $K = 3$, một mẫu có $\mathbf{p} = [0{,}2 \;\; 0{,}5 \;\; 0{,}3]$ và nhãn thật là lớp 2. Tính mất mát cross-entropy của mẫu và gradient theo vector điểm số.
2. Hai mẫu độc lập có xác suất mô hình gán cho lớp thật lần lượt là $q_1 = 0{,}8$ và $q_2 = 0{,}5$. Tính L, $\ln L$ và J (trung bình trên hai mẫu).
3. Với $K = 10$ lớp và $\mathbf{W} = \mathbf{0}$, giá trị của J là bao nhiêu? Độ chính xác của mô hình này trên một tập cân bằng sẽ cỡ bao nhiêu?
4. Một mô hình có $d = 20$ đặc trưng và $K = 10$ lớp, huấn luyện trên $n = 1000$ mẫu. Tìm kích thước của $\mathbf{W}$ và của $\nabla_{\mathbf{W}} J$. Mỗi epoch có bao nhiêu lần cập nhật khi dùng batch, SGD và mini-batch với $b = 50$?
5. Cho $\mathbf{x} = [1 \;\; 2 \;\; 1]$ (gồm số 1 của hệ số chệch), nhãn lớp 1 ($K = 3$) và $\mathbf{W} = \mathbf{0}$. Tính gradient $\mathbf{x}^T(\mathbf{p} - \mathbf{y})$ của mất mát một mẫu, thực hiện một bước SGD với $\eta = 0{,}3$, rồi tính xác suất lớp 1 và mất mát mới.
6. Chứng minh rằng $(\nabla_{\mathbf{W}} J)\mathbf{1}_K = \mathbf{0}$. Từ đó suy ra gì về tổng các cột của $\mathbf{W}$ khi khởi tạo $\mathbf{W} = \mathbf{0}$?

7. Số giờ học có $\mu = 6{,}295$ và $\sigma = 3{,}108$ (tính từ tập huấn luyện). Tính giá trị chuẩn hóa của một sinh viên học 10 giờ. Vì sao không chuẩn hóa cột 1 của hệ số chệch, và vì sao không tính μ, σ trên cả tập kiểm tra?

8. Khi huấn luyện, ta thấy J tăng dần qua các epoch. Có thể có nguyên nhân gì và cần làm gì?

Gợi ý đáp án

1. Lớp thật là lớp 2 nên $q = p_2 = 0{,}3$ và mất mát là $-\ln 0{,}3 \approx 1{,}204$. Gradient theo điểm số là $\mathbf{p} - \mathbf{y} = [0{,}2 \;\; 0{,}5 \;\; 0{,}3] - [0 \;\; 0 \;\; 1] = [0{,}2 \;\; 0{,}5 \;\; -0{,}7]$ (các phần tử cộng lại bằng 0).

2. $L = 0{,}8 \times 0{,}5 = 0{,}4$; $\ln L = \ln 0{,}4 \approx -0{,}916$; $J = -\frac{1}{2}\ln L \approx 0{,}458$. (Kiểm tra: $\frac{1}{2}(-\ln 0{,}8 - \ln 0{,}5) = \frac{1}{2}(0{,}223 + 0{,}693) = 0{,}458$.)

3. Mọi $p_{ik} = 1/10$ nên $J = \ln 10 \approx 2{,}303$. Mô hình dự đoán các lớp "hòa nhau" nên không đưa ra được dự đoán có ý nghĩa; nếu chọn một lớp bất kỳ thì độ chính xác kỳ vọng là $1/10 = 10\%$ trên tập cân bằng.

4. $\mathbf{W}$ có kích thước $(d+1) \times K = 21 \times 10$ (210 tham số) và $\nabla_{\mathbf{W}} J$ có cùng kích thước 21×10. Batch: 1 lần cập nhật mỗi epoch; SGD: 1000 lần; mini-batch: $\lceil 1000/50 \rceil = 20$ lần.

5. Với $\mathbf{W} = \mathbf{0}$ thì $\mathbf{p} = \left[\frac{1}{3} \;\; \frac{1}{3} \;\; \frac{1}{3}\right]$, $\mathbf{y} = [0 \;\; 1 \;\; 0]$ nên $\mathbf{p} - \mathbf{y} = \left[\frac{1}{3} \;\; -\frac{2}{3} \;\; \frac{1}{3}\right]$ và gradient là $\mathbf{x}^T(\mathbf{p} - \mathbf{y}) = \begin{bmatrix} 1/3 & -2/3 & 1/3 \\ 2/3 & -4/3 & 2/3 \\ 1/3 & -2/3 & 1/3 \end{bmatrix}$. Sau một bước, $\mathbf{W} = -0{,}3 \times$ gradient $= \begin{bmatrix} -0{,}1 & 0{,}2 & -0{,}1 \\ -0{,}2 & 0{,}4 & -0{,}2 \\ -0{,}1 & 0{,}2 & -0{,}1 \end{bmatrix}$ và $\mathbf{s} = \mathbf{x}\mathbf{W} = [-0{,}6 \;\; 1{,}2 \;\; -0{,}6]$. Lấy mũ: $0{,}549; \;\; 3{,}320; \;\; 0{,}549$, tổng 4,418, nên $\mathbf{p} \approx [0{,}124 \;\; 0{,}752 \;\; 0{,}124]$. Xác suất lớp 1 tăng từ $0{,}333$ lên $0{,}752$ và mất mát giảm từ $\ln 3 \approx 1{,}099$ xuống $-\ln 0{,}752 \approx 0{,}286$.

6. $(\nabla_{\mathbf{W}} J)\mathbf{1}_K = \frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})\mathbf{1}_K$ và $(\mathbf{P} - \mathbf{Y})\mathbf{1}_K = \mathbf{P}\mathbf{1}_K - \mathbf{Y}\mathbf{1}_K = \mathbf{1}_n - \mathbf{1}_n = \mathbf{0}$ (mỗi hàng của $\mathbf{P}$ và $\mathbf{Y}$ cộng lại bằng 1). Vậy mỗi cập nhật không làm đổi $\mathbf{W}\mathbf{1}_K$, nghĩa là tổng các cột $\sum_k \mathbf{w}_k$ giữ nguyên. Khởi tạo $\mathbf{W} = \mathbf{0}$ thì $\sum_k \mathbf{w}_k = \mathbf{0}$ ở mọi bước.

7. $z = (10 - 6{,}295)/3{,}108 \approx 1{,}192$. Cột 1 không chuẩn hóa vì nó là hằng số ($\sigma = 0$, phép chia vô nghĩa) và đóng vai trò hệ số chệch. Không tính μ, σ trên tập kiểm tra vì khi đó thông tin của tập kiểm tra đã tham gia vào quá trình huấn luyện, nên điểm đánh giá không còn phản ánh dữ liệu "chưa từng thấy".

8. Nguyên nhân thường gặp nhất là η quá lớn nên bước nhảy vọt qua điểm tối ưu và J dao động hoặc tăng; cách xử lý là giảm η (ví dụ chia 2 hoặc 10). Các nguyên nhân khác: đặc trưng chưa chuẩn hóa (cần chuẩn hóa), hoặc lỗi cài đặt gradient (kiểm tra bằng đạo hàm số như ở mục 3.7, và kiểm tra $J(\mathbf{0}) = \ln K$). Riêng với SGD và mini-batch, J đo trên từng batch dao động là bình thường; cần theo dõi J trên toàn bộ dữ liệu sau mỗi epoch.

4. Phân tích cơ chế học của Softmax Regression

Ở mục 3, ta đã xây dựng hàm mất mát cross-entropy nhiều lớp, tính được gradient $\frac{1}{n}\mathbf{X}^T(\mathbf{P}-\mathbf{Y})$ và cài đặt được thuật toán Gradient Descent giảm đều J từ $\ln 3$ xuống khoảng $0{,}373$ trên dữ liệu 600 sinh viên. Để hiểu thật sự mô hình học như thế nào, ta cần trả lời năm câu hỏi:

- Khi một mẫu bị phân loại sai, nó tác động lên các cột của $\mathbf{W}$ ra sao: cột nào bị kéo, cột nào bị đẩy, và mạnh đến mức nào?
- Khởi tạo, tốc độ học và thang đo của đặc trưng ảnh hưởng thế nào đến kết quả?
- Hàm mất mát có lồi không, và kết quả huấn luyện có phải một ma trận $\mathbf{W}$ duy nhất không?
- Điều gì xảy ra khi dữ liệu phân tách hoàn hảo, và điều chuẩn hóa L2 giúp gì?
- Mô hình không làm được gì, và ta có thể khắc phục bằng cách nào?

Mục này đặt cạnh nhau hai mô hình đã biết để so sánh. Với Logistic Regression, nghiệm tối ưu duy nhất và không phụ thuộc khởi tạo; câu hỏi tự nhiên là điều đó còn đúng khi $\mathbf{w}$ trở thành ma trận $\mathbf{W}$ hay không. Câu trả lời sẽ có một điểm khác biệt đáng chú ý (mục 4.3). Ta giữ quy ước ký hiệu cũ: chữ thường in đậm là vector, chữ hoa in đậm là ma trận; $\mathbf{x}_i$, $\mathbf{s}_i = \mathbf{x}_i\mathbf{W}$, $\mathbf{p}_i$ và $\mathbf{y}_i$ là các vector hàng; $\mathbf{w}_k$ là cột thứ k của $\mathbf{W}$; $q_i = p_{i,y_i}$ là xác suất gán cho lớp thật. Các thí nghiệm dùng lại dữ liệu 600 sinh viên (420 mẫu huấn luyện, 180 mẫu kiểm tra) và ma trận tối ưu $\mathbf{W}^*$ tìm được ở mục 3.7 ($\mathbf{W}$ đã chuẩn hóa, $J(\mathbf{W}^*) \approx 0{,}3730$; ở đây ta chạy thêm epoch với $\eta = 2$ nên các phần tử có thể lệch chút ít ở chữ số thập phân thứ ba so với ma trận in ở mục 3.7).

4.1. Điều gì xảy ra khi một mẫu bị phân loại sai?

Mức đóng góp của một mẫu vào gradient

Ở mục 3.3, ta đã có gradient của mất mát một mẫu là một tích ngoài $\nabla_{\mathbf{W}}\ell_i = \mathbf{x}_i^T(\mathbf{p}_i - \mathbf{y}_i)$. Vì là tích ngoài của hai vector (ma trận hạng một), chuẩn Frobenius của nó bằng tích hai chuẩn:

$$\| \nabla_{\mathbf{W}}\ell_i \|_F = \| \mathbf{x}_i \| \cdot \| \mathbf{p}_i - \mathbf{y}_i \|$$

Cũng như ở Logistic Regression, đó là tích của hai thừa số:

- $\| \mathbf{p}_i - \mathbf{y}_i \|$ là **mức sai của xác suất**. Nó bằng 0 khi mô hình gán xác suất 1 cho lớp thật, và không bao giờ vượt quá $\sqrt{2}$ (đạt khi toàn bộ xác suất dồn vào một lớp sai, tức $\mathbf{p}_i$ là vector one-hot của lớp sai).
- $\| \mathbf{x}_i \|$ là **độ lớn của vector đặc trưng**: mẫu có đặc trưng lớn (nằm xa tâm dữ liệu sau khi chuẩn hóa) kéo mạnh hơn.

Ta tính đóng góp của vài mẫu điển hình trong tập huấn luyện, tại $\mathbf{W}^*$ (mỗi mẫu ghi theo x_1 giờ học và x_2 buổi vắng; $\| \mathbf{x} \|$ tính trên vector đã chuẩn hóa kèm số 1):

Mẫu $(x_1;\ x_2)$	Lớp thật	$\mathbf{p}$	$\| \mathbf{p}-\mathbf{y} \|$	$\| \mathbf{x} \|$	$\| \nabla \ell \|_F$	Tình trạng
(2,27; 4,16)	rớt	[0,992 0,008 0,000]	0,012	1,94	0,023	Đúng, rất chắc chắn
(4,48; 2,59)	đạt	[0,451 0,544 0,005]	0,641	1,16	0,746	Đúng nhưng sát biên
(5,74; 4,11)	rớt	[0,381 0,602 0,017]	0,863	1,44	1,242	Sai nhẹ, sát biên
(5,67; 0,17)	rớt	[0,010 0,905 0,085]	1,344	1,67	2,243	Sai nặng
(10,00; 0,73)	đạt	[0,000 0,055 0,945]	1,337	1,84	2,464	Sai nặng và ở xa tâm

Quy luật giống Logistic Regression: mẫu đúng và chắc chắn gần như không đóng góp (gradient ≈ 0), mẫu sát biên đóng góp vừa phải, mẫu sai nặng đóng góp lớn nhất. Trên cả tập huấn luyện, có $58/420$ mẫu bị dự đoán sai và độ lớn gradient trung bình của nhóm sai là $1{,}423$, gấp hơn 5 lần nhóm đúng $(0{,}280)$. Hình 10a minh họa: kích thước mỗi điểm tỉ lệ với $(1-q_i)^2$, các điểm "to" tập trung quanh hai đường biên và ở phía sai, còn các điểm xa biên về phía đúng nhỏ gần như không thấy.

Mẫu cuối trong bảng, một sinh viên đạt nhưng học 10 giờ và gần như không vắng buổi nào (giống hệt một sinh viên giỏi), có gradient lớn nhất vì cả hai thừa số cùng lớn. Một mẫu như vậy có thể là nhãn nhiễu hay trường hợp đặc biệt, nhưng nó kéo các đường biên mạnh hơn hẳn mẫu thông thường. Đây là điểm yếu chung với Logistic Regression và là một lý do để quan tâm đến điều chuẩn hóa (mục 4.4).

Kéo lớp đúng, đẩy các lớp còn lại

Giả sử ta chỉ cập nhật theo một mẫu i có lớp thật c (một bước SGD): $\mathbf{W}' = \mathbf{W} - \eta\, \mathbf{x}_i^T (\mathbf{p}_i - \mathbf{y}_i)$. Xét từng cột, $\mathbf{w}_k{}' = \mathbf{w}_k - \eta\,(p_{ik} - y_{ik})\,\mathbf{x}_i^T$, tức là

$$\mathbf{w}_c{}' = \mathbf{w}_c + \eta\,(1-p_{ic})\,\mathbf{x}_i^T \quad \text{(kéo cột lớp đúng)}, \qquad \mathbf{w}_k{}' = \mathbf{w}_k - \eta\, p_{ik}\,\mathbf{x}_i^T \quad \big(\text{đẩy cột } k \neq c\big)$$

Điểm số mới của chính mẫu đó là $\mathbf{s}_i{}' = \mathbf{x}_i \mathbf{W}' = \mathbf{s}_i - \eta\,(\mathbf{p}_i - \mathbf{y}_i)\,\mathbf{x}_i \mathbf{x}_i^T$, nên

$$\mathbf{s}_i{}' = \mathbf{s}_i + \eta\ \| \mathbf{x}_i \|^2\ (\mathbf{y}_i - \mathbf{p}_i)$$

Đây là phiên bản nhiều lớp của công thức $s_i{}' = s_i + \eta(y_i - p_i) \| \mathbf{x}_i \|^2$ ở Logistic Regression. Có ba điều đáng chú ý:

- **Điểm số của lớp đúng tăng**, với độ tăng $\eta \| \mathbf{x}_i \|^2 (1-p_{ic})$ tỉ lệ với mức mô hình chưa tin vào lớp đúng.
- **Điểm số của mỗi lớp sai giảm**, với độ giảm $\eta \| \mathbf{x}_i \|^2 p_{ik}$ tỉ lệ với xác suất mô hình đang gán cho lớp đó. Lớp đang "thắng một cách sai" bị đẩy mạnh nhất, lớp hầu như không có khả năng thì gần như không bị đụng đến.

- **Tổng các thay đổi bằng** 0: $\sum_k (y_{ik} - p_{ik}) = 0$, nên độ tăng của lớp đúng đúng bằng tổng độ giảm của các lớp sai. Đây chính là tính chất "tổng các cột của **W** không đổi" ở mục 3.3, nhìn từ góc độ của một mẫu.

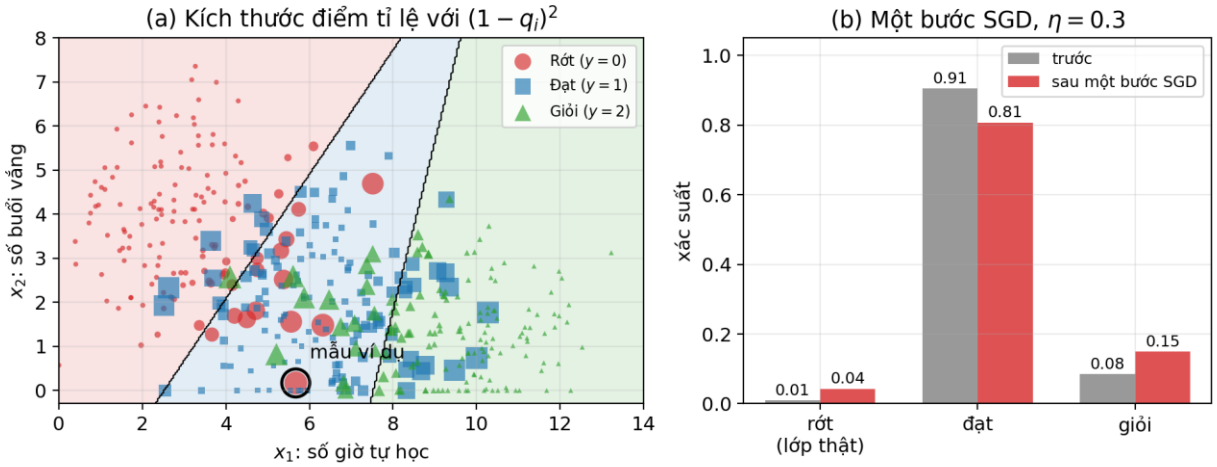
Ví dụ. Lấy mẫu (5,67; 0,17) ở bảng trên: lớp thật là rớt nhưng mô hình tin chắc là đạt. Sau chuẩn hóa, $\mathbf{x} = [1 \quad -0{,}2006 \quad -1{,}3205]$, $\| \mathbf{x} \|^2 \approx 2{,}784$. Điểm số và xác suất trước khi cập nhật:

$$\mathbf{s} \approx [-2{,}222 \quad 2{,}296 \quad -0{,}073], \qquad \mathbf{p} \approx [0{,}010 \quad 0{,}905 \quad 0{,}085]$$

Một bước SGD với $\eta = 0{,}3$ cho độ thay đổi điểm số $\eta \| \mathbf{x} \|^2 (\mathbf{y} - \mathbf{p}) \approx 0{,}835 \cdot [0{,}990 \quad -0{,}905 \quad -0{,}085] \approx [+0{,}827 \quad -0{,}756 \quad -0{,}071]$, và

$$\mathbf{s}' \approx [-1{,}396 \quad 1{,}540 \quad -0{,}144], \qquad \mathbf{p}' \approx [0{,}043 \quad 0{,}807 \quad 0{,}150]$$

(Hình 10b). Lớp rớt (lớp đúng) được kéo lên $+0{,}827$; lớp đạt, đang bị gán 0,905, bị đẩy xuống mạnh nhất ($-0{,}756$); lớp giỏi, chỉ bị gán 0,085, gần như đứng yên ($-0{,}071$). Mất mát của mẫu giảm từ $-\ln 0{,}010 \approx 4{,}618$ xuống $-\ln 0{,}043 \approx 3{,}149$. Hai điều cần lưu ý: một là xác suất của lớp giỏi vẫn tăng ($0{,}085 \rightarrow 0{,}150$) dù điểm số của nó giảm, vì điểm số của lớp đạt giảm nhiều hơn (xác suất là kết quả của so sánh tương đối giữa các điểm số); hai là J trên toàn bộ tập huấn luyện tăng từ 0,3730 lên 0,3923, vì $\mathbf{W}^*$ đã tối ưu cho toàn tập và một bước chỉ dành cho một mẫu thì làm lệch nó. Đó là lý do SGD dao động quanh điểm tối ưu (mục 3.5).



Hình 10. (a) Mức đóng góp của từng mẫu huấn luyện tại $\boldsymbol{W}^*$: kích thước điểm tỉ lệ với $(1 - q_i)^2$; nền màu là vùng quyết định. Các mẫu quanh đường biên và các mẫu bị sai có ảnh hưởng lớn nhất. (b) Xác suất của một mẫu lớp rớt bị dự đoán thành đạt, trước và sau một bước SGD ($\eta = 0{,}3$): xác suất lớp đúng tăng, xác suất lớp đang thắng sai giảm.

So sánh với Perceptron nhiều lớp

Perceptron nhiều lớp (mục 6.1 bài Perceptron) chỉ tác động khi mẫu sai: kéo cột lớp đúng thêm $\eta \mathbf{x}_i^T$ và đẩy **đúng một** cột, cột của lớp bị chọn nhầm, đi $\eta \mathbf{x}_i^T$, với độ lớn cố định không phụ thuộc mức sai. Softmax Regression mềm hơn ở ba điểm: mức kéo $(1 - p_{ic})$ thay đổi liên tục theo mức sai; mọi cột lớp sai đều bị đẩy, theo trọng số p_{ik} của chúng; và mẫu đã đúng nhưng

chưa chắc chắn vẫn tiếp tục được cải thiện. Nhờ vậy mô hình không dừng ở một đường biên sát mẫu mà tiếp tục đẩy các mẫu ra xa các đường biên.

4.2. Ảnh hưởng của khởi tạo, tốc độ học và chuẩn hóa đặc trưng

Chuẩn hóa đặc trưng

Ở mục 3.4, ta đã thấy chuẩn hóa giúp hội tụ nhanh hơn. Ta đo trực tiếp bằng số epoch cần để J cách mức tối ưu $J^* = 0{,}3730$ không quá $0{,}001$ (batch Gradient Descent từ $\mathbf{W} = \mathbf{0}$):

Tốc độ học η	Đặc trưng gốc	Đặc trưng đã chuẩn hóa
0,01	> 40000	24569
0,05	16341	4912
0,1	8170	2455
0,2	không hội tụ (dao động)	1227
0,5	không hội tụ (dao động)	490
1	không hội tụ (dao động)	244
2	không hội tụ (dao động)	121
5	không hội tụ (dao động)	45
10	không hội tụ (dao động)	không hội tụ (dao động)

Với cùng η (ví dụ 0,05 hay 0,1), chuẩn hóa nhanh hơn khoảng 3,3 lần. Quan trọng hơn, nó chịu được η lớn hơn nhiều: với đặc trưng gốc, $\eta = 0{,}2$ đã dao động, còn với đặc trưng chuẩn hóa ta dùng được đến $\eta = 5$. Kết quả là cấu hình tốt nhất cần 45 epoch, so với 8170 epoch của đặc trưng gốc (khoảng 180 lần). Mục 4.3 sẽ giải thích chính xác ngưỡng η này.

Chuyển ngược về thang đo gốc. Chuẩn hóa chỉ đổi cách đi đến nghiệm chứ không đổi mô hình. Nếu $\mathbf{W}'$ là ma trận học được trên đặc trưng đã chuẩn hóa $z_j = (x_j - \mu_j)/\sigma_j$, ta tìm $\mathbf{W}$ trên thang đo gốc sao cho điểm số không đổi, $[1 \;\; \mathbf{z}]\mathbf{W}' = [1 \;\; \mathbf{x}]\mathbf{W}$. Khai triển theo từng cột k:

$$w_{0k} = w_{0k}{}' - \sum_{j=1}^{d} \frac{\mu_j}{\sigma_j} w_{jk}{}', \qquad w_{jk} = \frac{w_{jk}{}'}{\sigma_j} \quad (j = 1, \dots, d)$$

Với $\mathbf{W}^*$ ở mục 3.7 và $\boldsymbol{\mu} = [6{,}295 \;\; 2{,}394]$, $\boldsymbol{\sigma} = [3{,}108 \;\; 1{,}685]$, ta được

$$\mathbf{W}_{\text{gốc}} \approx \begin{bmatrix} 5{,}633 & 1{,}876 & -7{,}509 \\ -1{,}409 & 0{,}081 & 1{,}328 \\ 0{,}813 & -0{,}244 & -0{,}569 \end{bmatrix}$$

Đúng ma trận này cũng thu được khi chạy Gradient Descent trực tiếp trên đặc trưng gốc (với $\eta = 0{,}1$ và rất nhiều epoch), và hai ma trận cho điểm số $\mathbf{XW}$ trùng nhau đến sai số cỡ 10^{-15}. Hệ số $-1{,}409$ ở hàng x_1 của lớp rớt nghĩa là mỗi giờ học thêm làm điểm số của lớp rớt giảm 1,409, còn của lớp giỏi tăng 1,328.

Chuẩn hóa còn làm các trọng số dễ so sánh với nhau hơn, vì chúng có cùng đơn vị "mỗi một độ lệch chuẩn". Trên $\mathbf{W}^*$ chuẩn hóa, hàng của số giờ học là $[-4{,}38 \;\; 0{,}25 \;\; 4{,}13]$ và hàng của số buổi vắng là $[1{,}37 \;\; -0{,}41 \;\; -0{,}96]$, cho thấy số giờ học là yếu tố tác động mạnh hơn nhiều.

Khởi tạo tham số

Với Logistic Regression, mọi khởi tạo đều dẫn đến cùng một $\mathbf{w}$. Ta thử điều tương tự với Softmax Regression, trên đặc trưng chuẩn hóa, $\eta = 2$, 8000 epoch, từ ba khởi tạo khác hẳn nhau:

$$\mathbf{W}_A^{(0)} = \begin{bmatrix} 2 & -1 & 0 \\ 1 & 3 & -2 \\ -3 & 1 & 2 \end{bmatrix}, \qquad \mathbf{W}_B^{(0)} = \begin{bmatrix} 1 & 1 & 1 \\ -2 & 0 & 2 \\ 2 & -2 & 0 \end{bmatrix}$$

và khởi tạo $\mathbf{0}$. Kết quả:

Khởi tạo	$J(\mathbf{W}^{(0)})$	J cuối	$\max\lvert\mathbf{P} - \mathbf{P}^*\rvert$	$\mathbf{W}$ cuối
$\mathbf{0}$	1,099	0,373000	0	$\begin{bmatrix} -1{,}292 & 1{,}804 & -0{,}512 \\ -4{,}380 & 0{,}253 & 4{,}127 \\ 1{,}370 & -0{,}411 & -0{,}959 \end{bmatrix}$
$\mathbf{W}_A^{(0)}$	6,461	0,373000	$2 \cdot 10^{-15}$	$\begin{bmatrix} -0{,}959 & 2{,}138 & -0{,}178 \\ -3{,}713 & 0{,}919 & 4{,}794 \\ 1{,}370 & -0{,}411 & -0{,}959 \end{bmatrix}$
$\mathbf{W}_B^{(0)}$	0,718	0,373000	$6 \cdot 10^{-16}$	$\begin{bmatrix} -0{,}292 & 2{,}804 & 0{,}488 \\ -4{,}380 & 0{,}253 & 4{,}127 \\ 1{,}370 & -0{,}411 & -0{,}959 \end{bmatrix}$

Có hai quan sát trái ngược nhau. **Về hàm mất mát và xác suất**, cả ba khởi tạo cho cùng $J = 0{,}373000$ và cùng ma trận xác suất $\mathbf{P}^*$ (sai khác cỡ 10^{-15}), giống Logistic Regression (Hình 11b). **Về chính ma trận W**, cả ba cho ba ma trận **khác nhau**, trái với Logistic Regression. Nhìn kỹ, hai ma trận cuối chỉ khác ma trận đầu theo một cách rất đều đặn: $\mathbf{W}_B^{(0)}$ cho kết quả chỉ lệch hàng đầu đúng $+1$ ở mọi cột, còn $\mathbf{W}_A^{(0)}$ cho hàng đầu lệch $+\frac{1}{3}$ và hàng thứ hai lệch $+\frac{2}{3}$ (so với kết quả từ $\mathbf{0}$). Đó chính là trung bình các phần tử của mỗi hàng của khởi tạo:

$$\mathbf{W}_B^{(0)}\mathbf{1}_K/K = [1 \;\; 0 \;\; 0]^T, \qquad \mathbf{W}_A^{(0)}\mathbf{1}_K/K = [1/3 \;\; 2/3 \;\; 0]^T$$

Hiện tượng này được giải thích ngay bằng tính chất ở mục 3.3: mỗi lần cập nhật không làm đổi $\mathbf{W}\mathbf{1}_K$. Vậy mọi khởi tạo hội tụ về $\mathbf{W}^*_{\text{tâm}} + \mathbf{v}_0\mathbf{1}_K^T$, trong đó $\mathbf{W}^*_{\text{tâm}}$ là nghiệm duy nhất có các hàng cộng lại bằng 0 (nghiệm tìm được từ $\mathbf{0}$) và $\mathbf{v}_0 = \mathbf{W}^{(0)}\mathbf{1}_K/K$ là vector trung bình theo hàng của khởi tạo:

$$\mathbf{W}^{(\infty)} = \mathbf{W}^*_{\text{tâm}} + \mathbf{v}_0\,\mathbf{1}_K^T, \qquad \mathbf{v}_0 = \frac{1}{K}\mathbf{W}^{(0)}\mathbf{1}_K$$

Ta kiểm chứng bằng mã ở cuối mục 4.3 (sai khác tối đa giữa $\mathbf{W}$ cuối và vế phải chỉ cỡ 10^{-14}). Khởi tạo vì thế chỉ ảnh hưởng đến số bước cần thiết và đến một thành phần "vô hình" của $\mathbf{W}$ (phần $\mathbf{v}_0\mathbf{1}_K^T$, không ảnh hưởng đến $\mathbf{P}$), không ảnh hưởng đến mô hình. Nguyên nhân sâu xa là tính dư thừa tham số đã gặp ở mục 2.5, sẽ được phân tích bằng Hessian ở mục 4.3.

Một nhận xét thêm: khởi tạo $\mathbf{W} = \mathbf{0}$ không làm các cột "giống hệt nhau" mãi mãi (đối xứng), vì các cột của $\mathbf{Y}$ khác nhau nên ngay bước đầu mỗi cột nhận một gradient khác nhau. Khác với mạng nơ-ron nhiều lớp (mục 6.3), ở đây ta không cần khởi tạo ngẫu nhiên.

4.3. Hàm mất mát lồi nhưng nghiệm không duy nhất: phân tích Hessian suy biến

Ma trận Hessian

Để biết hình dạng của $J(\mathbf{W})$, ta tính đạo hàm bậc hai. Gọi $\text{vec}(\mathbf{W}) = [\mathbf{w}_0^T \;\; \mathbf{w}_1^T \;\cdots\; \mathbf{w}_{K-1}^T]^T$ là vector dài $(d+1)K$ gồm các cột của $\mathbf{W}$ xếp nối nhau. Từ gradient theo $\mathbf{w}_k$ ở mục 3.3, $\nabla_{\mathbf{w}_k} J = \frac{1}{n}\sum_i (p_{ik} - y_{ik})\,\mathbf{x}_i^T$, và đạo hàm của softmax $\partial p_{ik} / \partial s_{il} = p_{ik}(\mathbb{1}[k=l] - p_{il})$ (mục 2.3), khối (k,l) của Hessian là $\frac{1}{n}\sum_i p_{ik}\,(\mathbb{1}[k=l] - p_{il})\,\mathbf{x}_i^T\mathbf{x}_i$. Gộp các khối lại bằng tích Kronecker:

$$\mathbf{H}(\mathbf{W}) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{A}_i \otimes (\mathbf{x}_i^T\mathbf{x}_i), \qquad \mathbf{A}_i = \text{diag}(\mathbf{p}_i) - \mathbf{p}_i^T\mathbf{p}_i$$

Ở đây $\mathbf{A}_i$ là ma trận $K \times K$ (chính là Jacobian của softmax tại $\mathbf{s}_i$), $\mathbf{x}_i^T\mathbf{x}_i$ là ma trận $(d+1)\times(d+1)$, và $\mathbf{H}$ có kích thước $(d+1)K \times (d+1)K$. Với $K = 2$ và vector $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_0$, công thức này trở về Hessian $\frac{1}{n}\mathbf{X}^T\mathbf{S}\mathbf{X}$ của Logistic Regression (vì $\mathbf{A}_i$ có phần tử $p_i(1-p_i)$).

Hàm mất mát là hàm lồi

Lấy một hướng bất kỳ $\mathbf{Z} \in \mathbb{R}^{(d+1)\times K}$ (cùng kích thước với $\mathbf{W}$) và đặt $\mathbf{u}_i = \mathbf{x}_i\mathbf{Z}$ (vector hàng $1 \times K$). Dạng toàn phương của Hessian là

$$\text{vec}(\mathbf{Z})^T\mathbf{H}\text{vec}(\mathbf{Z}) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{u}_i\,\mathbf{A}_i\mathbf{u}_i^T = \frac{1}{n}\sum_{i=1}^{n}\left[\sum_k p_{ik}\,u_{ik}^2 - (\sum_k p_{ik}\,u_{ik})^2\right]$$

Biểu thức trong ngoặc vuông chính là **phương sai** của đại lượng u_{ik} khi chọn lớp k theo phân phối $\mathbf{p}_i$, nên nó không âm. Vậy $\mathbf{H}$ nửa xác định dương tại mọi $\mathbf{W}$, tức J là hàm lồi, giống Logistic Regression. Hệ quả: mọi cực tiểu địa phương đều là cực tiểu toàn cục, và Gradient Descent với tốc độ học phù hợp hội tụ về một cực tiểu toàn cục từ mọi khởi tạo.

Nhưng Hessian suy biến

Phương sai bằng 0 khi và chỉ khi u_{ik} là hằng số theo k. Điều đó xảy ra với một họ hướng đặc biệt: $\mathbf{Z} = \mathbf{v}\mathbf{1}_K^T$ (cộng cùng một vector $\mathbf{v} \in \mathbb{R}^{d+1}$ vào mọi cột), vì khi đó $\mathbf{u}_i = (\mathbf{x}_i\mathbf{v})\mathbf{1}_K^T$ có mọi phần tử bằng nhau. Vậy

$$\text{vec}(\mathbf{v}\mathbf{1}_K^T)^T\,\mathbf{H}\,\text{vec}(\mathbf{v}\mathbf{1}_K^T) = 0 \quad \text{với mọi } \mathbf{v}$$

và vì $\mathbf{H}$ nửa xác định dương nên $\mathbf{H}\text{vec}(\mathbf{v}\mathbf{1}_K^T) = \mathbf{0}$: các hướng $\mathbf{v}\mathbf{1}_K^T$ là **vector riêng ứng với trị riêng** 0. Nếu các cột của $\mathbf{X}$ độc lập tuyến tính (hạng $d+1$) thì đây là *tất cả* các hướng phẳng: từ

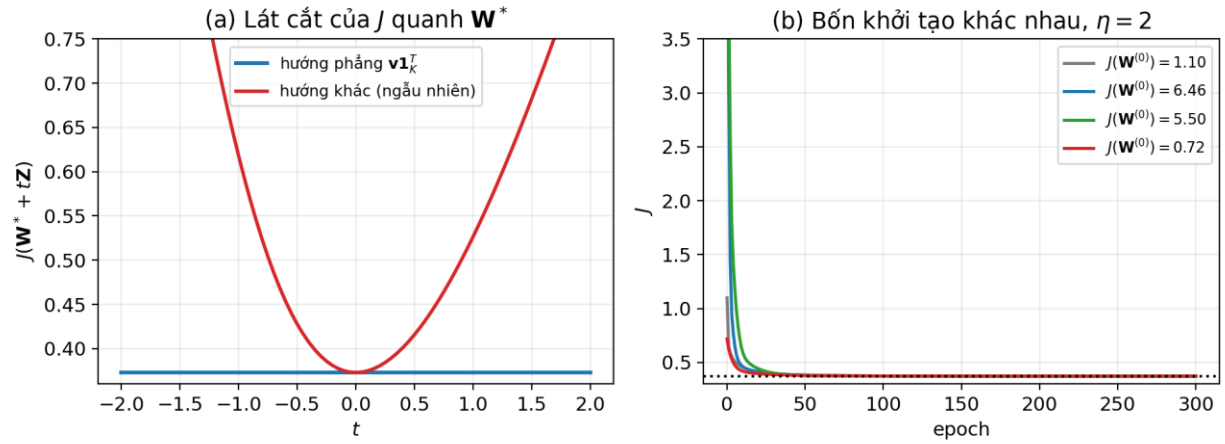
$\mathbf{u}_i = c_i \mathbf{1}_K^T$ với mọi i suy ra $\mathbf{XZ}_{:,k}$ giống nhau cho mọi cột k, và vì $\mathbf{X}$ có hạng đầy đủ nên các cột của $\mathbf{Z}$ phải bằng nhau. Vậy:

$$\dim\ker\mathbf{H} = d + 1, \qquad \text{rank}\mathbf{H} = (K-1)(d+1)$$

Ví dụ. Ta tính Hessian 9×9 tại $\mathbf{W}^*$ của bộ dữ liệu 600 sinh viên ($d = 2$, $K = 3$, đặc trưng chuẩn hóa). Các trị riêng là

$$\lambda \approx 0;\ \ 0;\ \ 0;\ \ 0{,}0058;\ \ 0{,}0135;\ \ 0{,}0295;\ \ 0{,}0738;\ \ 0{,}1333;\ \ 0{,}2286$$

đúng $d + 1 = 3$ trị riêng bằng 0 (cỡ 10^{-17}) và $(K-1)(d+1) = 6$ trị riêng dương. Với một vector $\mathbf{v}$ bất kỳ, $\| \mathbf{H}\text{vec}(\mathbf{v}\mathbf{1}_K^T) \| \approx 10^{-17}$. Hình 11a vẽ hai lát cắt của J quanh $\mathbf{W}^*$: dọc theo hướng $\mathbf{v}\mathbf{1}_K^T$ hàm là một đường **phẳng**, còn dọc theo một hướng khác, hàm là một parabol như thường lệ.



Hình 11. (a) Lát cắt của hàm mất mát quanh $\boldsymbol{W}^*$: dọc theo hướng $\boldsymbol{v}\mathbf{1}_K^T$ (cộng cùng một vector vào mọi cột) J không đổi; dọc theo hướng ngẫu nhiên khác J tăng dạng parabol. (b) Gradient Descent từ bốn khởi tạo khác nhau ($\eta = 2$, đặc trưng chuẩn hóa) đều hội tụ về cùng một giá trị $J \approx 0{,}373$, dù $\boldsymbol{W}$ cuối khác nhau.

Hệ quả

- **Tập nghiệm là một "thung lũng phẳng".** Nếu $\mathbf{W}^*$ là cực tiểu thì cả họ $\mathbf{W}^* + \mathbf{v}\mathbf{1}_K^T$ đều là cực tiểu, nên nghiệm **không duy nhất** (khác Logistic Regression, nơi Hessian xác định dương và nghiệm duy nhất). Nhưng $\mathbf{P}$, J và mọi dự đoán đều như nhau trên cả họ này, nên về mặt mô hình, đây là **một** nghiệm.
- **Gradient Descent không bị ảnh hưởng.** Vì $\nabla_{\mathbf{W}} J\ \mathbf{1}_K = \mathbf{0}$ (mục 3.3), gradient vuông góc với mọi hướng phẳng, nên thuật toán không bao giờ di chuyển dọc theo chúng. Thành phần của $\mathbf{W}$ theo các hướng này bị "đóng băng" ở giá trị khởi tạo, đó chính là công thức $\mathbf{W}^{(\infty)} = \mathbf{W}^*_{\text{tâm}} + \mathbf{v}_0 \mathbf{1}_K^T$ ở mục 4.2.

- **Không nên đọc từng cột của W một cách riêng lẻ.** Chỉ có hiệu giữa các cột có ý nghĩa (mục 2.5). Ví dụ trọng số $-1{,}409$ của số giờ học ở cột "rớt" ở mục 4.2 chỉ có nghĩa khi so với $0{,}081$ ở cột "đạt" và $1{,}328$ ở cột "giỏi".
- **Cách bỏ sự dư thừa.** Hoặc cố định một lớp tham chiếu ($\mathbf{w}_0 = \mathbf{0}$), hoặc thêm điều chuẩn hóa L2 (mục 4.4), hoặc khởi tạo $\mathbf{W} = \mathbf{0}$ như ta đã làm. Khi cố định cột $\mathbf{w}_0 = \mathbf{0}$, Hessian thu gọn (bỏ khối của $\mathbf{w}_0$) có cỡ $(K-1)(d+1) = 6$ và xác định dương: tại $\mathbf{W}^*$, các trị riêng là $0{,}0020;\ 0{,}0081;\ 0{,}0113;\ 0{,}0577;\ 0{,}0820;\ 0{,}2265$, đều dương nên nghiệm (trong cách tham số hóa này) là duy nhất.

Điều kiện tối ưu có ý nghĩa gì?

Tại nghiệm, gradient bằng $\mathbf{0}$:

$$\mathbf{X}^T(\mathbf{P} - \mathbf{Y}) = \mathbf{0} \Leftrightarrow \sum_{i=1}^{n} (p_{ik} - y_{ik})\ \mathbf{x}_i = \mathbf{0} \quad (k = 0, \dots, K-1)$$

Thành phần ứng với cột 1 của hệ số chệch cho

$$\frac{1}{n}\sum_{i=1}^{n} p_{ik} = \frac{1}{n}\sum_{i=1}^{n} y_{ik}$$

tức **trung bình xác suất dự đoán của mỗi lớp bằng đúng tỉ lệ của lớp đó** trong tập huấn luyện (đây là K phương trình, mở rộng điều kiện tương ứng của Logistic Regression). Với bộ dữ liệu của ta, mỗi lớp có $140/420 = 0{,}3333$ mẫu và trung bình xác suất dự đoán của cả ba lớp cũng bằng $0{,}3333$. Các thành phần còn lại nói rằng $\sum_i p_{ik}\, x_{ij} = \sum_i y_{ik}\, x_{ij}$: với mỗi đặc trưng, tổng "dự đoán nhân đặc trưng" bằng tổng "thực tế nhân đặc trưng" ở từng lớp. Ví dụ với đặc trưng số giờ học (đã chuẩn hóa), cả hai vế cho cùng bộ ba giá trị $[-149{,}28 \quad -1{,}08 \quad 150{,}36]$ ứng với ba lớp. Nói cách khác, mô hình đã "khớp mô-men" và khai thác hết thông tin tuyến tính trong các đặc trưng.

Độ cong quyết định tốc độ và độ ổn định

Gần điểm tối ưu, J xấp xỉ một hàm bậc hai với ma trận $\mathbf{H}$. Vì Gradient Descent chỉ di chuyển trong không gian trực giao với các hướng phẳng, nó hội tụ khi và chỉ khi $|1 - \eta\lambda| < 1$ với mọi trị riêng $\lambda > 0$ của $\mathbf{H}$, tức

$$\eta < \frac{2}{\lambda_{\max}(\mathbf{H})}$$

còn tốc độ hội tụ dọc hướng thoải nhất phụ thuộc vào trị riêng dương nhỏ nhất $\lambda^{+}_{\min}$. Tỉ số $\kappa = \lambda_{\max}/\lambda^{+}_{\min}$ (số điều kiện, tính trên các trị riêng dương) đo độ "dài hẹp" của thung lũng. Ta tính Hessian tại nghiệm cho cả hai cách biểu diễn đặc trưng:

	Đặc trưng gốc	Đặc trưng đã chuẩn hóa
$\lambda_{\max}$	10,05	0,2286
$\lambda^{+}_{\min}$	0,0021	0,0058
Số điều kiện κ	khoảng 4880	khoảng 39
Ngưỡng ổn định $2/\lambda_{\max}$	khoảng 0,199	khoảng 8,75

Bảng này giải thích trọn vẹn các quan sát trước đó:

- Với đặc trưng gốc, ngưỡng ổn định là $\eta \approx 0{,}199$: $\eta = 0{,}1$ hội tụ, còn $\eta = 0{,}2$ dao động, đúng như bảng ở mục 4.2. Với đặc trưng chuẩn hóa, ngưỡng là $\approx 8{,}75$: $\eta = 5$ hội tụ, $\eta = 10$ dao động.
- Chuẩn hóa làm số điều kiện giảm hơn 120 lần (từ khoảng 4880 xuống khoảng 39) và nâng ngưỡng ổn định lên gần 44 lần, nên có thể dùng η lớn hơn nhiều và hội tụ nhanh hơn nhiều.
- Số hướng phẳng (trị riêng bằng 0) không làm chậm thuật toán, vì thuật toán không đi theo các hướng đó.

Kiểm chứng bằng mã

Ta cài đặt Hessian bằng đúng công thức tích Kronecker và các hàm huấn luyện có tham số điều chuẩn hóa λ và khởi tạo $\mathbf{W}^{(0)}$ (dùng ở mục 4.4):

```
def huan_luyen_l2(Xb, Y, eta=1.0, so_epoch=1000, lam=0.0, W0=None):
    """Batch Gradient Descent với điều chuẩn hóa L2 (không phạt hàng đầu của
W, tức hệ số chệch)."""
    n, d1 = Xb.shape
    W = np.zeros((d1, Y.shape[1])) if W0 is None else W0.copy()
    D0 = np.eye(d1); D0[0, 0] = 0.0                    # ma trận D0 = diag(0,
1, ..., 1)
    for epoch in range(so_epoch):
        W = W - eta * (gradient(Xb, Y, W) + lam * D0 @ W)
    return W

def hessian(Xb, W):
    """Hessian của J theo vec(W) (xếp các cột w_0, w_1, ... nối nhau), kích t
hước (d+1)K x (d+1)K."""
    P = softmax(Xb @ W)
    H = np.zeros((Xb.shape[1] * W.shape[1],) * 2)
    for i in range(len(Xb)):
        A = np.diag(P[i]) - np.outer(P[i], P[i])        # diag(p_i) - p_i^T p
_i, kích thước K x K
        H += np.kron(A, np.outer(Xb[i], Xb[i]))         # A (x) x_i^T x_i
    return H / len(Xb)
```

Ta dùng lại *Zb_tr*, *Y_tr* ở mục 3.7. Đoạn mã sau tính độ đóng góp của mỗi mẫu, kiểm tra ảnh hưởng của khởi tạo, và phân tích Hessian tại nghiệm:

```
W_star = huan_luyen_l2(Zb_tr, Y_tr, eta=2.0, so_epoch=8000)
P = softmax(Zb_tr @ W_star)
g_mau = np.linalg.norm(Zb_tr, axis=1) * np.linalg.norm(P - Y_tr, axis=1)   # ||x_i|| * ||p_i - y_i||
dung = P.argmax(axis=1) == y_tr
print(f"so mau sai: {(~dung).sum()}/{len(y_tr)} | do lon gradient trung binh: dung {g_mau[dung].mean():.3f}, sai {g_mau[~dung].mean():.3f}")

W0s = {"A": np.array([[2, -1, 0], [1, 3, -2], [-3, 1, 2.]]), "B": np.array([[1, 1, 1], [-2, 0, 2], [2, -2, 0.]])}
Wc_star = W_star - W_star.mean(axis=1, keepdims=True)          # nghiệm có các hàng cộng lại bằng 0
for ten, W0 in W0s.items():
    Wf = huan_luyen_l2(Zb_tr, Y_tr, eta=2.0, so_epoch=8000, W0=W0)
    print(f"{ten}: J0 = {mat_mat(Zb_tr, Y_tr, W0):.3f} -> J = {mat_mat(Zb_tr, Y_tr, Wf):.6f}"
          f" | max|P - P*| = {np.abs(softmax(Zb_tr @ Wf) - P).max():.1e}"
          f" | max|W - (W*_tam + trung binh hang cua W0)| = {np.abs(Wf - (Wc_star + W0.mean(axis=1, keepdims=True))).max():.1e}")

H = hessian(Zb_tr, W_star)
print("tri rieng:", np.linalg.eigvalsh(H))
v = np.array([0.3, -1.2, 0.7]); Zv = np.outer(v, np.ones(3))
print("|| H vec(v 1^T) || =", np.linalg.norm(H @ Zv.T.ravel()))
print("trung binh P =", P.mean(axis=0), "| ti le lop =", Y_tr.mean(axis=0))
print("max|X^T (P - Y)| / n =", np.abs(Zb_tr.T @ (P - Y_tr)).max() / len(y_tr))
```

```
so mau sai: 58/420 | do lon gradient trung binh: dung 0.280, sai 1.423
A: J0 = 6.461 -> J = 0.373000 | max|P - P*| = 2.0e-15 | max|W - (W*_tam + trung binh hang cua W0)| = 3.0e-14
B: J0 = 0.718 -> J = 0.373000 | max|P - P*| = 5.6e-16 | max|W - (W*_tam + trung binh hang cua W0)| = 9.8e-15
tri rieng: [-0.     -0.      0.      0.0058  0.0135  0.0295  0.0738  0.1333  0.2286]
|| H vec(v 1^T) || = 8.413804089098481e-17
trung binh P = [0.3333 0.3333 0.3333] | ti le lop = [0.3333 0.3333 0.3333]
max|X^T (P - Y)| / n = 2.1712457629953243e-16
```

Kết quả khớp với phân tích: ba trị riêng bằng 0, các hướng $\mathbf{v}\mathbf{1}_K^T$ nằm trong nhân của $\mathbf{H}$, ma trận $\mathbf{W}$ cuối từ mọi khởi tạo bằng $\mathbf{W}_{\text{tâm}}^* + \mathbf{v}_0\mathbf{1}_K^T$, và trung bình xác suất dự đoán bằng tỉ lệ lớp.

So sánh với Logistic Regression

	Logistic Regression	Softmax Regression
Tham số	vector $\mathbf{w}$ ($d+1$ phần tử)	ma trận $\mathbf{W}$ ($(d+1)\times K$)

	Logistic Regression	Softmax Regression
Hessian	$\frac{1}{n}\mathbf{X}^T\mathbf{S}\mathbf{X}$, xác định dương nếu $\mathbf{X}$ hạng đầy đủ	$\frac{1}{n}\sum_i \mathbf{A}_i \otimes \mathbf{x}_i^T\mathbf{x}_i$, nửa xác định dương, **suy biến** ($d+1$ hướng phẳng)
Tính lồi	lồi	lồi
Nghiệm	duy nhất	duy nhất theo $\mathbf{P}$, nhưng $\mathbf{W}$ xác định sai khác $\mathbf{v}\mathbf{1}_K^T$
Kết quả theo khởi tạo	không phụ thuộc	$\mathbf{P}$ không phụ thuộc, $\mathbf{W}$ phụ thuộc qua $\mathbf{v}_0 = \mathbf{W}^{(0)}\mathbf{1}_K/K$
Điều kiện ổn định	$\eta < 2/\lambda_{\max}$	$\eta < 2/\lambda_{\max}$ (trên các trị riêng dương)

Như vậy, việc có nhiều hơn hai lớp không làm mất tính lồi, nhưng đổi tính "duy nhất" từ $\mathbf{W}$ sang $\mathbf{P}$: đây là cái giá của tính dư thừa tham số đã nêu ở mục 2.5.

4.4. Dữ liệu phân tách hoàn hảo và vai trò của điều chuẩn hóa L2

Hiện tượng

Dữ liệu 9 sinh viên ở mục 1.2 phân tách tuyến tính được hoàn toàn theo nghĩa nhiều lớp. Với Perceptron nhiều lớp, đó là tình huống thuận lợi. Với Softmax Regression, giống Logistic Regression, đó lại là tình huống "bất thường". Gọi **lề** (margin) của mẫu i là khoảng cách giữa điểm số của lớp thật và điểm số lớn nhất của các lớp còn lại:

$$m_i = s_{i,y_i} - \max_{j \neq y_i} s_{ij}$$

Nếu tồn tại $\mathbf{W}^\circ$ phân loại đúng mọi mẫu, tức mọi $m_i > 0$, thì nhân $\mathbf{W}^\circ$ với một hệ số $c > 0$ làm mọi lề nhân với c, và mất mát một mẫu trở thành

$$\ell_i(c\mathbf{W}^\circ) = \ln\left(1 + \sum_{j \neq y_i} e^{-c\left(s_{i,y_i} - s_{ij}\right)}\right) \leq \ln(1 + (K-1)e^{-c\,m_i}) \underset{c\to\infty}{\to} 0$$

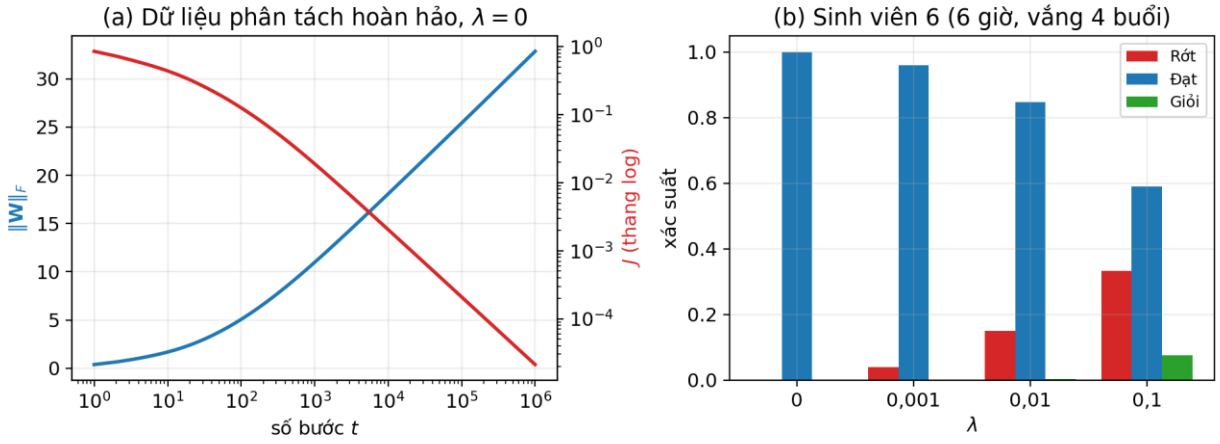
Do đó $J(c\mathbf{W}^\circ) \to 0$ khi $c \to \infty$, nhưng không bao giờ bằng 0 (vì $\ell_i > 0$ với mọi c). Không tồn tại cực tiểu hữu hạn: muốn J nhỏ hơn nữa thì $\mathbf{W}$ phải lớn hơn nữa. Tính lồi vẫn đúng, nhưng "đáy thung lũng" nằm ở vô cực.

Ta kiểm chứng bằng cách chạy Gradient Descent ($\eta = 0{,}5$, đặc trưng chuẩn hóa) trên 9 sinh viên rất lâu. Bảng ghi độ lớn $\|\mathbf{W}\|_F$ (căn bậc hai của tổng bình phương các phần tử), J và lề nhỏ nhất $\min_i m_i$:

Số bước t	$\|\mathbf{W}\|_F$	J	$\min_i m_i$
10	1,68	$4{,}4 \cdot 10^{-1}$	$-0{,}128$
100	5,06	$1{,}3 \cdot 10^{-1}$	1,056
1000	10,98	$1{,}9 \cdot 10^{-2}$	2,911

Số bước t	$\| \mathbf{W} \|_F$	J	$\min_i m_i$
10^4	18,07	$2{,}0 \cdot 10^{-3}$	5,173
10^5	25,42	$2{,}1 \cdot 10^{-4}$	7,477

Ở bước 10, lề nhỏ nhất còn âm, nghĩa là còn mẫu bị sai; từ bước 100 mọi mẫu đã đúng nhưng thuật toán **không dừng**. $\| \mathbf{W} \|_F$ tăng đều mỗi khi số bước tăng 10 lần (khoảng 7 đơn vị, tức tăng cỡ logarit của t), còn J giảm cỡ $1/t$ nhưng không bao giờ đạt 0 (Hình 12a). Các hàng của **W** vẫn cộng lại bằng 0 ở mọi bước, đúng tính chất ở mục 3.3.



Hình 12. (a) Trên dữ liệu phân tách hoàn hảo (9 sinh viên, $\lambda = 0$): $\| \boldsymbol{W} \|_F$ (đường xanh, trục trái) tăng không ngừng theo log của số bước, còn J (đường đỏ, trục phải) giảm dần về 0 mà không đạt. (b) Với điều chuẩn hóa L2, xác suất của sinh viên 6 (6 giờ học, vắng 4 buổi, sát ranh giới) trở nên dè dặt hơn khi λ tăng.

Hệ quả

- **Xác suất cực đoan và quá tự tin.** Khi **W** lớn, softmax rất "nhọn": mọi mẫu, kể cả mẫu sát đường biên, bị gán xác suất gần 0 hoặc gần 1. Ví dụ sinh viên 6 (6 giờ học, vắng 4 buổi, nằm sát ranh giới giữa rớt và đạt) được gán xác suất lớp đạt gần bằng 1,000 sau 10^5 bước, trong khi ở ma trận chọn tay ở mục 2.2 xác suất này chỉ 0,618.
- **Không có điểm dừng tự nhiên.** Thuật toán chỉ dừng vì $T_{\max}$ hoặc ngưỡng ε ta đặt. Dừng sớm vô hình trung đóng vai trò điều chuẩn hóa.
- **Kết quả phụ thuộc vào điều kiện dừng và bộ giải**, vì không có nghiệm hữu hạn để các phương pháp cùng hướng tới.

Với nhiều lớp, tình huống này còn dễ gặp hơn khi số đặc trưng lớn so với số mẫu hoặc khi ta thêm đặc trưng đa thức (mục 4.5).

Điều chuẩn hóa L2

Giải pháp phổ biến giống Logistic Regression: phạt độ lớn của các trọng số. Gọi $\widetilde{\mathbf{W}}$ là ma trận $\mathbf{W}$ bỏ hàng đầu (hàng hệ số chệch), tức gồm các hàng của d đặc trưng. Hàm mất mát điều chuẩn hóa L2 là

$$J_\lambda(\mathbf{W}) = J(\mathbf{W}) + \frac{\lambda}{2} \parallel \widetilde{\mathbf{W}} \parallel_F^2 = J(\mathbf{W}) + \frac{\lambda}{2}\mathrm{tr}(\mathbf{W}^T\mathbf{D}_0\mathbf{W}), \qquad \mathbf{D}_0 = \mathrm{diag}(0,1,1,\dots,1)$$

với $\lambda \geq 0$ là hệ số điều chuẩn hóa; phần tử đầu của $\mathbf{D}_0$ bằng 0 để không phạt hệ số chệch. Gradient và quy tắc cập nhật trở thành

$$\nabla_{\mathbf{W}} J_\lambda = \frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y}) + \lambda\,\mathbf{D}_0\mathbf{W}, \qquad \mathbf{W} \leftarrow (\mathbf{I} - \eta\lambda\mathbf{D}_0)\,\mathbf{W} - \frac{\eta}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$$

Dạng thứ hai cho thấy mỗi bước, các trọng số bị co lại một hệ số $(1 - \eta\lambda)$ trước khi cộng phần gradient, nên còn gọi là *weight decay*. Số hạng $\lambda\mathbf{D}_0\mathbf{W}$ luôn kéo $\widetilde{\mathbf{W}}$ về $\mathbf{0}$, chống lại xu hướng tăng vô hạn. Ba tác dụng:

- **Nghiệm hữu hạn.** J_λ vẫn lồi và giờ có cực tiểu hữu hạn ngay cả khi dữ liệu phân tách hoàn hảo, vì số hạng phạt tăng nhanh hơn phần lợi từ việc giảm J khi $\parallel \widetilde{\mathbf{W}} \parallel$ rất lớn.
- **Giải quyết một phần tính dư thừa.** Xét họ nghiệm $\mathbf{W} + \mathbf{v}\mathbf{1}_K^T$. Số hạng phạt của họ này là $\frac{\lambda}{2}\sum_k \parallel \widetilde{\mathbf{w}}_k + \tilde{\mathbf{v}} \parallel^2$, nhỏ nhất khi $\tilde{\mathbf{v}} = -\frac{1}{K}\sum_k \widetilde{\mathbf{w}}_k$, tức khi các cột của $\widetilde{\mathbf{W}}$ có **trung bình bằng** 0. Vậy L2 chọn đúng nghiệm "đã căn giữa" mà ta nhắc ở mục 2.5, và Hessian $\mathbf{H} + \lambda(\mathbf{I}_K \otimes \mathbf{D}_0)$ không còn suy biến theo các hướng $\tilde{\mathbf{v}}\mathbf{1}_K^T$. Riêng hàng hệ số chệch không bị phạt nên vẫn còn một hướng phẳng (cộng cùng một hằng số vào điểm số mọi lớp), vô hại và được cố định bởi khởi tạo $\mathbf{0}$.
- **Xác suất bớt cực đoan.** Ta chạy trên 9 sinh viên ($\eta = 0{,}5$, 10^5 bước, đặc trưng chuẩn hóa), với $\parallel \widetilde{\mathbf{W}} \parallel_F$ và xác suất của sinh viên 6:

λ	$\parallel \widetilde{\mathbf{W}} \parallel_F$	**p** của sinh viên 6 (rớt, đạt, giỏi)
0	23,00 (và vẫn tăng)	[0,000 1,000 0,000]
0,001	9,08	[0,040 0,960 0,000]
0,01	4,68	[0,150 0,846 0,004]
0,1	1,81	[0,334 0,589 0,077]

Khi $\lambda > 0$, $\parallel \widetilde{\mathbf{W}} \parallel_F$ hữu hạn và ổn định, và λ càng lớn thì $\widetilde{\mathbf{W}}$ càng nhỏ và xác suất càng dè dặt (Hình 12b): với $\lambda = 0{,}1$ xác suất của sinh viên 6 là $[0{,}334\ \ 0{,}589\ \ 0{,}077]$, gần với ma trận chọn tay ở mục 2.2 ($0{,}618$ cho lớp đạt).

Đánh đổi khi chọn λ

Trên dữ liệu 600 sinh viên (không phân tách hoàn hảo, chồng lấn vừa phải), điều chuẩn hóa không cần thiết để có nghiệm hữu hạn, nhưng ta vẫn thấy rõ đánh đổi. Bảng sau huấn luyện batch ($\eta = 1$, 8000 epoch) với các λ khác nhau và đo trên tập kiểm tra 180 mẫu (log-loss là J tính trên tập kiểm tra, sẽ được dùng ở mục 5.4):

λ	$\|\ \widetilde{\mathbf{W}}\ \|_F$	Accuracy huấn luyện	Accuracy kiểm tra	Log-loss kiểm tra
0	6,26	86,2%	85,0%	0,354
0,001	5,61	86,2%	85,6%	0,358
0,01	3,66	85,7%	85,0%	0,407
0,1	1,54	82,4%	80,0%	0,591
1	0,43	74,1%	73,3%	0,884

Với λ rất nhỏ, kết quả hầu như không đổi. Khi λ lớn, mô hình bị "bó" và khớp dữ liệu kém: độ chính xác giảm và log-loss tăng (xác suất bị kéo về $1/3$). Giá trị λ phù hợp thường được chọn bằng cách thử trên một tập kiểm định riêng (mục 5). Bảng này tạo bằng vòng lặp *`for lam in (0, 0.001, 0.01, 0.1, 1)`* gọi *`huan_luyen_l2(Zb_tr, Y_tr, eta=1.0, so_epoch=8000, lam=lam)`* rồi đo trên *`Zb_tr`*, *`Zb_te`*.

> **Liên hệ với scikit-learn.** *`LogisticRegression`* mặc định đã dùng điều chuẩn hóa L2 với tham số C, hàm mục tiêu của thư viện là $C\sum_i \ell_i + \frac{1}{2}\|\ \widetilde{\mathbf{W}}\ \|_F^2$ (không phạt hệ số chệch), tương đương $\lambda = \frac{1}{Cn}$ trong cách viết ở trên. C càng nhỏ thì điều chuẩn hóa càng mạnh. Ta sẽ gặp lại ở mục 5.5.

Các kết quả ở mục này cũng nối với mục 4.3: cả điều chuẩn hóa L2 lẫn dừng sớm đều dùng để tránh $\mathbf{W} \to \infty$, nhưng chỉ L2 cho một nghiệm hữu hạn, có định nghĩa rõ ràng và (với hàng chệch cố định bởi khởi tạo) duy nhất.

Đoạn mã sau tái hiện các bảng cho 9 sinh viên:

```
X_sv = np.array([[2, 4], [3, 3], [4, 5], [6, 3], [7, 2], [6, 4], [9, 1], [10,
0], [8, 0]], dtype=float)
y_sv = np.array([0, 0, 0, 1, 1, 1, 2, 2, 2]); Y_sv = one_hot(y_sv, 3)
Z_sv = them_bias((X_sv - X_sv.mean(axis=0)) / X_sv.std(axis=0))

def le_nho_nhat(Xb, y, W):
    S = Xb @ W; m = []
    for i in range(len(y)): m.append(S[i, y[i]] - np.delete(S[i], y[i]).max()
)
    return min(m)

W = np.zeros((3, 3))
for t in range(1, 10**5 + 1):
    W = W - 0.5 * gradient(Z_sv, Y_sv, W)
    if t in (10, 100, 1000, 10**4, 10**5):
        print(f"t = {t:6d} | ||W||_F = {np.linalg.norm(W):6.3f} | J = {mat_ma
```

```
t(Z_sv, Y_sv, W):.2e} | le nho nhat = {le_nho_nhat(Z_sv, y_sv, W):6.3f}")

x6 = them_bias(((np.array([[6, 4.]]) - X_sv.mean(axis=0)) / X_sv.std(axis=0))
)     # sinh viên 6
for lam in (0.0, 0.001, 0.01, 0.1):
    W = huan_luyen_l2(Z_sv, Y_sv, eta=0.5, so_epoch=100000, lam=lam)
    print(f"lam = {lam:<6} ||W~||_F = {np.linalg.norm(W[1:]):6.3f} | p(sv 6)
= {softmax(x6 @ W)[0].round(3)}")
```

```
t =     10 | ||W||_F =  1.682 | J = 4.38e-01 | le nho nhat = -0.128
t =    100 | ||W||_F =  5.064 | J = 1.26e-01 | le nho nhat =  1.056
t =   1000 | ||W||_F = 10.977 | J = 1.89e-02 | le nho nhat =  2.911
t =  10000 | ||W||_F = 18.068 | J = 2.04e-03 | le nho nhat =  5.173
t = 100000 | ||W||_F = 25.415 | J = 2.09e-04 | le nho nhat =  7.477
lam = 0.0    ||W~||_F = 23.000 | p(sv 6) = [0. 1. 0.]
lam = 0.001  ||W~||_F =  9.084 | p(sv 6) = [0.04 0.96 0.  ]
lam = 0.01   ||W~||_F =  4.676 | p(sv 6) = [0.15  0.846 0.004]
lam = 0.1    ||W~||_F =  1.806 | p(sv 6) = [0.334 0.589 0.077]
```

4.5. Giới hạn của vùng quyết định tuyến tính và đặc trưng đa thức

Vùng quyết định của Softmax Regression luôn lồi

Ở mục 2.5, ta đã biết lớp k được chọn khi $s_k \geq s_j$ với mọi $j \neq k$, tức khi $\mathbf{x}\mathbf{w}_k \geq \mathbf{x}\mathbf{w}_j$. Mỗi bất đẳng thức $\mathbf{x}(\mathbf{w}_k - \mathbf{w}_j) \geq 0$ là một nửa không gian, nên vùng của lớp k là giao của $K-1$ nửa không gian:

$$\mathcal{R}_k = \bigcap_{j \neq k} \{\mathbf{x}: \mathbf{x}(\mathbf{w}_k - \mathbf{w}_j) \geq 0\}$$

Giao của các tập lồi là tập lồi. Có thể thấy trực tiếp: nếu $\mathbf{x}_a, \mathbf{x}_b \in \mathcal{R}_k$ và $\mathbf{x} = t\mathbf{x}_a + (1-t)\mathbf{x}_b$ với $t \in [0,1]$ thì $\mathbf{x}(\mathbf{w}_k - \mathbf{w}_j) = t\,\mathbf{x}_a(\mathbf{w}_k - \mathbf{w}_j) + (1-t)\,\mathbf{x}_b(\mathbf{w}_k - \mathbf{w}_j) \geq 0$ với mọi j, tức $\mathbf{x} \in \mathcal{R}_k$. Vậy **mọi vùng quyết định của Softmax Regression đều lồi** (và các vùng còn là đa diện, ghép với nhau tại các đường biên thẳng). Do đó mô hình không thể phân loại đúng khi một lớp có hình dạng không lồi, ví dụ:

- một lớp nằm ở giữa, được bao quanh bởi lớp khác (như một chiếc nhẫn);
- một lớp gồm hai nhóm tách rời nhau, ở giữa là lớp khác (như hai phía của một dải);
- một lớp có hình lưỡi liềm hoặc hình chữ "C".

Đây là phiên bản nhiều lớp của hạn chế "không giải được XOR" của Logistic Regression và Perceptron. Dữ liệu sinh viên ở các mục trước thuận lợi vì ba lớp (rớt, đạt, giỏi) xếp thành ba dải gần song song từ trái sang phải, nên mỗi lớp gần như lồi.

Ví dụ: ba vòng tròn đồng tâm

Ta tạo bộ dữ liệu 600 điểm trong mặt phẳng, 200 điểm mỗi lớp, nằm trên ba vòng tròn đồng tâm bán kính 1, 3 và 5 (có nhiễu). Lớp 0 là hình tròn ở giữa, lớp 1 là vòng ở giữa (không lồi: nó vòng quanh lớp 0), lớp 2 là vòng ngoài cùng. Ta cũng chia 70% huấn luyện và 30% kiểm tra như trước. Mô hình tuyến tính có hai đặc trưng x_1, x_2 cho kết quả ở Hình 13a:

- Độ chính xác chỉ 36,2% trên tập huấn luyện và 35,0% trên tập kiểm tra, cỡ ngẫu nhiên (33,3%).
- $J = 1{,}0964$ chỉ thấp hơn $\ln 3 = 1{,}0986$ chút ít, và $\| \widetilde{\mathbf{W}} \|_F \approx 0{,}11$, rất gần $\mathbf{0}$. Mô hình "nhận ra mình không biết gì" và gán xác suất gần $\frac{1}{3}$ cho mọi lớp ở mọi điểm.

Lý do là đối xứng quay: các lớp đều có tâm ở gốc tọa độ nên trung bình của x_1 và x_2 trong mỗi lớp đều bằng 0, và gradient $\frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$ tại $\mathbf{W} = \mathbf{0}$ gần như bằng $\mathbf{0}$ (hàng chệch bằng 0 vì ba lớp cân bằng, hai hàng còn lại bằng 0 vì trung bình đặc trưng của mỗi lớp bằng 0). Điểm xuất phát $\mathbf{0}$ gần như đã là điểm dừng, và không có hướng tuyến tính nào cải thiện được J. Đây là hình ảnh tương tự của bộ dữ liệu XOR ở mục 4.5 bài Logistic Regression.

Giải pháp: thêm đặc trưng đa thức

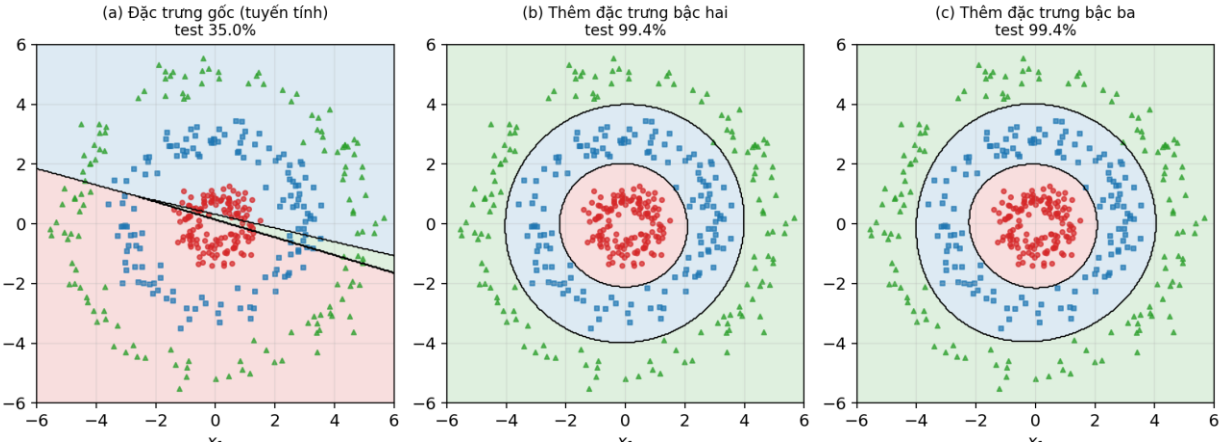
Chìa khóa là nhận ra rằng mô hình chỉ cần **tuyến tính theo tham số**, không nhất thiết tuyến tính theo dữ liệu gốc. Ta thêm các đặc trưng phi tuyến của $\mathbf{x}$, ví dụ với bậc 2:

$$\boldsymbol{\phi}(\mathbf{x}) = [\, x_1 \;\; x_2 \;\; x_1^2 \;\; x_1 x_2 \;\; x_2^2 \,]$$

rồi chạy Softmax Regression trên $\boldsymbol{\phi}(\mathbf{x})$ thay cho $\mathbf{x}$ (mọi công thức ở mục 3 giữ nguyên; chỉ cần thay ma trận $\mathbf{X}$ bằng ma trận đặc trưng mới, rồi chuẩn hóa như thường lệ). Khi đó điểm số của lớp k là một đa thức bậc hai theo (x_1, x_2):

$$s_k = w_{0k} + w_{1k} x_1 + w_{2k} x_2 + w_{3k} x_1^2 + w_{4k} x_1 x_2 + w_{5k} x_2^2$$

và đường biên giữa hai lớp ($s_k = s_j$) là một đường conic (đường tròn, ellipse, parabol hoặc hyperbol). Với dữ liệu vòng tròn đồng tâm, hiệu hai điểm số $s_k - s_j$ chỉ cần phụ thuộc vào $r^2 = x_1^2 + x_2^2$, nên các biên là những đường tròn đồng tâm. Hình 13b và 13c cho thấy đúng như vậy.



Hình 13. Softmax Regression trên dữ liệu ba vòng tròn đồng tâm. (a) Đặc trưng gốc: các vùng gần như suy biến, không phân loại được (vùng quyết định gần như một đường thẳng vì $\boldsymbol{W} \approx \mathbf{0}$). (b) Thêm đặc trưng bậc hai: các đường biên là hai đường tròn. (c) Thêm đặc trưng bậc ba: kết quả gần như không đổi so với bậc hai. Ba mô hình đều dùng $\lambda = 10^{-3}$.

Kết quả đo trên tập kiểm tra 180 mẫu (đặc trưng chuẩn hóa, batch Gradient Descent):

Mô hình	Số tham số	J (huấn luyện)	Acc. huấn luyện	Acc. kiểm tra	$\| \widetilde{\mathbf{W}} \|_F$
Tuyến tính	9	1,0964	36,2%	35,0%	0,11
Bậc 2, $\lambda = 0$	18	0,0085	100,0%	99,4%	22,12
Bậc 2, $\lambda = 10^{-3}$	18	0,0543	99,8%	99,4%	11,15
Bậc 3, $\lambda = 10^{-3}$	30	0,0538	100,0%	99,4%	11,13

Độ chính xác nhảy từ 35,0% lên 99,4% chỉ nhờ thêm ba đặc trưng bậc hai, trong khi thuật toán huấn luyện không đổi. Hai chi tiết đáng chú ý:

- Với $\lambda = 0$ và đặc trưng bậc hai, độ chính xác huấn luyện đạt 100% và J rất nhỏ (0,0085) nhưng $\| \widetilde{\mathbf{W}} \|_F = 22{,}12$ lớn hơn gấp đôi so với khi dùng $\lambda = 10^{-3}$ (11,15). Đây chính là hiện tượng ở mục 4.4: sau khi thêm đặc trưng, dữ liệu trở nên phân tách (gần) hoàn hảo, và L2 giữ cho các trọng số không phình ra.
- Bậc 3 không tốt hơn bậc 2 trên bộ dữ liệu này (cùng 99,4%), nhưng tốn thêm tham số. Biên thật là đường tròn nên bậc 2 đã đủ.

Cái giá của đặc trưng đa thức

Với d đặc trưng gốc và bậc tối đa k, số đặc trưng đa thức (không tính cột 1) là $\binom{d+k}{k} - 1$, và số tham số của Softmax Regression là $K\binom{d+k}{k}$ (gồm cả hệ số chệch mỗi lớp):

d	Bậc k	K	Số đặc trưng mới	Số tham số
2	2	3	5	18

d	Bậc k	K	Số đặc trưng mới	Số tham số
2	3	3	9	30
10	2	10	65	660
10	3	10	285	2860

Số tham số tăng rất nhanh theo d và k, cùng với nguy cơ khớp quá mức và phân tách hoàn hảo (mục 4.4), nên cần điều chuẩn hóa L2 và chuẩn hóa đặc trưng (các cột x_1^2, x_1x_2,… có thang đo rất khác nhau). Hơn nữa, ta phải tự đoán dạng đặc trưng phù hợp (ở đây ta "biết" biên là vòng tròn), điều không dễ khi dữ liệu có nhiều chiều. Hướng thoát là để mô hình tự học các đặc trưng phi tuyến, chính là ý tưởng của mạng nơ-ron nhiều lớp mà ta sẽ nhắc ở mục 6.3: Softmax Regression chính là lớp đầu ra của các mạng đó.

Mã Python

Các hàm sinh dữ liệu vòng tròn và tạo đặc trưng đa thức:

```
from itertools import combinations_with_replacement

def sinh_vong(seed=7, n_lop=200):
    """Ba vòng tròn đồng tâm (bán kính 1; 3; 5), mỗi lớp 200 điểm."""
    rng = np.random.default_rng(seed)
    r = np.r_[rng.normal(1.0, 0.35, n_lop), rng.normal(3.0, 0.35, n_lop), rng
.normal(5.0, 0.35, n_lop)]
    g = rng.uniform(0, 2 * np.pi, 3 * n_lop)
    return np.c_[r * np.cos(g), r * np.sin(g)], np.repeat([0, 1, 2], n_lop)

def da_thuc(X, bac=2):
    """Đặc trưng đa thức bậc 1..bac (không gồm cột 1): x1, x2, x1^2, x1*x2, x
2^2, ..."""
    cot = []
    for k in range(1, bac + 1):
        for idx in combinations_with_replacement(range(X.shape[1]), k):
            cot.append(np.prod(X[:, idx], axis=1))
    return np.column_stack(cot)

Xv, yv = sinh_vong()
Xa, Xb, ya, yb = chia_train_test(Xv, yv)
Ya = one_hot(ya, 3)
for ten, bac, lam in [("tuyen tinh", 1, 0.0), ("bac 2, lam = 0", 2, 0.0), ("b
ac 2, lam = 1e-3", 2, 1e-3), ("bac 3, lam = 1e-3", 3, 1e-3)]:
    Fa, Fb = da_thuc(Xa, bac), da_thuc(Xb, bac)
    Za, Zb, _, _ = chuan_hoa(Fa, Fb)
    Za, Zb = them_bias(Za), them_bias(Zb)
    W = huan_luyen_l2(Za, Ya, eta=1.0 if bac < 3 else 0.5, so_epoch=5000, lam
=lam)
    print(f"{ten:18s} so tham so = {W.size:2d} | J = {mat_mat(Za, Ya, W):.4f}
| train {np.mean(du_doan(Za, W) == ya):.3f}"
```

```
          f" | test {np.mean(du_doan(Zb, W) == yb):.3f} | ||W~||_F = {np.lina
lg.norm(W[1:]):.2f}")
```

```
tuyen tinh         so tham so =  9 | J = 1.0964 | train 0.362 | test 0.350 |
||W~||_F = 0.11
bac 2, lam = 0     so tham so = 18 | J = 0.0085 | train 1.000 | test 0.994 |
||W~||_F = 22.12
bac 2, lam = 1e-3  so tham so = 18 | J = 0.0543 | train 0.998 | test 0.994 |
||W~||_F = 11.15
bac 3, lam = 1e-3  so tham so = 30 | J = 0.0538 | train 1.000 | test 0.994 |
||W~||_F = 11.13
```

Tóm tắt mục 4

- **Mẫu sai và cơ chế kéo-đẩy.** Mức đóng góp của mẫu i vào gradient là $\| \mathbf{x}_i \| \cdot \| \mathbf{p}_i - \mathbf{y}_i \|$ (với $\| \mathbf{p}_i - \mathbf{y}_i \| < \sqrt{2}$). Một bước SGD đổi điểm số của chính mẫu đó thêm $\eta \| \mathbf{x}_i \|^2 (\mathbf{y}_i - \mathbf{p}_i)$: lớp đúng tăng theo $(1 - p_{ic})$, mỗi lớp sai giảm theo p_{ik}, tổng thay đổi bằng 0. Đây là phiên bản "mềm" và có trọng số của Perceptron nhiều lớp.

- **Khởi tạo, tốc độ học, chuẩn hóa.** Mọi khởi tạo đều cho cùng J và cùng $\mathbf{P}$, nhưng $\mathbf{W}$ cuối khác nhau theo công thức $\mathbf{W}^{(\infty)} = \mathbf{W}^{*}_{\text{tâm}} + \mathbf{v}_0 \mathbf{1}_K^T$ với $\mathbf{v}_0 = \mathbf{W}^{(0)} \mathbf{1}_K / K$. Chuẩn hóa đặc trưng làm hội tụ nhanh hơn và cho phép dùng η lớn hơn nhiều (trên dữ liệu 600 sinh viên: 45 epoch so với 8170); chuyển ngược về thang đo gốc bằng $w_{jk} = w_{jk}'/\sigma_j$ và $w_{0k} = w_{0k}' - \sum_j \mu_j w_{jk}'/\sigma_j$.

- **Lồi nhưng suy biến.** Hessian $\mathbf{H} = \frac{1}{n} \sum_i \mathbf{A}_i \otimes \mathbf{x}_i^T \mathbf{x}_i$ nửa xác định dương (dạng toàn phương là phương sai), nên J lồi; nhưng có đúng $d + 1$ trị riêng bằng 0 ứng với các hướng $\mathbf{v}\mathbf{1}_K^T$, nên $\mathbf{W}$ không duy nhất dù $\mathbf{P}$ duy nhất. Gradient Descent không đi theo các hướng này. Gần nghiệm, thuật toán ổn định khi $\eta < 2/\lambda_{\max}(\mathbf{H})$, và chuẩn hóa giảm số điều kiện từ khoảng 4880 xuống 39.

- **Dữ liệu phân tách hoàn hảo.** $J \to 0$ khi $\| \mathbf{W} \| \to \infty$ nên không có nghiệm hữu hạn, xác suất trở nên cực đoan. Điều chuẩn hóa L2, $J_\lambda = J + \frac{\lambda}{2} \| \widetilde{\mathbf{W}} \|_F^2$ (cập nhật $\mathbf{W} \leftarrow (\mathbf{I} - \eta\lambda \mathbf{D}_0)\mathbf{W} - \eta \nabla J$), cho nghiệm hữu hạn, chọn nghiệm có các cột trung bình bằng 0 và làm xác suất dè dặt hơn; λ lớn quá thì mô hình khớp kém.

- **Giới hạn tuyến tính.** Vùng quyết định luôn là tập lồi (giao của các nửa không gian) nên mô hình không phân loại được các lớp lồng nhau (ví dụ ba vòng tròn đồng tâm: chỉ 35,0% độ chính xác kiểm tra). Thêm đặc trưng đa thức đưa độ chính xác lên 99,4% với chỉ 18 tham số, nhưng số tham số tăng rất nhanh theo d và bậc; hướng tự nhiên tiếp theo là mạng nơ-ron nhiều lớp.

Câu hỏi kiểm tra nhanh

1. Cho $K = 3$, một mẫu có $\mathbf{p} = [0{,}1 \;\; 0{,}7 \;\; 0{,}2]$, lớp thật là lớp 0, $\| \mathbf{x} \|^2 = 10$. Thực hiện một bước SGD với $\eta = 0{,}05$. Tính độ thay đổi của từng điểm số và xác suất mới (biết điểm số ban đầu là $\mathbf{s} = \ln \mathbf{p}$).

2. Hai mẫu cùng $\| \mathbf{x} \|$ có lớp thật là lớp 1: mẫu A có $\mathbf{p} = [0{,}05 \;\; 0{,}90 \;\; 0{,}05]$, mẫu B có $\mathbf{p} = [0{,}5 \;\; 0{,}4 \;\; 0{,}1]$. Gradient của mẫu nào lớn hơn, và lớn hơn bao nhiêu lần?

3. Chứng minh $\| \mathbf{p} - \mathbf{y} \| < \sqrt{2}$ với mọi mẫu, với $\mathbf{y}$ one-hot và $\mathbf{p}$ có mọi phần tử dương.

4. Một mô hình có $d = 5$ đặc trưng và $K = 4$ lớp, ma trận $\mathbf{X}$ có hạng đầy đủ. Hessian có kích thước bao nhiêu, hạng bao nhiêu, và có bao nhiêu trị riêng bằng 0?

5. Nghiệm thu được từ khởi tạo $\mathbf{W} = \mathbf{0}$ có hàng hệ số chệch là $[-2 \;\; 0 \;\; 2]$. Nếu ta chạy lại từ khởi tạo có hàng hệ số chệch là $[1 \;\; 2 \;\; 6]$ (các hàng còn lại giữ nguyên bằng 0), hàng hệ số chệch của nghiệm mới là gì? Xác suất dự đoán có đổi không?

6. Một mẫu phân loại đúng trong bài toán $K = 3$, có lề $m = s_{i,y_i} - s_{ij} = 2$ với cả hai lớp sai. Tính mất mát khi nhân $\mathbf{W}$ với $c = 1, 2, 3$. Từ đó giải thích vì sao Gradient Descent không dừng khi dữ liệu phân tách hoàn hảo.

7. Với điều chuẩn hóa L2, $\eta = 0{,}5$ và $\lambda = 0{,}1$, nếu một đặc trưng có gradient dữ liệu bằng 0 thì sau 10 bước độ lớn trọng số của nó còn bao nhiêu phần so với ban đầu? Vì sao không phạt hàng hệ số chệch?

8. Mô hình có $d = 3$ đặc trưng, thêm các đặc trưng đa thức bậc tối đa 2, với $K = 4$ lớp. Tính số đặc trưng mới và tổng số tham số. Vì sao Softmax Regression tuyến tính không phân loại được dữ liệu ba lớp là ba vòng tròn đồng tâm?

Gợi ý đáp án

1. $\Delta\mathbf{s} = \eta \| \mathbf{x} \|^2 (\mathbf{y} - \mathbf{p}) = 0{,}5 \cdot [0{,}9 \;\; -0{,}7 \;\; -0{,}2] = [0{,}45 \;\; -0{,}35 \;\; -0{,}10]$. Điểm số mới là $\ln \mathbf{p} + \Delta\mathbf{s}$, nên $\mathbf{p}' \propto [0{,}1e^{0{,}45} \;\; 0{,}7e^{-0{,}35} \;\; 0{,}2e^{-0{,}10}]$, chuẩn hóa được $\mathbf{p}' \approx [0{,}189 \;\; 0{,}594 \;\; 0{,}218]$. Xác suất lớp đúng tăng từ 0,1 lên 0,189, mất mát giảm từ 2,303 xuống 1,668. Tổng ba thay đổi bằng 0.

2. $\| \mathbf{p} - \mathbf{y} \|$: mẫu A là $\sqrt{0{,}05^2 + 0{,}1^2 + 0{,}05^2} \approx 0{,}122$; mẫu B là $\sqrt{0{,}5^2 + 0{,}6^2 + 0{,}1^2} \approx 0{,}787$. Vì $\| \mathbf{x} \|$ như nhau, gradient của B lớn hơn A khoảng 6,4 lần.

3. $\| \mathbf{p} - \mathbf{y} \|^2 = (1 - p_c)^2 + \sum_{k \neq c} p_k^2$ với c là lớp thật. Vì $\sum_{k \neq c} p_k = 1 - p_c$ và các $p_k > 0$ nên $\sum_{k \neq c} p_k^2 < (1 - p_c)^2$. Suy ra $\| \mathbf{p} - \mathbf{y} \|^2 < 2(1 - p_c)^2 < 2$ (vì $p_c > 0$), nên $\| \mathbf{p} - \mathbf{y} \| < \sqrt{2}$.

4. $\mathbf{H}$ có kích thước $(d+1)K \times (d+1)K = 24 \times 24$, hạng $(K-1)(d+1) = 18$, và có $d + 1 = 6$ trị riêng bằng 0 (ứng với các hướng $\mathbf{v}\mathbf{1}_K^T$, $\mathbf{v} \in \mathbb{R}^6$).

5. Khởi tạo có hàng chệch trung bình $(1 + 2 + 6)/3 = 3$, nên $\mathbf{v}_0 = [3 \;\; 0 \;\; 0]^T$ và nghiệm mới có hàng chệch bằng $[-2 \;\; 0 \;\; 2] + 3 = [1 \;\; 3 \;\; 5]$, các hàng khác không đổi. Xác suất dự đoán **không đổi**, vì cộng cùng một hằng số 3 vào điểm số của mọi lớp không làm đổi softmax.

6. $\ell(c) = \ln(1 + 2e^{-2c})$: $\ell(1) \approx 0{,}240$, $\ell(2) \approx 0{,}036$, $\ell(3) \approx 0{,}0049$. Mất mát luôn giảm khi c tăng và không bao giờ bằng 0, nên gradient luôn khác 0 và thuật toán cứ tiếp tục làm tăng $\mathbf{W}$ (xu hướng $\| \mathbf{W} \| \to \infty$).

7. Chỉ còn phần gradient từ điều chuẩn hóa: mỗi bước nhân trọng số với $1 - \eta\lambda = 0{,}95$, nên sau 10 bước còn $0{,}95^{10} \approx 0{,}599$, tức khoảng 60%. Hệ số chệch không phạt vì nó chỉ dịch điểm số, không ảnh hưởng đến "độ dốc" hay độ phức tạp của mô hình; phạt nó sẽ làm mô hình bị kéo về dự đoán xác suất bằng nhau một cách không cần thiết.

8. Số đặc trưng mới là $\binom{3+2}{2} - 1 = 9$ (ba đặc trưng bậc một và sáu đặc trưng bậc hai), số tham số là $K \times (9 + 1) = 40$. Với ba vòng tròn đồng tâm, vùng quyết định của Softmax Regression tuyến tính phải là tập lồi, nhưng lớp ở giữa (vòng tròn thứ hai) là một hình vành khuyên, không lồi; đồng thời do đối xứng quay, mô hình tuyến tính gần như chỉ học được $\mathbf{W} \approx \mathbf{0}$.

5. Đánh giá và trực quan hóa mô hình

Ở mục 3 ta đã huấn luyện được $\mathbf{W}$, và ở mục 4 ta hiểu mô hình học thế nào và khi nào nó gặp khó. Câu hỏi còn lại là **mô hình tốt đến đâu**. Với Logistic Regression, ta đã có nhiều công cụ hơn Perceptron nhờ xác suất; với Softmax Regression, ta có cả bộ công cụ đó nhưng phải mở rộng cho K lớp: ma trận nhầm lẫn trở thành ma trận $K \times K$, precision và recall phải tính theo từng lớp rồi lấy trung bình, và chất lượng xác suất phải xét trên cả vector $\mathbf{p}_i$ chứ không chỉ một số.

Mục này gồm năm phần:

- Quan sát quá trình học và bản đồ xác suất của mô hình trên mặt phẳng đặc trưng.
- Đánh giá trên tập huấn luyện và tập kiểm tra, và nhận diện hiện tượng học thuộc (overfitting).
- Accuracy, ma trận nhầm lẫn $K \times K$, precision, recall, F1 theo từng lớp và các cách lấy trung bình; cách ra quyết định khi các loại sai có chi phí khác nhau.
- Đánh giá chất lượng xác suất: log-loss đa lớp, điểm Brier, top-k accuracy, hiệu chuẩn và AUC kiểu one-vs-rest.
- So sánh với `LogisticRegression` của scikit-learn, với one-vs-rest, và chọn siêu tham số bằng kiểm định chéo.

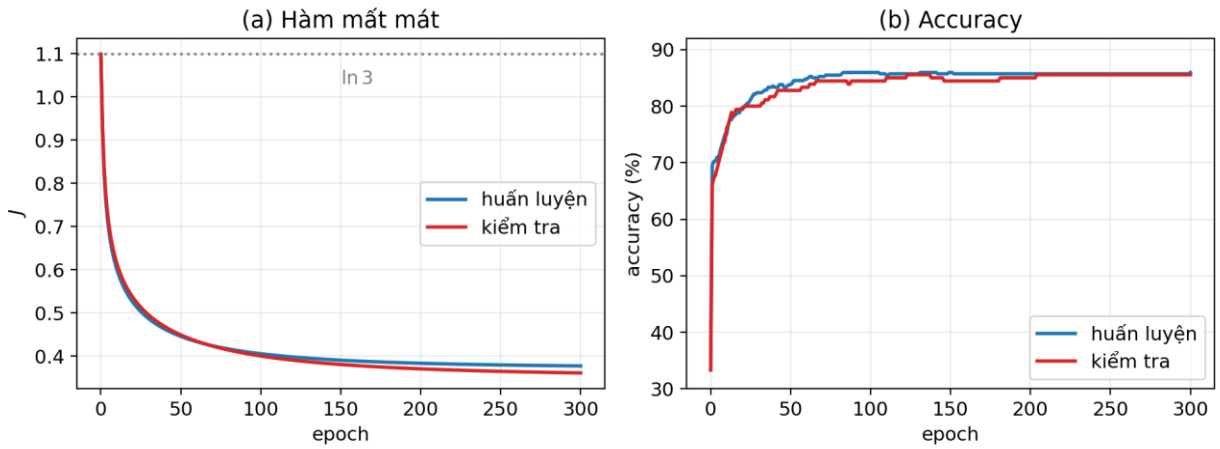
Ta tiếp tục dùng quy ước ký hiệu cũ: chữ thường in đậm là vector, chữ hoa in đậm là ma trận; $\mathbf{x}_i$, $\mathbf{p}_i$ và $\mathbf{y}_i$ là các vector hàng; $q_i = p_{i,y_i}$ là xác suất gán cho lớp thật; nhãn $y_i \in \{0,1,2\}$ ứng với rớt, đạt, giỏi. Toàn bộ mục này dùng bộ dữ liệu 600 sinh viên (420 mẫu huấn luyện, 180 mẫu kiểm tra, mỗi lớp 60 mẫu kiểm tra) và mô hình đã huấn luyện ở mục 3.7: batch Gradient Descent, $\eta = 0{,}5$, 2000 epoch, đặc trưng chuẩn hóa, với

$$\mathbf{W} = \begin{bmatrix} -1{,}290 & 1{,}803 & -0{,}513 \\ -4{,}376 & 0{,}251 & 4{,}125 \\ 1{,}369 & -0{,}410 & -0{,}958 \end{bmatrix}, \qquad J(\mathbf{W}) = 0{,}3730 \text{ (huấn luyện)}$$

5.1. Minh họa hàm mất mát giảm dần, vùng quyết định và bản đồ xác suất

Đường cong học

Cách đơn giản nhất để "nhìn thấy" quá trình học là vẽ hàm mất mát và độ chính xác theo từng epoch, trên cả hai tập. Hình 14 vẽ kết quả của batch Gradient Descent từ $\mathbf{W} = \mathbf{0}$ với $\eta = 0{,}5$ (ta chỉ tính J và accuracy trên tập kiểm tra để theo dõi, không dùng nó để cập nhật $\mathbf{W}$).



Hình 14. Đường cong học của Softmax Regression trên dữ liệu 600 sinh viên. (a) Hàm mất mát J trên tập huấn luyện và tập kiểm tra giảm từ $\ln 3$ rồi ổn định. (b) Accuracy tăng vọt trong khoảng 50 epoch đầu rồi gần như đi ngang.

Mã tính các giá trị ở epoch chọn trước (dùng lại các hàm ở mục 3.7 và 4.3):

```
Y_kt = one_hot(y_te, 3)
W = np.zeros((3, 3))
print("epoch | J train | J test | acc train | acc test")
for epoch in range(2001):
    if epoch in (0, 10, 50, 300, 2000):
        print(f"{epoch:5d} | {mat_mat(Zb_tr, Y_tr, W):.4f} | {mat_mat(Zb_te,
Y_kt, W):.4f} | "
              f"{np.mean(du_doan(Zb_tr, W) == y_tr):.4f} | {np.mean(du_doan(Z
b_te, W) == y_te):.4f}")
    W = W - 0.5 * gradient(Zb_tr, Y_tr, W)              # batch Gradient Des
cent, eta = 0.5
```

```
epoch | J train | J test | acc train | acc test
    0 | 1.0986 | 1.0986 | 0.3333 | 0.3333
   10 | 0.6001 | 0.6147 | 0.7500 | 0.7611
   50 | 0.4455 | 0.4489 | 0.8381 | 0.8278
  300 | 0.3772 | 0.3610 | 0.8595 | 0.8556
 2000 | 0.3730 | 0.3536 | 0.8619 | 0.8500
```

Có thể rút ra ba nhận xét:

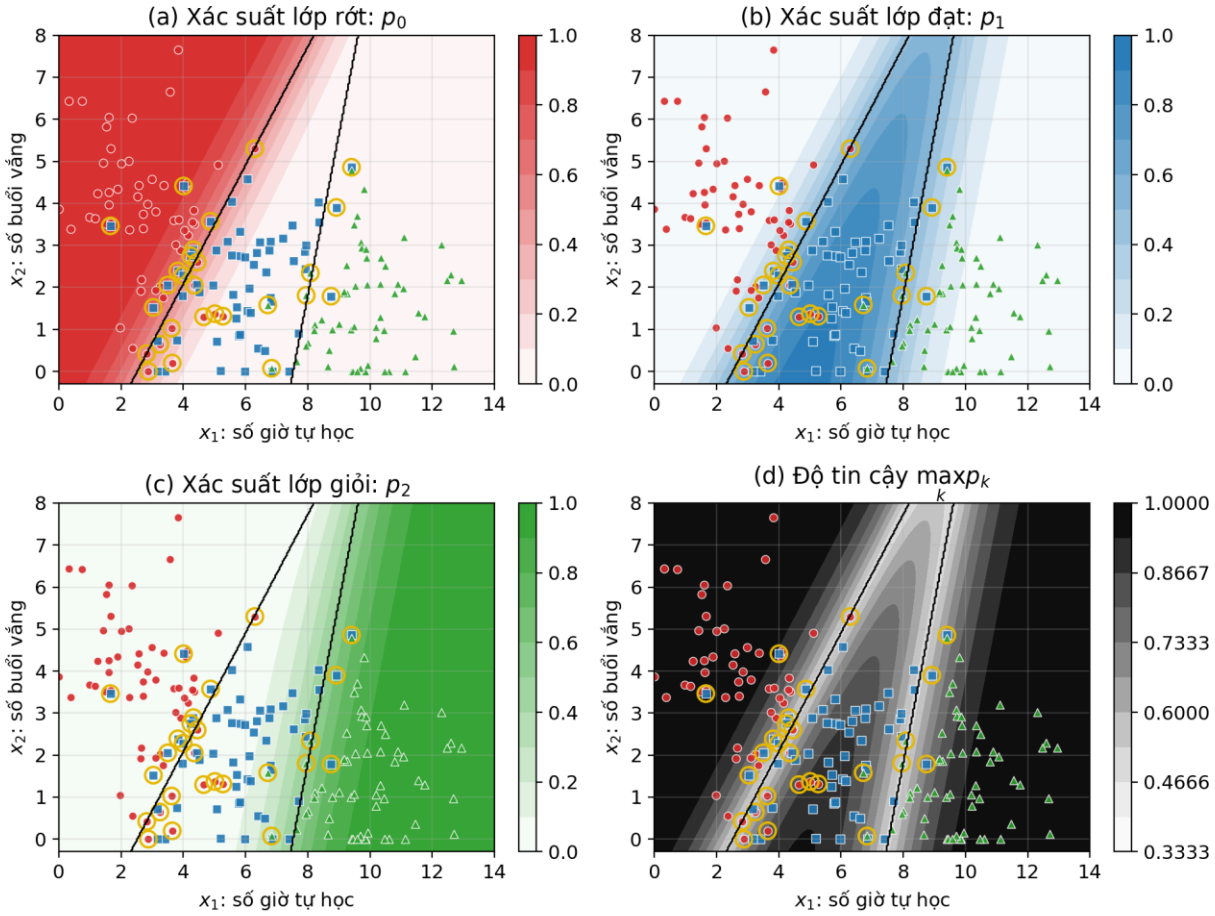
- Ở epoch 0, $\mathbf{W} = \mathbf{0}$ nên mọi $p_{ik} = \frac{1}{3}$ và $J = \ln 3 \approx 1{,}0986$ trên cả hai tập (đúng như mục 3.1). Accuracy lúc này bằng đúng $\frac{1}{3}$: khi cả ba xác suất bằng nhau, hàm *argmax* trả về lớp có chỉ số nhỏ nhất (lớp rớt), và mỗi tập có đúng một phần ba số mẫu thuộc lớp rớt ($140/420$ và $60/180$).
- J trên tập kiểm tra sau khi hội tụ lại **thấp hơn** J trên tập huấn luyện (0,354 so với 0,373), trái với trực giác "mô hình quen dữ liệu đã học hơn". Đây là dao động

ngẫu nhiên của việc chia dữ liệu (tập kiểm tra tình cờ dễ hơn một chút) và là lý do ta không nên đọc quá nhiều từ chênh lệch nhỏ giữa hai tập (xem mục 5.2).

- Accuracy bão hòa sớm hơn nhiều so với J: sau epoch 300, accuracy kiểm tra đi từ 85,6% xuống 85,0% (đúng một mẫu trên 180) trong khi J huấn luyện vẫn giảm từ 0,3772 xuống 0,3730. Như ở Logistic Regression, hàm mất mát phản ánh chi tiết hơn accuracy: sau khi nhãn đã đúng, mô hình vẫn đang cải thiện chất lượng của xác suất, điều mà accuracy không "thấy" được.

Vùng quyết định và bản đồ xác suất

Hình 15 vẽ, tại mọi điểm của mặt phẳng đặc trưng (x_1, x_2), xác suất của từng lớp p_0, p_1, p_2 (ba bản đồ đầu, màu càng đậm thì xác suất càng cao) và độ tin cậy $\max_k p_k$ (bản đồ cuối). Các đường nét liền là đường biên quyết định, nơi hai lớp có xác suất cao nhất bằng nhau. Các điểm là 180 mẫu kiểm tra, và 27 mẫu bị dự đoán sai được khoanh vàng.



Hình 15. Bản đồ xác suất của Softmax Regression trên mặt phẳng đặc trưng. (a), (b), (c): xác suất của lớp rớt, đạt, giỏi. (d): độ tin cậy $\max_k p_k$ (càng tối càng chắc chắn). Đường liền là biên quyết định; điểm to là mẫu kiểm tra, vòng vàng là mẫu bị dự đoán sai.

Bản đồ cho thấy một số điều gắn với những gì ta đã biết:

- **Biên là các đường thẳng, vùng là các dải lồi** (mục 2.5 và 4.5). Hai đường biên gần song song vì số giờ học x_1 là yếu tố chi phối.
- **Độ rộng của vùng "chưa chắc chắn".** Từ $\ln\frac{p_k}{p_j} = \mathbf{x}(\mathbf{w}_k - \mathbf{w}_j)$ (mục 2.5), các đường đồng mức của tỉ số p_k/p_j là những đường thẳng song song với biên, cách biên một khoảng $\ln r / \| \tilde{\mathbf{w}}_k - \tilde{\mathbf{w}}_j \|$ khi $p_k/p_j = r$ (đây là kết quả tương tự khoảng cách $\ln 9/\| \tilde{\mathbf{w}} \|$ ở Logistic Regression). Trên thang đo gốc, $\mathbf{W}_{\text{gốc}}$ cho $\| \tilde{\mathbf{w}}_0 - \tilde{\mathbf{w}}_1 \| \approx 1{,}825$ và $\| \tilde{\mathbf{w}}_1 - \tilde{\mathbf{w}}_2 \| \approx 1{,}288$. Do đó, để tỉ số xác suất hai lớp lên $9:1$, ta phải đi cách biên rớt/đạt khoảng $\ln 9/1{,}825 \approx 1{,}20$ đơn vị và cách biên đạt/giỏi khoảng $\ln 9/1{,}288 \approx 1{,}71$ đơn vị. Dải chưa chắc chắn giữa rớt và đạt hẹp hơn dải giữa đạt và giỏi, vì hiệu hai cột $\| \tilde{\mathbf{w}}_0 - \tilde{\mathbf{w}}_1 \|$ lớn hơn nên điểm số thay đổi nhanh hơn khi ta rời biên.
- **Lớp "đạt" nằm giữa nên khó nhất.** Xác suất p_1 (bản đồ b) chỉ đạt giá trị cao trong một dải hẹp; hai bên là hai biên mà ở đó mô hình phải chọn giữa đạt và một lớp khác. Ta sẽ thấy điều này lặp lại ở ma trận nhầm lẫn (mục 5.3).

Các lỗi nằm ở đâu?

Trong 27 mẫu kiểm tra bị dự đoán sai, mô hình thường đã "báo trước" bằng độ tin cậy thấp:

- 18 mẫu có $\max_k p_k < 0{,}7$, nằm trong dải chưa chắc chắn quanh các đường biên (xác suất lớp được chọn chỉ từ $0{,}498$ đến $0{,}697$).
- 6 mẫu có độ tin cậy từ $0{,}7$ đến $0{,}9$.
- 3 mẫu là những lỗi **sai chắc chắn** với $\max_k p_k \geq 0{,}9$. Ví dụ mẫu $(x_1; x_2) = (1{,}65;\ 3{,}46)$ là sinh viên đạt nhưng chỉ học $1{,}65$ giờ, mô hình gán $\mathbf{p} \approx [0{,}993\ \ 0{,}007\ \ 0{,}000]$ nên mất mát của mẫu này là $-\ln 0{,}007 \approx 4{,}95$.

Mức đóng góp vào mất mát tổng thể cho thấy cross-entropy phạt rất nặng việc sai một cách chắc chắn. Tổng mất mát của 180 mẫu kiểm tra là 63,64 (trung bình 0,3536). Riêng 27 mẫu sai (15% số mẫu) đóng góp 58,2% tổng mất mát. Ba lỗi sai chắc chắn (chỉ 1,7% số mẫu) đóng góp khoảng 16,2%, và riêng mẫu $(1{,}65;\ 3{,}46)$ đã chiếm 7,8%. Ngược lại, trong 72 mẫu có độ tin cậy từ 0,9 trở lên chỉ có 3 mẫu sai (4,2%), còn trong 42 mẫu có độ tin cậy dưới 0,7 có tới 18 mẫu sai (43%). Độ tin cậy vì thế là một tín hiệu hữu ích để quyết định mẫu nào cần được xem lại bằng tay.

Mã tính các số liệu trên (dùng lại `W` và `P_te` từ mô hình ở mục 3.7):

```
W, _ = huan_luyen(Zb_tr, Y_tr, eta=0.5, so_epoch=2000)   # mô hình ở mục 3.7
P_te = softmax(Zb_te @ W); y_hat = P_te.argmax(axis=1)
q = P_te[np.arange(len(y_te)), y_te]                     # xác suất gán cho l
ớp thật
```

```
mat_mat_mau = -np.log(q); sai = y_hat != y_te; tin_cay = P_te.max(axis=1)
print("so mau sai:", sai.sum(), "| sai voi do tin cay < 0.7:", (sai & (tin_ca
y < 0.7)).sum(),
      "| 0.7-0.9:", (sai & (tin_cay >= 0.7) & (tin_cay < 0.9)).sum(), "| >= 0
.9:", (sai & (tin_cay >= 0.9)).sum())
print("ti le mat mat do cac mau sai dong gop:", (mat_mat_mau[sai].sum() / mat
_mat_mau.sum()).round(3))
i = mat_mat_mau.argmax()
print("mau mat mat lon nhat:", X_te[i].round(2), "lop that", y_te[i], "p =",
P_te[i].round(3), "| mat mat =", mat_mat_mau[i].round(2))
```

```
so mau sai: 27 | sai voi do tin cay < 0.7: 18 | 0.7-0.9: 6 | >= 0.9: 3
ti le mat mat do cac mau sai dong gop: 0.582
mau mat mat lon nhat: [1.65 3.46] lop that 1 p = [0.993 0.007 0.   ] | mat ma
t = 4.95
```

5.2. Đánh giá trên tập huấn luyện và tập kiểm tra

Nguyên tắc

Nguyên tắc như ở Logistic Regression: tập kiểm tra bị giấu đi trong suốt quá trình huấn luyện và chỉ dùng để đánh giá ở cuối. Các chỉ số trên tập huấn luyện cho biết mô hình nhớ dữ liệu cũ tốt đến đâu, còn trên tập kiểm tra cho biết nó dự đoán dữ liệu mới tốt đến đâu. Trong bộ dữ liệu này, mỗi lớp có đúng 140 mẫu huấn luyện và 60 mẫu kiểm tra nên hai tập cân bằng. Với dữ liệu thật, ta nên chia **phân tầng** (stratified) để mỗi lớp có tỉ lệ tương tự ở cả hai tập, đặc biệt khi có lớp hiếm.

Kết quả của mô hình trên hai tập:

Chỉ số	Tập huấn luyện ($n = 420$)	Tập kiểm tra ($n = 180$)
Log-loss (cross-entropy) J	0,3730	0,3536
Accuracy	86,2%	85,0%
Điểm Brier (mục 5.4)	0,2173	0,2106
AUC one-vs-rest trung bình (mục 5.4)	0,956	0,961

Các chỉ số của hai tập khá gần nhau, nghĩa là mô hình không học thuộc dữ liệu huấn luyện (với 3 tham số cho mỗi lớp và 420 mẫu, ta cũng không kỳ vọng điều đó). Accuracy kiểm tra thấp hơn huấn luyện 1,2 điểm phần trăm, trong khi log-loss lại ngược lại, chứng tỏ các chênh lệch này nằm trong dao động ngẫu nhiên. Với $n = 180$ mẫu, sai số chuẩn của một tỉ lệ cỡ 85% là

$$\mathrm{SE} \approx \sqrt{\frac{\mathrm{acc}\,(1-\mathrm{acc})}{n}} = \sqrt{\frac{0{,}85 \cdot 0{,}15}{180}} \approx 0{,}027$$

tức khoảng $\pm 2{,}7$ điểm phần trăm. Một con số accuracy trên tập kiểm tra nhỏ có "độ nhiễu" đáng kể. Ma trận nhầm lẫn của hai tập (hàng là nhãn thật, cột là nhãn dự đoán; thứ tự rớt, đạt, giỏi) là

$$\mathbf{C}_{\text{kiểm tra}} = \begin{bmatrix} 49 & 11 & 0 \\ 9 & 48 & 3 \\ 0 & 4 & 56 \end{bmatrix}, \qquad \mathbf{C}_{\text{huấn luyện}} = \begin{bmatrix} 126 & 14 & 0 \\ 10 & 112 & 18 \\ 1 & 15 & 124 \end{bmatrix}$$

Ma trận kiểm tra khớp đúng với ma trận ở mục 1.3. Cả hai ma trận có cùng dạng: lỗi tập trung ở các ô kề đường chéo (rớt với đạt, đạt với giỏi) và gần như không có nhầm lẫn rớt với giỏi. Ta sẽ phân tích kỹ ở mục 5.3.

Hiện tượng học thuộc (overfitting)

Ở mục 4.4 và 4.5, ta thấy mô hình có nhiều tham số và ít dữ liệu dễ rơi vào tình trạng phân tách hoàn hảo với **W** rất lớn. Ta kiểm chứng ảnh hưởng của điều đó lên chất lượng dự đoán bằng một thí nghiệm. Chỉ lấy 45 mẫu huấn luyện (15 mẫu mỗi lớp), dựng các đặc trưng đa thức bậc k của (x_1, x_2) (tương ứng 2,5,9,14,20 đặc trưng cho $k = 1, \ldots, 5$; chuẩn hóa như thường lệ), huấn luyện bằng bộ giải L-BFGS của SciPy với hàm mất mát và gradient đã cài đặt (tối đa 300 vòng lặp), rồi đánh giá trên cả 180 mẫu kiểm tra. Ta so hai trường hợp: không điều chuẩn hóa ($\lambda = 0$) và có điều chuẩn hóa L2 ($\lambda = 0{,}01$, mục 4.4).

```
from scipy.optimize import minimize
from itertools import combinations_with_replacement

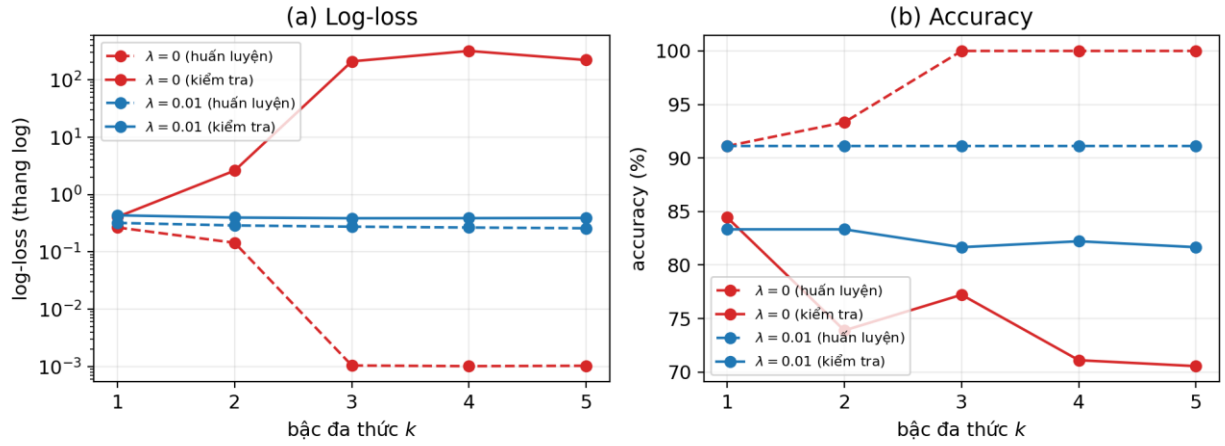
def da_thuc(X, bac=2):
    """Đặc trưng đa thức bậc 1..bac (không gồm cột 1)."""
    cot = []
    for k in range(1, bac + 1):
        for idx in combinations_with_replacement(range(X.shape[1]), k):
            cot.append(np.prod(X[:, idx], axis=1))
    return np.column_stack(cot)

def giai_lbfgs(Xb, Y, lam=0.0, max_iter=300):
    """Tối ưu J + (lam/2)||W~||_F^2 bằng L-BFGS của SciPy, dùng lại hàm mất m
át và gradient đã cài đặt."""
    d1, K = Xb.shape[1], Y.shape[1]
    D0 = np.eye(d1); D0[0, 0] = 0
    f = lambda v: mat_mat(Xb, Y, v.reshape(d1, K)) + 0.5 * lam * np.sum((D0 @
v.reshape(d1, K)) ** 2)
    g = lambda v: (gradient(Xb, Y, v.reshape(d1, K)) + lam * D0 @ v.reshape(d
1, K)).ravel()
    r = minimize(f, np.zeros(d1 * K), jac=g, method="L-BFGS-B", options={"max
iter": max_iter})
    return r.x.reshape(d1, K)

rng = np.random.default_rng(3)
idx = np.concatenate([rng.choice(np.where(y_tr == k)[0], 15, replace=False) f
or k in range(3)])   # 15 mẫu mỗi lớp
X_nho, y_nho = X_tr[idx], y_tr[idx]; Y_nho = one_hot(y_nho, 3)
Y_kt = one_hot(y_te, 3)
for bac in (1, 2, 3, 5):
    F_nho, F_kt = da_thuc(X_nho, bac), da_thuc(X_te, bac)
    Z_nho, Z_kt, _, _ = chuan_hoa(F_nho, F_kt)
```

```
    Z_nho, Z_kt = them_bias(Z_nho), them_bias(Z_kt)
    for lam in (0.0, 0.01):
        Wn = giai_lbfgs(Z_nho, Y_nho, lam=lam, max_iter=300)
        print(f"bac {bac} | lam = {lam:<4} | tham so = {Wn.size:2d} | J train
= {mat_mat(Z_nho, Y_nho, Wn):6.3f}"
              f" | J test = {mat_mat(Z_kt, Y_kt, Wn):8.3f} | acc train = {np.
mean(du_doan(Z_nho, Wn) == y_nho):.3f}"
              f" | acc test = {np.mean(du_doan(Z_kt, Wn) == y_te):.3f}")
```

```
bac 1 | lam = 0.0  | tham so =  9 | J train =  0.265 | J test =    0.407 | ac
c train = 0.911 | acc test = 0.844
bac 1 | lam = 0.01 | tham so =  9 | J train =  0.318 | J test =    0.434 | ac
c train = 0.911 | acc test = 0.833
bac 2 | lam = 0.0  | tham so = 18 | J train =  0.143 | J test =    2.609 | ac
c train = 0.933 | acc test = 0.739
bac 2 | lam = 0.01 | tham so = 18 | J train =  0.287 | J test =    0.396 | ac
c train = 0.911 | acc test = 0.833
bac 3 | lam = 0.0  | tham so = 30 | J train =  0.000 | J test =  209.778 | ac
c train = 1.000 | acc test = 0.772
bac 3 | lam = 0.01 | tham so = 30 | J train =  0.274 | J test =    0.385 | ac
c train = 0.911 | acc test = 0.817
bac 5 | lam = 0.0  | tham so = 63 | J train =  0.000 | J test =  221.082 | ac
c train = 1.000 | acc test = 0.706
bac 5 | lam = 0.01 | tham so = 63 | J train =  0.257 | J test =    0.390 | ac
c train = 0.911 | acc test = 0.817
```



Hình 16. Log-loss (a, thang log) và accuracy (b) trên tập huấn luyện (nét đứt) và tập kiểm tra (nét liền) khi tăng bậc đa thức, với 45 *mẫu huấn luyện. Không điều chuẩn hóa (*$\lambda = 0$*, đỏ): log-loss kiểm tra tăng vọt và accuracy kiểm tra giảm. Có điều chuẩn hóa (*$\lambda = 0{,}01$*, xanh): ổn định ở mọi bậc. (Để vẽ được thang log, log-loss huấn luyện đã được cộng thêm* 0,001*.)*

Bảng và Hình 16 cho thấy:

- Khi không điều chuẩn hóa và mô hình đủ phức tạp ($k \geq 3$), accuracy huấn luyện đạt 100% và J huấn luyện về gần 0 (mô hình học thuộc), nhưng J kiểm tra vọt lên

hàng trăm và accuracy kiểm tra giảm từ 84,4% xuống 70,6% khi $k = 5$. Đây là hậu quả đã dự báo ở mục 4.4: dữ liệu phân tách hoàn hảo làm $\mathbf{W}$ phình lên ($\|\widetilde{\mathbf{W}}\|_F$ từ vài trăm đến hàng chục nghìn, so với khoảng 3 khi có điều chuẩn hóa), xác suất cực đoan, và những mẫu kiểm tra bị dự đoán sai bị phạt cực nặng. Giá trị cụ thể như 209,8 phụ thuộc vào giới hạn 300 vòng lặp (chạy thêm thì còn tăng), nên điều cần nhớ là bậc độ lớn, không phải con số chính xác.

- Log-loss phát hiện overfitting nhạy hơn accuracy. Ở $k = 2$, accuracy kiểm tra giảm 9 điểm (từ 83,3% xuống 73,9%) trong khi log-loss kiểm tra tăng gấp 6,6 lần (0,396 → 2,609); đến $k = 3$ khoảng cách này rất lớn (0,385 so với 209,8). Khoảng cách $J_{\text{kiểm tra}} - J_{\text{huấn luyện}}$ rất lớn là dấu hiệu rõ rệt của overfitting.
- Điều chuẩn hóa L2 giữ log-loss kiểm tra ổn định (0,385 đến 0,434) ở mọi bậc đa thức, đánh đổi bằng log-loss huấn luyện cao hơn. Accuracy kiểm tra ở mức 81,7% đến 83,3%, thấp hơn mô hình dùng đủ 420 mẫu (85,0%), điều hợp lý vì mô hình này chỉ thấy 45 mẫu.

Lưu ý. Trong thí nghiệm này, ta dùng tập kiểm tra để so sánh các cấu hình chỉ để minh họa. Trong thực tế, các quyết định như chọn bậc đa thức hay chọn λ phải dựa trên một tập kiểm định (validation set) riêng hoặc kiểm định chéo, để tập kiểm tra vẫn "chưa bị nhìn thấy" cho đến lần đánh giá cuối cùng. Ta sẽ làm điều đó ở mục 5.5.

5.3. Accuracy, ma trận nhầm lẫn $K \times K$, precision, recall, F1 theo từng lớp

Ma trận nhầm lẫn $K \times K$

Với K lớp, ma trận nhầm lẫn $\mathbf{C}$ có kích thước $K \times K$, trong đó C_{ij} là số mẫu có lớp thật là i nhưng bị dự đoán là lớp j (hàng là nhãn thật, cột là nhãn dự đoán):

$$\mathbf{C} = \begin{bmatrix} C_{00} & C_{01} & C_{02} \\ C_{10} & C_{11} & C_{12} \\ C_{20} & C_{21} & C_{22} \end{bmatrix}$$

Đường chéo chứa các mẫu đúng, các ô ngoài đường chéo là các loại nhầm lẫn cụ thể (có $K(K-1) = 6$ loại, thay vì chỉ hai loại FP và FN của bài toán nhị phân). Tổng hàng i là số mẫu thật của lớp i (gọi là *support*, n_i), tổng cột j là số mẫu được dự đoán là lớp j, và accuracy là tỉ lệ đường chéo:

$$\text{Accuracy} = \frac{\text{tr}(\mathbf{C})}{n} = \frac{\sum_k C_{kk}}{n}$$

Trên tập kiểm tra (mục 5.2):

$$\mathbf{C} = \begin{bmatrix} 49 & 11 & 0 \\ 9 & 48 & 3 \\ 0 & 4 & 56 \end{bmatrix}, \qquad \text{Accuracy} = \frac{49 + 48 + 56}{180} = \frac{153}{180} = 0{,}850$$

Cách đọc: sinh viên rớt bị nhầm thành đạt 11 lần, sinh viên đạt bị nhầm thành rớt 9 lần và thành giỏi 3 lần, sinh viên giỏi bị nhầm thành đạt 4 lần. **Không có** sinh viên rớt nào bị đoán là giỏi và ngược lại. Mô hình nhầm lẫn "theo thứ bậc": các lớp kề nhau mới bị lẫn, đúng như hình học của bài toán (lớp đạt nằm giữa rớt và giỏi, Hình 15). Hình 17a vẽ ma trận này dưới dạng bản đồ nhiệt.

Cài đặt:

```
def ma_tran_nham_lan(y, y_hat, K):
    """Ma trận nhầm lẫn K x K: C[i, j] = số mẫu thuộc lớp thật i bị dự đoán l
à lớp j."""
    C = np.zeros((K, K), dtype=int)
    for t, p in zip(y, y_hat):
        C[t, p] += 1
    return C
```

Precision, recall và F1 theo từng lớp

Các chỉ số precision, recall, F1 vốn định nghĩa cho hai lớp. Với K lớp, ta xét từng lớp k như một bài toán nhị phân "lớp k hay không phải lớp k" (cùng ý tưởng one-vs-rest ở mục 1.3), rồi đọc các đại lượng từ ma trận $\mathbf{C}$:

$$\mathrm{TP}_k = C_{kk}, \qquad \mathrm{FN}_k = \sum_{j \neq k} C_{kj}, \qquad \mathrm{FP}_k = \sum_{i \neq k} C_{ik}, \qquad \mathrm{TN}_k = n - \mathrm{TP}_k - \mathrm{FN}_k - \mathrm{FP}_k$$

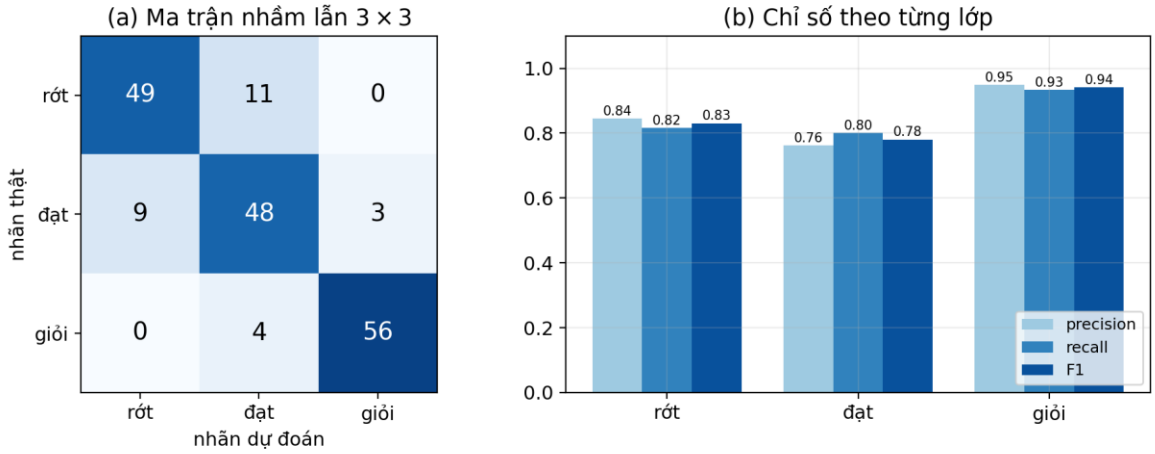
Nói cách khác, TP_k là phần tử trên đường chéo, FN_k là phần còn lại của **hàng** k (mẫu lớp k bị đoán sang lớp khác) và FP_k là phần còn lại của **cột** k (mẫu của lớp khác bị đoán thành lớp k). Khi đó

$$\mathrm{Precision}_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FP}_k} = \frac{C_{kk}}{\sum_i C_{ik}}, \qquad \mathrm{Recall}_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FN}_k} = \frac{C_{kk}}{\sum_j C_{kj}}, \qquad F1_k = \frac{2\,\mathrm{TP}_k}{2\,\mathrm{TP}_k + \mathrm{FP}_k + \mathrm{FN}_k}$$

Precision của lớp k trả lời "trong các mẫu mô hình dự đoán là lớp k, bao nhiêu thật sự thuộc lớp k" (chia cho tổng **cột**), còn recall trả lời "trong các mẫu thật sự thuộc lớp k, mô hình tìm ra được bao nhiêu" (chia cho tổng **hàng**). Với ma trận kiểm tra:

Lớp	TP_k	FP_k	FN_k	Precision	Recall	F1	Support
Rớt	49	9	11	$49/58 = 0{,}845$	$49/60 = 0{,}817$	$98/118 = 0{,}831$	60
Đạt	48	15	12	$48/63 = 0{,}762$	$48/60 = 0{,}800$	$96/123 = 0{,}780$	60
Giỏi	56	3	4	$56/59 = 0{,}949$	$56/60 = 0{,}933$	$112/119 = 0{,}941$	60

Lớp giỏi có cả precision và recall cao nhất (mô hình dễ nhận ra sinh viên giỏi vì họ nằm ở rìa dữ liệu), còn lớp đạt thấp nhất ($F1 = 0{,}780$), vì nằm giữa nên bị lẫn với cả hai lớp kề. Hình 17b vẽ ba chỉ số theo từng lớp.



Hình 17. (a) Ma trận nhầm lẫn 3×3 trên tập kiểm tra: lỗi chỉ xảy ra giữa các lớp kề nhau. (b) Precision, recall và F1 theo từng lớp: lớp giỏi cao nhất, lớp đạt (nằm giữa) thấp nhất.

Các cách lấy trung bình

Để tóm tắt K giá trị thành một con số, có ba cách lấy trung bình, đều có trong scikit-learn:

- **Macro**: trung bình đơn giản của các chỉ số theo từng lớp, mỗi lớp có trọng số như nhau:

$$\text{Precision}_{\text{macro}} = \frac{1}{K}\sum_{k=0}^{K-1} \text{Precision}_k\,, \quad \text{Recall}_{\text{macro}} = \frac{1}{K}\sum_{k=0}^{K-1} \text{Recall}_k\,, \quad F1_{\text{macro}} = \frac{1}{K}\sum_{k=0}^{K-1} F\,1_k$$

- **Weighted**: trung bình có trọng số theo support n_k, $F1_{\text{weighted}} = \frac{1}{n}\sum_k n_k\ F1_k$ (tương tự cho precision và recall).
- **Micro**: gộp TP, FP, FN của mọi lớp trước rồi mới tính:

$$\text{Precision}_{\text{micro}} = \frac{\sum_k \text{TP}_k}{\sum_k (\text{TP}_k + \text{FP}_k)}, \qquad \text{Recall}_{\text{micro}} = \frac{\sum_k \text{TP}_k}{\sum_k (\text{TP}_k + \text{FN}_k)}$$

Trong bài toán một nhãn cho mỗi mẫu như ở đây, mỗi mẫu bị sai là một FP của đúng một lớp và một FN của đúng một lớp, nên $\sum_k \text{FP}_k = \sum_k \text{FN}_k = n - \text{tr}(\mathbf{C})$. Suy ra **micro-precision** = **micro-recall** = **micro-F1** = **accuracy**: ba số này không cung cấp thông tin mới so với accuracy.

Trên tập kiểm tra:

Cách lấy trung bình	Precision	Recall	F1
Macro	0,852	0,850	0,851
Weighted	0,852	0,850	0,851

Cách lấy trung bình	Precision	Recall	F1
Micro	0,850	0,850	0,850

Macro và weighted trùng nhau vì ba lớp cân bằng ($n_k = 60$). Chúng khác nhau rõ rệt khi lớp lệch. Xét một ví dụ giả định với $n = 160$ mẫu, lớp giỏi hiếm (chỉ 10 mẫu):

$$\mathbf{C} = \begin{bmatrix} 90 & 5 & 5 \\ 10 & 30 & 10 \\ 2 & 3 & 5 \end{bmatrix}, \qquad \text{Accuracy} = \frac{125}{160} = 0{,}781$$

Lớp	Precision	Recall	F1	Support
Rớt	0,882	0,900	0,891	100
Đạt	0,789	0,600	0,682	50
Giỏi	0,250	0,500	0,333	10
Macro F1			0,635	
Weighted F1			0,791	
Micro F1 (= accuracy)			0,781	

Macro-F1 (0,635) thấp hơn nhiều so với accuracy (0,781) vì lớp hiếm "giỏi" có $F1$ chỉ 0,333 và đóng góp $\frac{1}{3}$ vào macro dù chỉ chiếm 6% số mẫu. Weighted-F1 (0,791) và micro bám theo các lớp lớn nên che mất điểm yếu đó. Quy tắc chọn: nếu mọi lớp quan trọng như nhau (kể cả lớp hiếm), nhìn **macro**; nếu chỉ quan tâm tỉ lệ đúng trên toàn dữ liệu, nhìn accuracy hoặc weighted. Tốt nhất là luôn xem cả bảng theo từng lớp.

Cài đặt:

```
def chi_so_theo_lop(C):
    """Precision, recall, F1 của từng lớp (coi lớp đó là 'dương', các lớp còn
lại là 'âm')."""
    tp = np.diag(C).astype(float)
    fp = C.sum(axis=0) - tp          # tổng cột trừ đường chéo
    fn = C.sum(axis=1) - tp          # tổng hàng trừ đường chéo
    return tp / (tp + fp), tp / (tp + fn), 2 * tp / (2 * tp + fp + fn), C.sum
(axis=1)

def trung_binh(C):
    prec, rec, f1, sup = chi_so_theo_lop(C)
    return {"macro": (prec.mean(), rec.mean(), f1.mean()),
            "weighted": tuple(np.average(v, weights=sup) for v in (prec, rec,
f1)),
            "micro": (np.trace(C) / C.sum(),) * 3}          # micro: precisio
n = recall = F1 = accuracy

P_tr, P_te = softmax(Zb_tr @ W), softmax(Zb_te @ W)
yh_tr, yh_te = P_tr.argmax(axis=1), P_te.argmax(axis=1)
C_te = ma_tran_nham_lan(y_te, yh_te, 3)
prec, rec, f1, sup = chi_so_theo_lop(C_te)
```

```
print("precision =", prec.round(3), "| recall =", rec.round(3), "| F1 =", f1.
round(3), "| support =", sup)
for ten, v in trung_binh(C_te).items(): print(f"{ten:9s}", np.round(v, 3))
C_lech = np.array([[90, 5, 5], [10, 30, 10], [2, 3, 5]])          # ví dụ tập
lệch
prec, rec, f1, sup = chi_so_theo_lop(C_lech)
print("precision =", prec.round(3), "| recall =", rec.round(3), "| F1 =", f1.
round(3), "| support =", sup)
for ten, v in trung_binh(C_lech).items(): print(f"{ten:9s}", np.round(v, 3))
```

```
precision = [0.845 0.762 0.949] | recall = [0.817 0.8   0.933] | F1 = [0.831
0.78  0.941] | support = [60 60 60]
macro     [0.852 0.85  0.851]
weighted  [0.852 0.85  0.851]
micro     [0.85 0.85 0.85]
precision = [0.882 0.789 0.25 ] | recall = [0.9 0.6 0.5] | F1 = [0.891 0.682
0.333] | support = [100  50  10]
macro     [0.641 0.667 0.635]
weighted  [0.814 0.781 0.791]
micro     [0.781 0.781 0.781]
```

Từ ngưỡng quyết định đến quyết định theo chi phí

Với hai lớp, Logistic Regression cho phép đổi ngưỡng τ để cân bằng hai loại sai, và $\tau^* = \frac{c_{\text{FP}}}{c_{\text{FP}}+c_{\text{FN}}}$ là ngưỡng tối ưu khi biết chi phí. Với K lớp, ta không còn một ngưỡng mà có **ma trận chi phí c** với c_{kj} là chi phí khi mẫu thuộc lớp thật k bị dự đoán là lớp j ($c_{kk} = 0$). Nếu mô hình ước lượng xác suất tốt, chi phí kỳ vọng khi dự đoán lớp j là $\sum_k p_k\, c_{kj}$, và quyết định tối ưu là

$$\hat{y} = \operatorname*{argmin}_j \sum_{k=0}^{K-1} p_k\ c_{kj} = \operatorname*{argmin}_j\ (\mathbf{p}\,\mathbf{c})_j$$

Khi mọi sai lầm tốn như nhau ($c_{kj} = 1$ với $k \neq j$), chi phí kỳ vọng của việc dự đoán j là $1 - p_j$ và quy tắc trở về $\hat{y} = \operatorname{argmax}_j p_j$ (quy tắc mặc định ở mục 2.4). Với $K = 2$, quy tắc này cho đúng ngưỡng $p \geq \tau^* = \frac{c_{\text{FP}}}{c_{\text{FP}}+c_{\text{FN}}}$.

Ví dụ. Nhà trường muốn phát hiện sinh viên có nguy cơ rớt, và bỏ sót một sinh viên rớt (dự đoán đạt hoặc giỏi) tốn gấp 3 lần mọi sai lầm khác:

$$\mathbf{c} = \begin{bmatrix} 0 & 3 & 3 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

Với một sinh viên có $\mathbf{p} = [0{,}2\ \ 0{,}5\ \ 0{,}3]$, quy tắc argmax chọn "đạt", nhưng chi phí kỳ vọng là $\mathbf{pc} = [0{,}8\ \ 0{,}9\ \ 1{,}1]$ nên quy tắc theo chi phí chọn "rớt" (dù xác suất rớt chỉ 0,2). Áp dụng cho cả tập kiểm tra:

```
chi_phi = np.array([[0, 3, 3], [1, 0, 1], [1, 1, 0]])          # chi_phi[thật
, dự đoán]
```

```
y_cp = (P_te @ chi_phi).argmin(axis=1)                    # chọn lớp có
chi phí kỳ vọng nhỏ nhất
C_cp = ma_tran_nham_lan(y_te, y_cp, 3)
print(C_cp); print("tong chi phi: argmax =", (C_te * chi_phi).sum(), "| theo
chi phi =", (C_cp * chi_phi).sum(), "| acc =", np.mean(y_cp == y_te).round(4)
)
```

```
[[56  4  0]
 [18 39  3]
 [ 0  4 56]]
tong chi phi: argmax = 49 | theo chi phi = 37 | acc = 0.8389
```

Recall của lớp rớt tăng từ $49/60 = 81{,}7\%$ lên $56/60 = 93{,}3\%$, tổng chi phí giảm từ 49 xuống 37, và đổi lại accuracy giảm từ 85,0% xuống 83,9% (nhiều sinh viên đạt bị báo nhầm là rớt: 18 thay vì 9). Đây đúng là đánh đổi giữa hai loại sai mà ta muốn kiểm soát. Giống Logistic Regression, quy tắc này chỉ tối ưu khi các xác suất được hiệu chuẩn tốt (mục 5.4), và ma trận chi phí phải do bài toán quy định trước, không được chọn bằng cách thử trên tập kiểm tra.

5.4. Đánh giá chất lượng xác suất: log-loss đa lớp, top-k accuracy, hiệu chuẩn, AUC theo kiểu one-vs-rest

Các chỉ số ở mục 5.3 chỉ dùng nhãn dự đoán $\text{argmax}_k p_{ik}$. Nhưng Softmax Regression cho ra cả một phân phối trên K lớp, và ta muốn đánh giá chính phân phối đó.

Log-loss đa lớp và điểm Brier

Chỉ số đầu tiên là chính hàm mất mát cross-entropy tính trên tập kiểm tra, còn gọi là log-loss đa lớp:

$$\text{Log-loss} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=0}^{K-1} y_{ik}\ln p_{ik} = -\frac{1}{n}\sum_{i=1}^{n}\ln q_i$$

Trên tập kiểm tra, log-loss là 0,3536. Mô hình "không biết gì" luôn trả $p_{ik} = \frac{1}{3}$ có log-loss $\ln 3 \approx 1{,}0986$, và mô hình hằng luôn trả tỉ lệ các lớp của tập huấn luyện (mỗi lớp $\frac{1}{3}$ vì cân bằng) cũng cho 1,0986. Mô hình Softmax giảm log-loss từ 1,0986 xuống 0,3536. Diễn giải trực quan: $e^{-0{,}3536} \approx 0{,}702$, tức trung bình nhân của xác suất mô hình gán cho lớp thật là khoảng 0,70, so với 0,33 nếu chỉ đoán mò.

Một thước đo xác suất đơn giản hơn là **điểm Brier đa lớp**: trung bình bình phương khoảng cách giữa vector xác suất và vector one-hot (chính là đại lượng $\| \mathbf{p}_i - \mathbf{y}_i \|^2$ ở mục 4.1):

$$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n} \| \mathbf{p}_i - \mathbf{y}_i \|^2$$

Trên tập kiểm tra, điểm Brier của mô hình là 0,2106 so với $\frac{2}{3} \approx 0{,}6667$ của mô hình hằng (càng nhỏ càng tốt; điểm này nằm trong $[0,2]$).

Top-k accuracy

Accuracy thường (top-1) chỉ tính một mẫu là đúng nếu lớp thật có xác suất lớn nhất. **Top-k accuracy** nới lỏng: mẫu được tính là đúng nếu lớp thật nằm trong k lớp có xác suất cao nhất:

$$\text{Top-}k = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}\left[\text{số lớp có } p_{ij} > q_i \text{ nhỏ hơn } k\right]$$

Chỉ số này hữu ích khi số lớp K lớn và việc "gợi ý vài lớp" là chấp nhận được (ví dụ gợi ý nhãn cho ảnh hoặc từ tiếp theo). Với $K = 3$ như ở đây, top-2 có phần tầm thường: trên tập kiểm tra, top-1 là 85,0%, còn top-2 và top-3 đều là 100%. Kết quả top-2 $= 100\%$ cho biết mô hình **không bao giờ** để lớp thật xuống hạng cuối: dù sai, lớp thật vẫn là lớp có xác suất cao thứ hai, chính là một khía cạnh của nhận xét "chỉ nhầm lẫn giữa các lớp kề nhau" ở mục 5.3.

Hiệu chuẩn

Xác suất có đúng nghĩa là xác suất không? Với Softmax Regression, cách kiểm tra đơn giản và phổ biến nhất là hiệu chuẩn theo **độ tin cậy** (confidence) $\max_k p_{ik}$: chia các mẫu theo độ tin cậy thành vài nhóm, rồi so độ tin cậy trung bình với tỉ lệ dự đoán đúng thực tế của nhóm. Nếu mô hình nói "tôi chắc 80%" cho 100 sinh viên thì ta mong khoảng 80 dự đoán đó đúng.

Nhóm theo $\max_k p_k$	Số mẫu	Độ tin cậy trung bình	Tỉ lệ đúng thực tế
[0,333; 0,5)	1	0,498	0,000
[0,5; 0,6)	21	0,553	0,571
[0,6; 0,7)	20	0,655	0,600
[0,7; 0,8)	30	0,765	0,933
[0,8; 0,9)	36	0,854	0,889
[0,9; 1]	72	0,968	0,958

Có thể gộp thành một con số bằng **sai số hiệu chuẩn kỳ vọng** (Expected Calibration Error):

$$\text{ECE} = \sum_{b} \frac{n_b}{n} \left|\text{acc}_b - \text{conf}_b\right|$$

với n_b là số mẫu trong nhóm b. Trên tập kiểm tra, ECE $= 0{,}050$, nghĩa là độ tin cậy lệch khỏi tỉ lệ đúng thực tế trung bình 5 điểm phần trăm. Các điểm ở Hình 18c nằm gần đường chéo (hiệu chuẩn hoàn hảo), và độ tin cậy trung bình toàn tập (0,826) hơi thấp hơn accuracy (0,850): mô hình hơi **dè dặt** chứ không quá tự tin, trái với tình huống phân tách hoàn hảo ở mục 4.4 (nơi **W** lớn làm xác suất cực đoan). Cần thận trọng khi đọc bảng này: nhiều nhóm chỉ có 20 đến 36 mẫu nên các con số rất nhiễu (ví dụ nhóm [0,7; 0,8) đúng 93,3% trong khi độ tin cậy trung bình chỉ 0,765 chủ yếu là dao động ngẫu nhiên).

Ngoài ra, mỗi lớp có thể kiểm tra riêng: trung bình của p_k trên tập kiểm tra so với tỉ lệ thực của lớp k là 0,331; 0,341; 0,329 so với 0,333 cho cả ba lớp. Trên **tập huấn luyện**, hai vế bằng nhau

chính xác theo điều kiện tối ưu ở mục 4.3 ($\frac{1}{n}\sum_i p_{ik} = \frac{1}{n}\sum_i y_{ik}$); trên tập kiểm tra chúng chỉ xấp xỉ.

AUC theo kiểu one-vs-rest

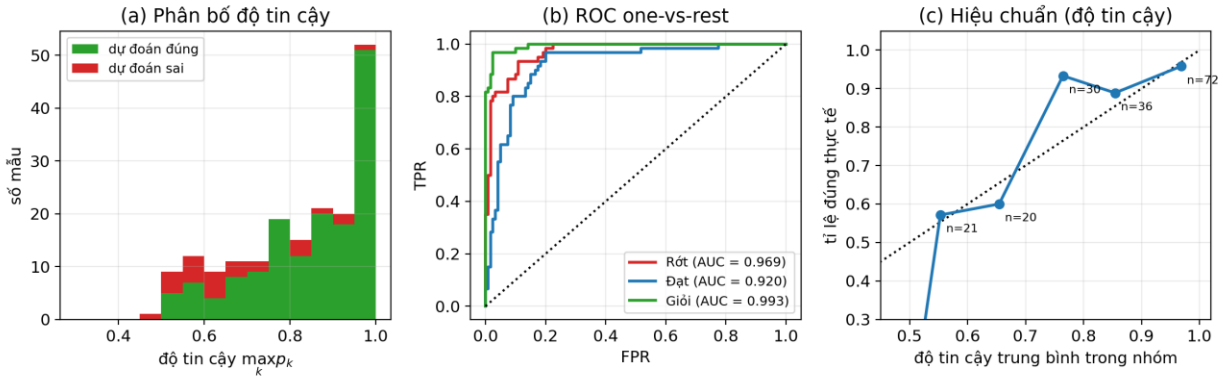
Ở mục 5.3, mỗi lớp được xem như một bài toán nhị phân "lớp k hay không". Ta có thể làm tương tự với AUC: coi p_{ik} là điểm số của bộ phân loại nhị phân "lớp k hay không", và tính đường ROC và AUC của nó. Như ở Logistic Regression, AUC của lớp k có diễn giải xác suất: đó là xác suất để một mẫu thuộc lớp k chọn ngẫu nhiên có p_k **cao hơn** một mẫu không thuộc lớp k chọn ngẫu nhiên:

$$\text{AUC}_k = \frac{1}{n_k\,(n - n_k)} \sum_{i:\,y_i = k} \; \sum_{j:\,y_j \neq k} \left(\mathbb{1}[p_{ik} > p_{jk}] + 1/2\,\mathbb{1}[p_{ik} = p_{jk}] \right)$$

Với mỗi lớp trong bộ dữ liệu này có $60 \times 120 = 7200$ cặp (dương, âm). Kết quả (Hình 18b): $\text{AUC}_{\text{rót}} = 0{,}969$, $\text{AUC}_{\text{đạt}} = 0{,}920$, $\text{AUC}_{\text{giỏi}} = 0{,}993$. Lớp đạt (nằm giữa) có AUC thấp nhất, giống precision và recall ở mục 5.3: điểm số p_1 phải phân biệt đạt với cả rớt lẫn giỏi, hai phía khác nhau, nên khó xếp hạng đúng hơn. AUC trung bình theo kiểu one-vs-rest là

$$\text{AUC}_{\text{macro}} = \frac{1}{K}\sum_{k=0}^{K-1} \text{AUC}_k = \frac{0{,}969 + 0{,}920 + 0{,}993}{3} \approx 0{,}961$$

(scikit-learn còn có `average="weighted"` theo support, và cách one-vs-one `multi_class="ovo"` so sánh từng cặp lớp; ở đây cả ba cho 0,961 vì các lớp cân bằng.) Như ở bài toán hai lớp, AUC chỉ phụ thuộc vào thứ tự các xác suất, nên không đánh giá được độ hiệu chuẩn: thay p bằng p^2 (giữ thứ tự) thì AUC không đổi dù xác suất bị méo.



Hình 18. (a) Phân bố độ tin cậy $\max_k p_k$ trên tập kiểm tra, tách theo dự đoán đúng (xanh) và sai (đỏ): lỗi dồn về vùng độ tin cậy thấp. (b) Đường ROC one-vs-rest cho từng lớp (lớp đạt thấp nhất). (c) Biểu đồ hiệu chuẩn theo độ tin cậy: các điểm gần đường chéo; n là số mẫu trong nhóm.

Cài đặt các chỉ số trên:

```python
def log_loss_da_lop(P, y):
    return -np.mean(np.log(P[np.arange(len(y)), y]))

def brier_da_lop(P, y):
    return np.mean(np.sum((P - one_hot(y, P.shape[1])) ** 2, axis=1))

def top_k_accuracy(P, y, k):
    so_lop_cao_hon = (P > P[np.arange(len(y)), y][:, None]).sum(axis=1)     # số lớp có xác suất cao hơn lớp thật
    return np.mean(so_lop_cao_hon < k)

def ece(P, y):
    """Expected calibration error theo độ tin cậy max_k p_k."""
    do_tin_cay, dung = P.max(axis=1), P.argmax(axis=1) == y
    bien = [1 / P.shape[1], 0.5, 0.6, 0.7, 0.8, 0.9, 1.0001]
    tong = 0.0
    for a, b in zip(bien[:-1], bien[1:]):
        m = (do_tin_cay >= a) & (do_tin_cay < b)
        if m.any():
            tong += m.mean() * abs(dung[m].mean() - do_tin_cay[m].mean())
    return tong

def auc_cap(diem, nhan_duong):
    """AUC bằng cách đếm cặp (dương, âm)."""
    pos, neg = diem[nhan_duong], diem[~nhan_duong]
    return np.mean([(a > b) + 0.5 * (a == b) for a in pos for b in neg])

print("log-loss =", round(log_loss_da_lop(P_te, y_te), 4), "| Brier =", round(brier_da_lop(P_te, y_te), 4))
p0 = Y_tr.mean(axis=0); print("hang (ti le lop): log-loss =", round(-np.mean(np.log(p0[y_te])), 4),
                             "| Brier =", round(np.mean(np.sum((p0 - one_hot(y_te, 3)) ** 2, axis=1)), 4))
print("top-1 =", top_k_accuracy(P_te, y_te, 1), "| top-2 =", top_k_accuracy(P_te, y_te, 2), "| top-3 =", top_k_accuracy(P_te, y_te, 3))
print("ECE =", round(ece(P_te, y_te), 4))
auc = [auc_cap(P_te[:, k], y_te == k) for k in range(3)]
print("AUC OvR theo lop =", np.round(auc, 4), "| macro =", round(np.mean(auc), 4))
```

```
log-loss = 0.3536 | Brier = 0.2106
hang (ti le lop): log-loss = 1.0986 | Brier = 0.6667
top-1 = 0.85 | top-2 = 1.0 | top-3 = 1.0
ECE = 0.05
AUC OvR theo lop = [0.969  0.9197 0.9929] | macro = 0.9606
```

Khi nào dùng chỉ số nào?

Câu hỏi	Chỉ số phù hợp
Tỉ lệ dự đoán nhãn đúng (các lớp cân bằng, mọi loại sai như nhau)	Accuracy
Lớp lệch, hoặc các lớp có tầm quan trọng khác nhau	Ma trận nhầm lẫn, precision, recall, F1 theo từng lớp; macro-F1
Mô hình nhầm lẫn giữa những cặp lớp nào	Ma trận nhầm lẫn $K \times K$
Lớp thật có nằm trong vài ứng viên đầu không	Top-k accuracy
Mô hình xếp hạng từng lớp tốt đến đâu, không phụ thuộc ngưỡng	ROC và AUC one-vs-rest
Chất lượng của chính xác suất	Log-loss, điểm Brier, hiệu chuẩn (ECE)
Các loại sai có chi phí khác nhau	Ma trận chi phí và quy tắc $\operatorname{argmin}_j (\mathbf{pc})_j$
So sánh các cấu hình trong lúc huấn luyện	Log-loss (nhạy hơn accuracy, mục 5.2)

5.5. Huấn luyện và so sánh với Softmax Regression trong scikit-learn

Mô hình của scikit-learn

Trong thực tế ta thường dùng lớp *`LogisticRegression`* của scikit-learn: khi có nhiều hơn hai lớp, thư viện dùng chính hàm mất mát cross-entropy nhiều lớp (multinomial) với softmax, tức Softmax Regression. Ta huấn luyện trên đặc trưng gốc với C rất lớn (gần như không điều chuẩn hóa) để so sánh công bằng với mô hình tự cài đặt:

```
from sklearn.linear_model import LogisticRegression

W_goc = np.vstack([W[0] - (mu / s) @ W[1:], W[1:] / s[:, None]])  # đổi về th
ang đo gốc (mục 4.2)
clf = LogisticRegression(C=1e10, max_iter=10000).fit(X_tr, y_tr)
print("intercept_ =", clf.intercept_.round(4)); print("coef_ ="); print(clf.c
oef_.round(4))
print("tu cai dat (thang goc), W^T ="); print(W_goc.T.round(4))
p_sk = clf.predict_proba(X_te); p_ours = softmax(them_bias(X_te) @ W_goc)
print("max |p_sk - p_ours| =", np.abs(p_sk - p_ours).max().round(5))
```

```
intercept_ = [ 5.6324  1.8761 -7.5085]
coef_ =
[[-1.4092  0.8129]
 [ 0.0813 -0.2438]
 [ 1.3279 -0.5692]]
tu cai dat (thang goc), W^T =
[[ 5.6282 -1.4079  0.8123]
 [ 1.8766  0.0809 -0.2435]
 [-7.5049  1.3271 -0.5688]]
max |p_sk - p_ours| = 0.00035
```

Các tham số và thuộc tính cần chú ý:

- `C`: nghịch đảo độ mạnh của điều chuẩn hóa L2. Như đã nói ở mục 4.4, hàm mục tiêu của thư viện là $C\sum_i \ell_i + \frac{1}{2}\|\widetilde{\mathbf{W}}\|_F^2$, tương đương $\lambda = \frac{1}{Cn}$. C càng nhỏ thì điều chuẩn hóa càng mạnh; mặc định là $C = 1$ (tức $\lambda = 1/420 \approx 0{,}0024$ với $n = 420$).
- `intercept_` và `coef_`: thư viện lưu hệ số chệch của K lớp trong `intercept_` (vector độ dài K) và các hệ số còn lại trong `coef_` (ma trận $K \times d$, mỗi **hàng** là một lớp), không cần thêm cột 1 vào $\mathbf{X}$. So với ký hiệu của ta, $\mathbf{W} = [\text{intercept_};\ \text{coef_}^T]$, tức `coef_` là phần $\widetilde{\mathbf{W}}^T$ và `intercept_` là hàng đầu của $\mathbf{W}$.
- `predict_proba` trả ma trận $n \times K$ gồm $\mathbf{P}$ (cột theo thứ tự `classes_`), `decision_function` trả ma trận điểm số $\mathbf{S} = \mathbf{XW}$, và `predict` trả lớp có điểm số lớn nhất.
- `max_iter` và `solver` (mặc định `lbfgs`, một phương pháp dùng thông tin độ cong như đã nhắc ở mục 4.3; ở mục 5.2 ta đã dùng chính bộ giải này của SciPy).

Hệ số của thư viện gần như trùng với mô hình tự cài đặt (sau khi đổi về thang đo gốc), và xác suất dự đoán lệch nhau tối đa chỉ 0,00035. Khác biệt nhỏ này là do mô hình của ta dừng sau 2000 epoch (rất gần nhưng chưa chính xác tại điểm tối ưu), còn bộ giải của thư viện chạy đến khi đạt dung sai chặt hơn. Còn một chi tiết đáng chú ý nữa, liên quan đến mục 4.3: các hàng của ma trận hệ số của thư viện cũng cộng lại bằng 0 (ví dụ `intercept_`: $5{,}6324 + 1{,}8761 - 7{,}5085 = 0$). Ta không ép điều này, nhưng thư viện khởi tạo từ $\mathbf{0}$ nên tổng các cột của $\mathbf{W}$ không đổi (mục 3.3), và với C hữu hạn thì điều chuẩn hóa L2 chọn nghiệm có các cột trung bình bằng 0 (mục 4.4).

So sánh các mô hình

Ta so sánh trên cùng dữ liệu bốn mô hình: Softmax Regression tự cài đặt, `LogisticRegression` với $C = 10^{10}$, với $C = 1$ (mặc định), và one-vs-rest (K bộ phân loại Logistic độc lập, bọc bằng `OneVsRestClassifier`, mục 1.3).

```
from sklearn.multiclass import OneVsRestClassifier
from sklearn.metrics import log_loss, roc_auc_score

clf_md = LogisticRegression().fit(X_tr, y_tr)
# C = 1 (mặc định)
ovr = OneVsRestClassifier(LogisticRegression(C=1e10, max_iter=10000)).fit(X_t
r, y_tr)
for ten, m in [("tu cai dat", None), ("sklearn C=1e10", clf), ("sklearn mac d
inh C=1", clf_md), ("sklearn one-vs-rest", ovr)]:
    p = p_ours if m is None else m.predict_proba(X_te)
    a_tr = np.mean(du_doan(Zb_tr, W) == y_tr) if m is None else m.score(X_tr,
y_tr)
    a_te = np.mean(p.argmax(axis=1) == y_te)
```

```
    print(f"{ten:22s} train {a_tr:.4f} | test {a_te:.4f} | log-loss {log_loss
(y_te, p):.4f} | AUC {roc_auc_score(y_te, p, multi_class='ovr'):.4f}")
```

```
tu cai dat             train 0.8619 | test 0.8500 | log-loss 0.3536 | AUC 0.9
606
sklearn C=1e10         train 0.8619 | test 0.8500 | log-loss 0.3536 | AUC 0.9
606
sklearn mac dinh C=1   train 0.8619 | test 0.8556 | log-loss 0.3541 | AUC 0.9
607
sklearn one-vs-rest    train 0.8333 | test 0.8056 | log-loss 0.4772 | AUC 0.9
337
```

Mô hình	Accuracy huấn luyện	Accuracy kiểm tra	Log-loss kiểm tra	AUC kiểm tra (OvR)
Softmax Regression, tự cài đặt	86,2%	85,0%	0,3536	0,9606
Softmax, scikit-learn ($C = 10^{10}$)	86,2%	85,0%	0,3536	0,9606
Softmax, scikit-learn mặc định ($C = 1$)	86,2%	85,6%	0,3541	0,9607
One-vs-rest, scikit-learn ($C = 10^{10}$)	83,3%	80,6%	0,4772	0,9337

Có hai nhận xét quan trọng:

- Mô hình tự cài đặt và *`LogisticRegression`* cho kết quả trùng nhau đến chữ số thập phân thứ tư, xác nhận rằng ta đã cài đặt đúng thuật toán. Với $C = 1$, điều chuẩn hóa nhẹ gần như không đổi kết quả (khác một mẫu kiểm tra), vì dữ liệu không phân tách được và chỉ có hai đặc trưng, đúng như bảng ở mục 4.4.
- Softmax Regression vượt rõ rệt one-vs-rest trên cùng dữ liệu: accuracy kiểm tra cao hơn 4,4 điểm phần trăm, log-loss thấp hơn đáng kể ($0{,}354$ so với $0{,}477$) và AUC cao hơn. Lý do là K bộ phân loại nhị phân độc lập không biết nhau, nên đặc biệt yếu với lớp đạt nằm giữa (không có đường thẳng nào tách đạt khỏi cả rớt lẫn giỏi), và các xác suất $\sigma(s_k)$ không cộng lại bằng 1 nên phải chuẩn hóa lại (thư viện làm điều này trong *`predict_proba`*). Softmax huấn luyện chung K cột bằng một hàm mất mát duy nhất, và xác suất cộng lại bằng 1 ngay từ đầu (mục 1.3 và 2.3).

Chọn siêu tham số bằng kiểm định chéo

Tham số C (độ mạnh điều chuẩn hóa) và bậc của đặc trưng đa thức là các siêu tham số: không học được từ gradient mà phải chọn. Ta không được chọn chúng theo tập kiểm tra. Cách đúng là kiểm định chéo 5 lần trên tập huấn luyện: chia tập huấn luyện thành 5 phần, lần lượt dùng mỗi phần làm tập kiểm định và 4 phần còn lại để huấn luyện, rồi lấy trung bình điểm số. Việc chuẩn

hóa nằm **bên trong** pipeline để mỗi lần chia chỉ tính μ, σ từ phần huấn luyện của lần đó (tránh rò rỉ thông tin, mục 3.4):

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
for bac in (1, 2):
    for C in (0.001, 0.01, 0.1, 1, 10, 100):
        mo_hinh = make_pipeline(PolynomialFeatures(bac, include_bias=False),
StandardScaler(), LogisticRegression(C=C, max_iter=10000))
        acc = cross_val_score(mo_hinh, X_tr, y_tr, cv=cv)
        ll = -cross_val_score(mo_hinh, X_tr, y_tr, cv=cv, scoring="neg_log_lo
ss")
        print(f"bac {bac} | C = {C:<6} CV accuracy = {acc.mean():.4f} +- {acc
.std():.4f} | CV log-loss = {ll.mean():.4f}")
```

```
bac 1 | C = 0.001  CV accuracy = 0.7095 +- 0.0278 | CV log-loss = 0.9938
bac 1 | C = 0.01   CV accuracy = 0.7762 +- 0.0190 | CV log-loss = 0.7144
bac 1 | C = 0.1    CV accuracy = 0.8381 +- 0.0233 | CV log-loss = 0.4791
bac 1 | C = 1      CV accuracy = 0.8524 +- 0.0297 | CV log-loss = 0.3963
bac 1 | C = 10     CV accuracy = 0.8548 +- 0.0295 | CV log-loss = 0.3932
bac 1 | C = 100    CV accuracy = 0.8524 +- 0.0325 | CV log-loss = 0.3944
bac 2 | C = 0.001  CV accuracy = 0.7667 +- 0.0233 | CV log-loss = 0.9276
bac 2 | C = 0.01   CV accuracy = 0.8262 +- 0.0288 | CV log-loss = 0.6242
bac 2 | C = 0.1    CV accuracy = 0.8452 +- 0.0353 | CV log-loss = 0.4300
bac 2 | C = 1      CV accuracy = 0.8452 +- 0.0310 | CV log-loss = 0.3876
bac 2 | C = 10     CV accuracy = 0.8381 +- 0.0342 | CV log-loss = 0.3915
bac 2 | C = 100    CV accuracy = 0.8381 +- 0.0342 | CV log-loss = 0.3936
```

Kết quả cho thấy:

- C quá nhỏ (0,001) làm điều chuẩn hóa quá mạnh, mô hình bị "bó" (underfit): accuracy chỉ 71% đến 77% và log-loss gần mức đoán mò ($\ln 3 = 1{,}0986$), như đã dự báo ở bảng λ lớn ở mục 4.4.
- Từ $C \approx 1$ trở lên, kết quả gần như không đổi, vì với 2 đặc trưng và 420 mẫu, mô hình không có nguy cơ overfit.
- Đặc trưng bậc hai không cải thiện đáng kể: log-loss kiểm định chéo tốt nhất của bậc hai là 0,3876 (ở $C = 1$), chỉ thấp hơn 0,0056 so với bậc một (0,3932 ở $C = 10$), nhỏ hơn nhiều so với dao động giữa các lần chia (độ lệch chuẩn accuracy khoảng $\pm 0{,}03$). Điều này hợp lý: ba lớp xếp thành ba dải gần song song và mỗi lớp gần như lồi (mục 4.5), nên đường biên thẳng đã đủ. Khi hai mô hình ngang nhau, ta ưu tiên mô hình đơn giản hơn: bậc một với $C = 10$.

Chỉ sau khi chọn xong, ta mới huấn luyện lại trên toàn bộ tập huấn luyện và đánh giá **một lần** trên tập kiểm tra:

```
bac 1, C = 10: test accuracy = 0.8500 | log-loss = 0.3543
bac 2, C = 1: test accuracy = 0.8444 | log-loss = 0.3564
```

Mô hình được chọn ($k = 1, C = 10$) cho độ chính xác kiểm tra 85,0%, bằng mô hình ở mục 3.7, và mô hình bậc hai không tốt hơn (84,4%), xác nhận quyết định chọn mô hình đơn giản.

Báo cáo đầy đủ bằng các hàm có sẵn

Các chỉ số ở mục 5.3 và 5.4 đều có hàm dựng sẵn. Ta kiểm tra rằng kết quả trùng với phép tính tay:

```
from sklearn.metrics import confusion_matrix, classification_report, top_k_ac
curacy_score
print(confusion_matrix(y_te, clf.predict(X_te)))
print(classification_report(y_te, clf.predict(X_te), target_names=["rot", "da
t", "gioi"], digits=3))
print("top-2 =", top_k_accuracy_score(y_te, p_sk, k=2))
```

```
[[49 11  0]
 [ 9 48  3]
 [ 0  4 56]]
              precision    recall  f1-score   support

         rot      0.845     0.817     0.831        60
         dat      0.762     0.800     0.780        60
        gioi      0.949     0.933     0.941        60

    accuracy                          0.850       180
   macro avg      0.852     0.850     0.851       180
weighted avg      0.852     0.850     0.851       180

top-2 = 1.0
```

Đúng bằng các giá trị tính tay ở mục 5.3 (ba dòng theo lớp, macro, weighted) và top-2 ở mục 5.4. Dòng *accuracy* ở đây chính là micro-F1.

> **Lời khuyên thực hành.** Khi dùng Softmax Regression trên dữ liệu thật: (1) chuẩn hóa đặc trưng bên trong pipeline; (2) chọn siêu tham số như C và bậc đa thức bằng kiểm định chéo trên tập huấn luyện, nên dựa vào log-loss thay vì chỉ accuracy; (3) báo cáo kết quả một lần trên tập kiểm tra, gồm ma trận nhầm lẫn $K \times K$, precision, recall, F1 theo từng lớp (kèm macro-F1 nếu lớp lệch), log-loss và AUC, chứ không chỉ accuracy; (4) nếu các loại sai có hậu quả khác nhau, ra quyết định bằng ma trận chi phí trên các xác suất đã được kiểm tra hiệu chuẩn; (5) so sánh với one-vs-rest khi nghi ngờ lợi ích của mô hình xác suất chung.

Tóm tắt mục 5

- **Đường cong học và bản đồ xác suất.** J giảm từ $\ln 3$ xuống khoảng 0,37 trên tập huấn luyện, accuracy bão hòa sớm hơn J rất nhiều. Bản đồ xác suất cho thấy các vùng lồi, hai đường biên thẳng và dải chưa chắc chắn quanh biên (độ rộng đến

mức tỉ số xác suất 9:1 là $\ln 9 / \| \widetilde{\mathbf{w}}_k - \widetilde{\mathbf{w}}_j \|$). Trong 27 lỗi kiểm tra, 18 lỗi có độ tin cậy dưới 0,7; chỉ 3 lỗi "sai chắc chắn" nhưng chúng đóng góp 16% tổng mất mát.

- **Đánh giá trên tập huấn luyện và tập kiểm tra.** Hai tập cho kết quả gần nhau (85,0% kiểm tra, sai số chuẩn khoảng $\pm 2{,}7$ điểm), nên không có overfitting. Với ít dữ liệu và nhiều đặc trưng, overfitting xuất hiện rõ nhất qua log-loss kiểm tra (từ 0,4 lên hàng trăm khi $\lambda = 0$), và điều chuẩn hóa L2 giữ nó ổn định.
- **Ma trận nhầm lẫn $K \times K$ và các chỉ số theo từng lớp.** Precision_k chia cho tổng cột k, Recall_k chia cho tổng hàng k, $F1_k = \frac{2\text{TP}_k}{2\text{TP}_k + \text{FP}_k + \text{FN}_k}$. Macro-trung bình coi các lớp như nhau (nhạy với lớp hiếm), weighted theo support, micro bằng accuracy. Lỗi của mô hình chỉ xảy ra giữa các lớp kề nhau, và lớp đạt (nằm giữa) có F1 thấp nhất (0,780). Với ma trận chi phí $\mathbf{C}$, quyết định tối ưu là $\text{argmin}_j (\mathbf{pC})_j$, mở rộng ngưỡng τ^* của Logistic Regression.
- **Chất lượng xác suất.** Log-loss đa lớp (0,354 so với $\ln 3 = 1{,}099$), điểm Brier (0,211 so với 0,667), top-k accuracy (top-2 = 100%), hiệu chuẩn theo độ tin cậy (ECE = 0,050) và AUC one-vs-rest theo từng lớp (0,969; 0,920; 0,993, macro 0,961).
- **scikit-learn.** `LogisticRegression` đa lớp thực hiện đúng Softmax Regression (khớp mã tự cài đặt đến sai số $3 \cdot 10^{-4}$ về xác suất), với $\lambda = \frac{1}{Cn}$ và hệ số lưu dạng `coef_` ($K \times d$), `intercept_`. Softmax vượt one-vs-rest (85,0% so với 80,6%, log-loss 0,354 so với 0,477). Siêu tham số được chọn bằng kiểm định chéo trên tập huấn luyện (pipeline có chuẩn hóa), và tập kiểm tra chỉ dùng một lần ở cuối.

Câu hỏi kiểm tra nhanh

1. Vì sao ở epoch 0 ($\mathbf{W} = \mathbf{0}$), $J = \ln 3$ và accuracy bằng $\frac{1}{3}$ trên một tập có ba lớp cân bằng?
2. Cho ma trận nhầm lẫn $\mathbf{C} = \begin{bmatrix} 50 & 5 & 5 \\ 10 & 30 & 10 \\ 0 & 5 & 35 \end{bmatrix}$ (hàng là nhãn thật, cột là nhãn dự đoán). Tính accuracy, precision, recall và F1 của từng lớp, rồi macro-F1, weighted-F1 và micro-F1.
3. Một bộ phân loại ba lớp luôn dự đoán lớp 0 trên tập có 95% mẫu thuộc lớp 0 và 2,5% mỗi lớp còn lại. Tính accuracy và macro-recall. Điều này nói gì về việc chọn chỉ số?

4. Một mẫu có $\mathbf{p} = [0{,}5\ \ 0{,}3\ \ 0{,}2]$ và lớp thật là lớp 1. Mẫu này đúng ở top-k với k nhỏ nhất là bao nhiêu? Tính mất mát $-\ln q$ và $\| \mathbf{p} - \mathbf{y} \|^2$ (đóng góp vào Brier) của mẫu.

5. Với ma trận chi phí $\mathbf{c} = \begin{bmatrix} 0 & 3 & 3 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$, quyết định theo chi phí cho mẫu $\mathbf{p} = [0{,}4\ \ 0{,}35\ \ 0{,}25]$ là lớp nào? Khi nào quyết định theo chi phí khác với argmax?

6. Trong bảng hiệu chuẩn, nhóm $[0{,}7;\ 0{,}8)$ có 30 mẫu, độ tin cậy trung bình $0{,}765$ và tỉ lệ đúng thực tế $0{,}933$ trên tổng $n = 180$. Tính phần đóng góp của nhóm này vào ECE. Vì sao không nên kết luận mô hình "dè dặt" chỉ dựa vào nhóm này?

7. Lớp đạt có $\text{AUC}_{\text{đạt}} = 0{,}9197$ với 60 mẫu dương và 120 mẫu âm. Khoảng bao nhiêu cặp (dương, âm) được xếp đúng? Vì sao AUC của lớp này thấp hơn hai lớp kia?

8. Với $C = 1$ và $n = 420$, hệ số điều chuẩn hóa λ trong ký hiệu của ta là bao nhiêu? Nếu `clf.intercept_` và `clf.coef_` có kích thước $(3,)$ và $(3,2)$, ma trận $\mathbf{W}$ của ta có kích thước bao nhiêu và được dựng thế nào? Vì sao phải đặt `StandardScaler` bên trong pipeline khi kiểm định chéo?

Gợi ý đáp án

1. Khi $\mathbf{W} = \mathbf{0}$, mọi điểm số bằng 0 nên mọi $p_{ik} = \frac{1}{3}$ và mỗi mẫu bị phạt $-\ln\frac{1}{3} = \ln 3$, nên $J = \ln 3 \approx 1{,}0986$. Vì ba xác suất bằng nhau, *argmax* trả về lớp có chỉ số nhỏ nhất (lớp 0) cho mọi mẫu, nên chỉ các mẫu thật sự thuộc lớp 0 được đoán đúng, tức $\frac{1}{3}$ số mẫu.

2. $n = 150$, đường chéo $50 + 30 + 35 = 115$ nên accuracy $= 115/150 \approx 0{,}767$. Tổng cột là $60,\ 40,\ 50$ và tổng hàng là $60,\ 50,\ 40$. Precision $= 50/60,\ 30/40,\ 35/50 = 0{,}833;\ 0{,}750;\ 0{,}700$. Recall $= 50/60,\ 30/50,\ 35/40 = 0{,}833;\ 0{,}600;\ 0{,}875$. F1 $= \frac{2\cdot 50}{120}, \frac{2\cdot 30}{90}, \frac{2\cdot 35}{90} = 0{,}833;\ 0{,}667;\ 0{,}778$. Macro-F1 $= 0{,}759$; weighted-F1 $= \frac{60\cdot 0{,}833 + 50\cdot 0{,}667 + 40\cdot 0{,}778}{150} \approx 0{,}763$; micro-F1 $=$ accuracy $\approx 0{,}767$.

3. Accuracy $= 95\%$. Recall của lớp 0 là 1, của hai lớp còn lại là 0, nên macro-recall $= \frac{1+0+0}{3} \approx 0{,}333$. Accuracy rất cao nhưng mô hình vô dụng với hai lớp hiếm; đây là lý do khi lớp lệch phải nhìn ma trận nhầm lẫn, chỉ số theo từng lớp và macro-trung bình thay vì chỉ accuracy.

4. Lớp thật (lớp 1) có xác suất $0{,}3$, đứng thứ hai sau lớp 0 ($0{,}5$), nên mẫu đúng ở top-2 (và không đúng ở top-1). Mất mát $-\ln 0{,}3 \approx 1{,}204$. $\mathbf{y} = [0\ \ 1\ \ 0]$ nên $\| \mathbf{p} - \mathbf{y} \|^2 = 0{,}5^2 + 0{,}7^2 + 0{,}2^2 = 0{,}25 + 0{,}49 + 0{,}04 = 0{,}78$.

5. Chi phí kỳ vọng khi dự đoán rớt: $0{,}35 \cdot 1 + 0{,}25 \cdot 1 = 0{,}60$; đạt: $0{,}4 \cdot 3 + 0{,}25 \cdot 1 = 1{,}45$; giỏi: $0{,}4 \cdot 3 + 0{,}35 \cdot 1 = 1{,}55$. Vậy chọn lớp rớt, trùng với argmax ($0{,}4$) trong trường hợp này. Hai quy tắc khác nhau khi lớp có chi phí nhầm lẫn cao: ví dụ $\mathbf{p} = [0{,}2\ \ 0{,}5\ \ 0{,}3]$ có argmax là đạt nhưng chi phí kỳ vọng là $[0{,}8\ \ 0{,}9\ \ 1{,}1]$ nên quy tắc theo chi phí chọn rớt, vì bỏ sót một sinh viên rớt tốn gấp 3 lần.

6. Đóng góp của nhóm là $\frac{30}{180} \cdot |0{,}933 - 0{,}765| = 0{,}1667 \cdot 0{,}168 \approx 0{,}028$ (khoảng $0{,}028$ trên tổng ECE $0{,}050$). Nhóm chỉ có 30 mẫu nên tỉ lệ $0{,}933$ có sai số chuẩn khoảng $\sqrt{0{,}933 \cdot 0{,}067/30} \approx 0{,}046$, và các nhóm lân cận có kết quả theo chiều ngược lại (nhóm $[0{,}6;\ 0{,}7)$ có tỉ lệ đúng $0{,}600$ thấp hơn độ tin cậy $0{,}655$); do đó cần nhìn cả bảng chứ không dựa vào một nhóm.

7. Số cặp đúng khoảng $0{,}9197 \times 60 \cdot 120 = 0{,}9197 \times 7200 \approx 6622$ cặp trên 7200. Lớp đạt nằm giữa nên điểm số p_1 phải phân biệt đạt với hai lớp ở hai phía (rớt và giỏi), trong khi p_0 chỉ cần phân biệt rớt với phần còn lại ở một phía, nên xếp hạng khó hơn.

8. $\lambda = \frac{1}{Cn} = \frac{1}{1 \cdot 420} \approx 0{,}0024$. Ma trận $\mathbf{W}$ có kích thước $(d + 1) \times K = 3 \times 3$, dựng bằng `W = np.vstack([clf.intercept_, clf.coef_.T])` (hàng đầu là `intercept_`, hai hàng sau là `coef_` chuyển vị). Chuẩn hóa phải nằm trong pipeline để mỗi lần chia chỉ tính μ, σ từ phần huấn luyện của lần đó; nếu chuẩn hóa toàn bộ dữ liệu trước thì thông tin của phần kiểm định đã "rò rỉ" vào huấn luyện và điểm kiểm định chéo bị lạc quan.

6. Tổng kết và mở rộng

Qua năm mục, ta đã đi một vòng đầy đủ với Softmax Regression: từ động cơ (mục 1), mô hình (mục 2), huấn luyện (mục 3), cơ chế học (mục 4) đến đánh giá (mục 5). Mục cuối này làm ba việc: gom lại ưu điểm và hạn chế để biết khi nào nên dùng mô hình; đặt Softmax Regression cạnh Logistic Regression và one-vs-rest để thấy mối quan hệ giữa chúng; và chỉ ra hướng đi tiếp theo, nơi Softmax Regression trở thành lớp đầu ra của mạng nơ-ron nhiều lớp. Cuối cùng là bộ bài tập thực hành. Bảng sau nhắc lại mạch kiến thức và kết quả chính đã có:

Mục	Câu hỏi chính	Kết quả cần nhớ
1	Vì sao Logistic Regression chưa đủ khi $K > 2$?	Nhãn số làm hồi quy phụ thuộc cách đánh số; one-vs-rest cho xác suất không cộng lại bằng 1; cần mô hình xác suất chung với nhãn one-hot $\mathbf{Y}$
2	Mô hình là gì?	$\mathbf{S} = \mathbf{XW}$, $\mathbf{P} = \text{softmax}(\mathbf{S})$; $K = 2$ trở về sigmoid; vùng quyết định lồi; $\mathbf{W}$ dư thừa tham số
3	Huấn luyện thế nào?	Cross-entropy $J(\mathbf{W}) = -\frac{1}{n}\sum_i \ln q_i$; $\nabla_{\mathbf{W}} J = \frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$; $J(\mathbf{0}) = \ln K$
4	Mô hình học ra sao?	Kéo lớp đúng, đẩy các lớp sai theo p_{ik}; J lồi nhưng Hessian suy biến ($d + 1$ hướng phẳng); phân tách hoàn hảo thì $\mathbf{W} \to \infty$ (cần L2); vùng quyết định luôn lồi
5	Mô hình tốt đến đâu?	Ma trận nhầm lẫn $K \times K$, F1 theo lớp, log-loss, hiệu chuẩn, AUC; trên 600 sinh viên: 85,0% kiểm tra, log-loss 0,354

Ta giữ quy ước ký hiệu cũ: chữ thường in đậm là vector, chữ hoa in đậm là ma trận; $\mathbf{x}_i$, $\mathbf{s}_i$, $\mathbf{p}_i$, $\mathbf{y}_i$ là các vector hàng; $\mathbf{w}_k$ là cột thứ k của $\mathbf{W}$.

6.1. Ưu điểm và hạn chế của Softmax Regression

Ưu điểm

- **Xác suất cho mọi lớp, cộng lại bằng 1.** Đầu ra $\mathbf{p}_i$ là một phân phối thật sự trên K lớp (mục 2.3), khác với one-vs-rest (mục 6.2). Ta dùng được các chỉ số xác suất như log-loss, điểm Brier, hiệu chuẩn (mục 5.4) và ra quyết định theo chi phí (mục 5.3).
- **Học chung K lớp bằng một hàm mất mát duy nhất.** Các cột $\mathbf{w}_k$ được huấn luyện cùng lúc, "cạnh tranh" nhau qua mẫu số chung của softmax. Nhờ vậy lớp nằm giữa (lớp đạt) vẫn được học tốt, điều mà các bộ phân loại độc lập làm kém: trên cùng dữ liệu, Softmax đạt 85,0% so với 80,6% của one-vs-rest (mục 5.5).
- **Có cơ sở xác suất rõ ràng.** Cross-entropy chính là âm log-hợp lý của phân phối loại (mục 3.2), và hàm mất mát không phụ thuộc cách đặt tên các lớp.

- **Bài toán tối ưu lồi.** J lồi (mục 4.3) nên mọi cực tiểu địa phương là cực tiểu toàn cục, Gradient Descent hội tụ từ mọi khởi tạo, và ma trận xác suất $\mathbf{P}$ cuối cùng là duy nhất.
- **Gradient rất gọn và dễ cài đặt.** $\frac{1}{n}\mathbf{X}^T(\mathbf{P}-\mathbf{Y})$ có cùng dạng với hồi quy tuyến tính và Logistic Regression (mục 3.3); toàn bộ thuật toán gồm vài chục dòng NumPy (mục 3.7). Chi phí mỗi epoch là $O(ndK)$, tuyến tính theo số mẫu, số đặc trưng và số lớp.
- **Có thể diễn giải.** Hiệu hai cột trọng số cho biết ảnh hưởng của từng đặc trưng lên tỉ lệ giữa hai lớp (xem bên dưới).
- **Nền tảng cho các mô hình sâu hơn.** Softmax là lớp đầu ra chuẩn của mạng nơ-ron phân loại nhiều lớp (mục 6.3).

Diễn giải trọng số bằng tỉ lệ odds giữa hai lớp

Từ $p_{ik}/p_{il} = e^{s_{ik}-s_{il}}$ (mục 2.5), tăng đặc trưng x_j thêm một đơn vị làm tỉ lệ odds giữa lớp k và lớp l nhân với $e^{w_{jk}-w_{jl}}$, bất kể các đặc trưng khác và các lớp khác. Với ma trận $\mathbf{W}_{\text{gốc}}$ trên thang đo gốc (mục 4.2):

$$\mathbf{W}_{\text{gốc}} \approx \begin{bmatrix} 5{,}628 & 1{,}877 & -7{,}505 \\ -1{,}408 & 0{,}081 & 1{,}327 \\ 0{,}812 & -0{,}244 & -0{,}569 \end{bmatrix}$$

ta có bảng hệ số nhân:

Thay đổi	Odds đạt : rớt	Odds giỏi : đạt	Odds giỏi : rớt
Thêm 1 giờ tự học (x_1)	$e^{1{,}489} \approx 4{,}43$	$e^{1{,}246} \approx 3{,}48$	$e^{2{,}735} \approx 15{,}4$
Thêm 1 buổi vắng (x_2)	$e^{-1{,}056} \approx 0{,}348$	$e^{-0{,}325} \approx 0{,}722$	$e^{-1{,}381} \approx 0{,}251$

Chẳng hạn, mỗi giờ học thêm nhân odds "đạt so với rớt" lên 4,43 lần, còn mỗi buổi vắng thêm làm odds đó giảm còn 0,348 lần (giảm khoảng 65%). Kiểm tra bằng số: sinh viên học 6 giờ, vắng 2 buổi có $\mathbf{p} \approx [0{,}041 \;\; 0{,}890 \;\; 0{,}069]$, odds đạt:rớt $\approx 21{,}5$; thêm một giờ học (7 giờ) thì $\mathbf{p} \approx [0{,}008 \;\; 0{,}781 \;\; 0{,}211]$ và odds đạt:rớt $\approx 95{,}5 = 21{,}5 \times 4{,}43$. Lưu ý: mỗi hệ số chỉ có nghĩa khi so hai lớp với nhau, và đây là hệ quả của tính dư thừa tham số (mục 4.3): trọng số riêng lẻ của một cột không có ý nghĩa vì cộng cùng một vector vào mọi cột không đổi mô hình.

Hạn chế và cách khắc phục

Hạn chế	Nguyên nhân	Hướng khắc phục
Vùng quyết định luôn lồi, biên là các đường thẳng	Điểm số tuyến tính theo đặc trưng (mục 2.5, 4.5)	Thêm đặc trưng đa thức/tương tác; mạng nơ-ron (mục 6.3)

Hạn chế	Nguyên nhân	Hướng khắc phục
Nghiệm $\mathbf{W}$ không duy nhất	Softmax bất biến khi cộng cùng một vector vào mọi cột, Hessian suy biến (mục 4.3)	Khởi tạo $\mathbf{0}$; cố định một lớp tham chiếu; điều chuẩn hóa L2
$\mathbf{W} \to \infty$ khi dữ liệu phân tách hoàn hảo	$J \to 0$ nhưng không đạt (mục 4.4)	Điều chuẩn hóa L2; dừng sớm; thêm dữ liệu
Nhạy với thang đo đặc trưng	Số điều kiện của Hessian lớn (mục 4.2, 4.3)	Chuẩn hóa đặc trưng (chỉ dùng thống kê của tập huấn luyện)
Mẫu ngoại lai hoặc nhãn nhiễu kéo mạnh	Gradient tỉ lệ với $\| \mathbf{x}_i \| \cdot \| \mathbf{p}_i - \mathbf{y}_i \|$ (mục 4.1)	Điều chuẩn hóa; làm sạch dữ liệu; hàm mất mát bền vững hơn
Lớp mất cân bằng	Cross-entropy trung bình ưu tiên lớp đông (mục 5.3)	Trọng số lớp, lấy mẫu lại, đánh giá bằng macro-F1
Chi phí tăng theo số lớp K	Có $(d+1)K$ tham số và phải tính softmax trên cả K lớp	Softmax phân cấp, lấy mẫu lớp âm (khi K rất lớn)
Giả định các lớp loại trừ nhau và độc lập với phương án khác	Công thức softmax (xem bên dưới)	Dùng K sigmoid độc lập (nhiều nhãn); mô hình khác (nested logit)

Hai giả định ẩn trong bảng này cần nói rõ hơn.

Một mẫu có đúng một nhãn. Softmax ép $\sum_k p_k = 1$ nên các lớp là **loại trừ nhau**: tăng xác suất của một lớp là giảm xác suất của các lớp còn lại. Với bài toán gán **nhiều nhãn** cho mỗi mẫu (ví dụ gắn nhiều thẻ cho một bài báo), ta không dùng softmax mà dùng K sigmoid độc lập, mỗi nhãn một cross-entropy nhị phân, và tổng xác suất không cần bằng 1. Đó cũng chính là điểm mà one-vs-rest tỏ ra phù hợp (mục 6.2).

Tính độc lập với phương án không liên quan (IIA). Từ $p_k/p_l = e^{s_k - s_l}$, tỉ lệ giữa hai lớp chỉ phụ thuộc điểm số của chính hai lớp đó. Với $\mathbf{s} = [1 \;\; 2 \;\; 3]$, ta có $\mathbf{p} \approx [0{,}090 \;\; 0{,}245 \;\; 0{,}665]$ và $p_1/p_0 = e \approx 2{,}72$; bỏ lớp thứ ba đi, hai lớp còn lại có $\mathbf{p} \approx [0{,}269 \;\; 0{,}731]$, tỉ lệ vẫn bằng e. Điều này đôi khi không thực tế. Ví dụ kinh điển: một người đi làm chọn giữa ô tô và xe buýt đỏ với xác suất $\frac{1}{2}$ mỗi loại. Khi thêm xe buýt xanh hoàn toàn giống xe buýt đỏ, người đó thực tế sẽ chia đôi phần xe buýt, tức $\left[\frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right]$. Nhưng mô hình softmax, giữ nguyên tỉ lệ ô tô : xe buýt đỏ $= 1$ và cho hai xe buýt cùng điểm số, sẽ dự đoán $\left[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right]$. Khi các lớp có thể rất giống nhau (hoặc lồng nhau), cần cẩn trọng với giả định này; ở bài toán sinh viên rớt/đạt/giỏi, ba lớp phân biệt rõ nên giả định này tạm chấp nhận được.

6.2. So sánh Logistic Regression, one-vs-rest và Softmax Regression

Bảng so sánh

	Logistic Regression	One-vs-rest (K Logistic)	Softmax Regression
Số lớp	2	$K \geq 2$	$K \geq 2$
Tham số	vector $\mathbf{w}$ $(d+1)$	K vector $\mathbf{w}_k$ (ma trận $\mathbf{W}$, huấn luyện **riêng lẻ**)	ma trận $\mathbf{W}$ $((d+1) \times K)$, huấn luyện **chung**
Điểm số	$s = \mathbf{x}\mathbf{w}$	$s_k = \mathbf{x}\mathbf{w}_k$	$\mathbf{s} = \mathbf{x}\mathbf{W}$
Hàm biến đổi	sigmoid $\sigma(s)$	sigmoid $\sigma(s_k)$ cho từng k	softmax trên cả vector $\mathbf{s}$
Đầu ra	$P(y=1)$	K xác suất **độc lập**, $\sum_k \sigma(s_k) \neq 1$	phân phối trên K lớp, $\sum_k p_k = 1$
Hàm mất mát	cross-entropy nhị phân	K cross-entropy nhị phân độc lập	cross-entropy nhiều lớp (một hàm duy nhất)
Gradient	$\frac{1}{n}\mathbf{X}^T(\boldsymbol{\sigma} - \mathbf{y})$	cột k: $\frac{1}{n}\mathbf{X}^T(\sigma(\mathbf{s}_{:,k}) - \mathbf{y}_{:,k})$	$\frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$
Nghiệm	$\mathbf{w}$ duy nhất (nếu không phân tách hoàn hảo)	mỗi $\mathbf{w}_k$ duy nhất	$\mathbf{P}$ duy nhất, $\mathbf{W}$ xác định sai khác $\mathbf{v}\mathbf{1}_K^T$
Dự đoán	$\hat{y} = \mathbb{1}[p \geq 0{,}5]$	$\text{argmax}_k \sigma(s_k)$	$\text{argmax}_k p_k$
Đa nhãn	không	**có** (mỗi bộ phân loại là một nhãn)	không (một nhãn mỗi mẫu)

Softmax Regression với $K = 2$ chính là Logistic Regression

Ở mục 2.3 ta đã thấy softmax hai lớp cho $p_1 = \sigma(s_1 - s_0)$. Phần huấn luyện cũng khớp: gradient theo $\mathbf{w}_1$ là $\frac{1}{n}\mathbf{X}^T(\mathbf{p}_{:,1} - \mathbf{y}_{:,1})$ và theo $\mathbf{w}_0$ là đúng số đối của nó (vì $\mathbf{p}_{:,0} - \mathbf{y}_{:,0} = -(\mathbf{p}_{:,1} - \mathbf{y}_{:,1})$). Một bước Gradient Descent với tốc độ học η cho

$$\mathbf{w}_1 \leftarrow \mathbf{w}_1 - \eta\,\nabla_{\mathbf{w}_1} J, \qquad \mathbf{w}_0 \leftarrow \mathbf{w}_0 + \eta\,\nabla_{\mathbf{w}_1} J \quad \Rightarrow \quad (\mathbf{w}_1 - \mathbf{w}_0) \leftarrow (\mathbf{w}_1 - \mathbf{w}_0) - 2\eta\,\nabla_{\mathbf{w}_1} J$$

Đặt $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_0$, đây đúng là bước Gradient Descent của Logistic Regression với tốc độ học 2η, vì $\nabla_{\mathbf{w}_1} J = \frac{1}{n}\mathbf{X}^T(\boldsymbol{\sigma}(\mathbf{X}\mathbf{w}) - \mathbf{y})$. Vậy hai quá trình huấn luyện cho **cùng quỹ đạo** của $\mathbf{w}_1 - \mathbf{w}_0$ nếu tốc độ học của Logistic gấp đôi của Softmax. Ta kiểm chứng bằng số trên hai lớp rớt và giỏi:

```
def sigmoid(s): return 0.5 * (1 + np.tanh(0.5 * s))            # sigmoid ổn định

def huan_luyen_logistic(Xb, t, eta=0.5, so_epoch=2000):
    w = np.zeros(Xb.shape[1])
    for _ in range(so_epoch):
        w = w - eta * Xb.T @ (sigmoid(Xb @ w) - t) / len(t)
    return w
```

```
m = np.isin(y_tr, [0, 2])                                          # chỉ giữ
lớp rớt và lớp giỏi
Z2, t2 = Zb_tr[m], (y_tr[m] == 2).astype(float)
W2l, _ = huan_luyen(Z2, one_hot(t2.astype(int), 2), eta=0.5, so_epoch=300)
# Softmax, K = 2
w_lg = huan_luyen_logistic(Z2, t2, eta=1.0, so_epoch=300)
# Logistic, eta gấp đôi
print("w1 - w0 (softmax) =", (W2l[:, 1] - W2l[:, 0]).round(4), "| w (logistic
) =", w_lg.round(4))
print("max |khac biet| =", np.abs(W2l[:, 1] - W2l[:, 0] - w_lg).max())
```

```
w1 - w0 (softmax) = [ 0.2941  4.7815 -1.6007] | w (logistic) = [ 0.2941  4.78
15 -1.6007]
max |khac biet| = 8.881784197001252e-16
```

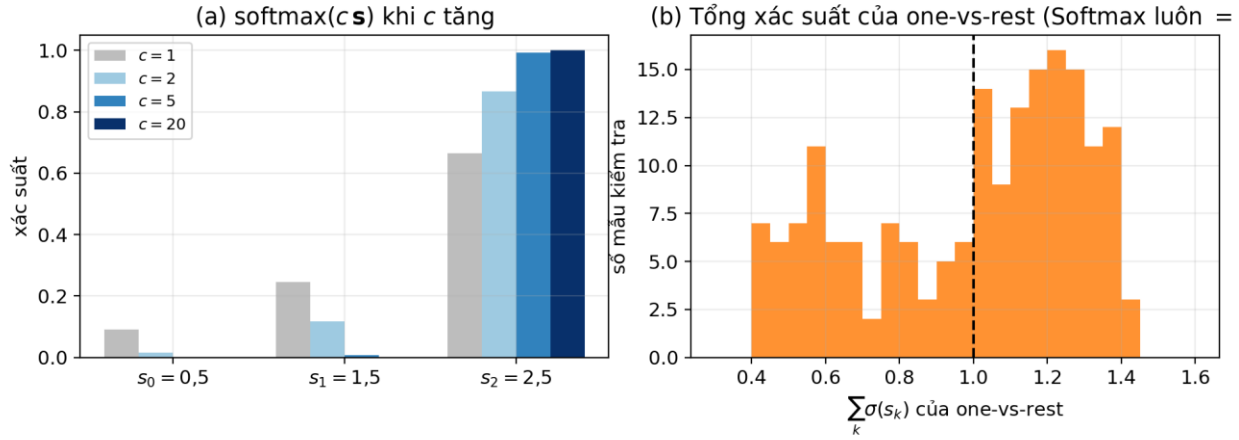
Hai kết quả trùng nhau đến sai số làm tròn số học. Cũng vì khởi tạo từ $\mathbf{0}$, $\mathbf{w}_1 = -\mathbf{w}_0 = \mathbf{w}/2$ ở mọi bước (đúng tính chất "tổng các cột không đổi" ở mục 3.3). Logistic Regression do đó chỉ là trường hợp đặc biệt, gọn hơn, của Softmax Regression: nó dùng một vector thay vì hai vì một trong hai cột là dư thừa (mục 2.5).

One-vs-rest và Softmax trên cùng dữ liệu

Ta huấn luyện $K = 3$ bộ phân loại Logistic độc lập trên dữ liệu 600 sinh viên (đặc trưng chuẩn hóa, $\eta = 0{,}5$, 2000 epoch, mỗi bộ phân loại "lớp k hay không") rồi so với Softmax:

```
W_ovr = np.column_stack([huan_luyen_logistic(Zb_tr, (y_tr == k).astype(float)
) for k in range(3)])
for k in range(3):
    t = (y_tr == k).astype(float); p = np.clip(sigmoid(Zb_tr @ W_ovr[:, k]),
1e-12, 1 - 1e-12)
    print(f"bo phan loai {k}: J = {-np.mean(t * np.log(p) + (1 - t) * np.log(
1 - p)):.3f}")
P_ovr = sigmoid(Zb_te @ W_ovr)                                     # xác suất
độc lập của từng bộ phân loại
tong = P_ovr.sum(axis=1)
print(f"tong xac suat: trung binh {tong.mean():.3f}, nho nhat {tong.min():.3f
}, lon nhat {tong.max():.3f}")
print("ti le mau co tong ngoai [0.8, 1.2]:", np.mean((tong < 0.8) | (tong > 1
.2)).round(3),
      "| khong lop nao >= 0.5:", int((P_ovr < 0.5).all(axis=1).sum()), "| hon
mot lop >= 0.5:", int(((P_ovr >= 0.5).sum(axis=1) > 1).sum()))
P_ovr_n = P_ovr / tong[:, None]                                    # chuẩn hó
a lại để có thể so log-loss
P_sm = softmax(Zb_te @ W37)
for ten, P in [("one-vs-rest", P_ovr_n), ("softmax", P_sm)]:
    print(f"{ten:12s} test acc = {np.mean(P.argmax(axis=1) == y_te):.4f} | ma
cro-F1 = {f1_score(y_te, P.argmax(axis=1), average='macro'):.4f}"
          f" | log-loss = {log_loss(y_te, P):.4f} | AUC = {roc_auc_score(y_te
, P, multi_class='ovr'):.4f}")
```

```
bo phan loai 0: J = 0.154
bo phan loai 1: J = 0.609
bo phan loai 2: J = 0.221
tong xac suat: trung binh 0.995, nho nhat 0.410, lon nhat 1.435
ti le mau co tong ngoai [0.8, 1.2]: 0.606 | khong lop nao >= 0.5: 48 | hon mo
t lop >= 0.5: 3
one-vs-rest  test acc = 0.8056 | macro-F1 = 0.7982 | log-loss = 0.4772 | AUC
= 0.9337
softmax      test acc = 0.8500 | macro-F1 = 0.8507 | log-loss = 0.3536 | AUC
= 0.9606
```



Hình 19. (a) Softmax với "nhiệt độ" c: khi c tăng, $softmax(c\,\boldsymbol{s})$ dồn xác suất vào lớp có điểm số lớn nhất (với $\boldsymbol{s} = [0{,}5\ \ 1{,}5\ \ 2{,}5]$). (b) Tổng các xác suất của one-vs-rest trên tập kiểm tra, rất khác 1; tổng xác suất của Softmax luôn bằng 1.

Các điểm cần chú ý:

- **Bộ phân loại "đạt" gần như không học được.** Mất mát của bộ phân loại $k = 1$ là 0,609, chỉ thấp hơn $\ln 2 \approx 0{,}693$ một chút, trong khi hai bộ phân loại còn lại có 0,154 và 0,221. Lý do: lớp đạt nằm giữa hai lớp kia, không có đường thẳng nào tách "đạt" khỏi "rớt và giỏi" cùng lúc (đúng hạn chế ở mục 4.5, nay xảy ra cho từng bộ phân loại), nên bộ phân loại này chỉ có thể học một hướng yếu. Softmax không gặp vấn đề này vì chỉ cần **so sánh** điểm số của đạt với từng lớp kề, không cần tách đạt khỏi cả hai.
- **Xác suất không cộng lại bằng** 1 (Hình 19b). Tổng nằm từ 0,41 đến 1,44 và 60,6% mẫu kiểm tra có tổng ngoài $[0{,}8;\ 1{,}2]$. Có 48 mẫu (26,7%) mà không bộ phân loại nào cho xác suất từ 0,5 trở lên, và 3 mẫu mà nhiều hơn một bộ phân loại cùng "nhận". Đúng như đã dự báo ở mục 1.3. Để so sánh công bằng ta phải chuẩn hóa lại (chia cho tổng), một thủ thuật không có cơ sở xác suất rõ ràng.

- **Hệ quả về chất lượng.** Softmax tốt hơn ở mọi chỉ số: accuracy (85,0% so với 80,6%), macro-F1 (0,851 so với 0,798), log-loss (0,354 so với 0,477) và AUC (0,961 so với 0,934).

One-vs-rest vẫn có chỗ đứng: nó dùng được cho bài toán **đa nhãn**, cho phép huấn luyện song song K bộ phân loại và thay thế bộ phân loại nhị phân bằng loại khác (ví dụ SVM), và tránh được giả định IIA. Nhưng khi các lớp loại trừ nhau và ta cần xác suất đúng nghĩa, Softmax là lựa chọn mặc định.

Perceptron nhiều lớp là giới hạn "cứng" của Softmax

Giống như Perceptron là "phiên bản cứng" của Logistic Regression, Perceptron nhiều lớp là giới hạn của Softmax khi ta nhân điểm số lên một hệ số $c \to \infty$ (đóng vai trò nghịch đảo nhiệt độ). Với $\text{softmax}(c\,\mathbf{s})$, xác suất dồn vào lớp có điểm số lớn nhất khi c tăng (Hình 19a). Mất mát của một mẫu chia cho c là

$$\frac{1}{c}\,\ell_i(c\,\mathbf{s}_i) = -s_{i,y_i} + \frac{1}{c}\ln\sum_j e^{c\,s_{ij}} \underset{c\to\infty}{\to} \max_j s_{ij} - s_{i,y_i}$$

vì $\max_j s_{ij} \le \frac{1}{c}\ln\sum_j e^{c\,s_{ij}} \le \max_j s_{ij} + \frac{\ln K}{c}$. Giới hạn $\max_j s_{ij} - s_{i,y_i}$ chính là mất mát của Perceptron nhiều lớp: bằng 0 nếu lớp thật có điểm số lớn nhất, ngược lại bằng khoảng cách tới lớp đang thắng. Gradient theo điểm số cũng tiến tới $\mathbf{e}_{\hat{y}_i} - \mathbf{y}_i$, với $\mathbf{e}_{\hat{y}_i}$ là vector one-hot của lớp đang thắng, tức là "kéo lớp đúng, đẩy đúng một lớp đang thắng" đã nhắc ở mục 4.1. Với mẫu có $\mathbf{s} = [0{,}5\ \ 1{,}5\ \ 2{,}5]$ và lớp thật là lớp 1:

Hệ số c	$\mathbf{p} = \text{softmax}(c\,\mathbf{s})$	ℓ_i/c
1	[0,090 0,245 0,665]	1,408
2	[0,016 0,117 0,867]	1,072
5	[0,000 0,007 0,993]	1,001
20	[0,000 0,000 1,000]	1,000
$\to \infty$	one-hot của lớp 2	$\max_j s_j - s_1 = 2{,}5 - 1{,}5 = 1$

Ngược lại, khi $c \to 0$, softmax tiến tới phân phối đều. Hệ số c này thường gọi là nghịch đảo của *nhiệt độ* và xuất hiện trong nhiều kỹ thuật học sâu hiện đại.

6.3. Từ Softmax Regression đến mạng nơ-ron nhiều lớp và học sâu

Softmax Regression là mạng nơ-ron không có lớp ẩn

Softmax Regression chạy trên đặc trưng $\mathbf{x}$ có sẵn. Ở mục 4.5, khi dữ liệu là ba vòng tròn đồng tâm, ta phải tự đoán đặc trưng đa thức bậc hai. Ý tưởng của mạng nơ-ron nhiều lớp là để chính mô hình **học ra đặc trưng**. Với một lớp ẩn gồm m nơ-ron:

$$\mathbf{H} = \tanh\big(\tilde{\mathbf{X}}\mathbf{W}_1\big), \qquad \mathbf{S} = \tilde{\mathbf{H}}\mathbf{W}_2, \qquad \mathbf{P} = \text{softmax}(\mathbf{S})$$

trong đó $\widetilde{\mathbf{X}}$ và $\widetilde{\mathbf{H}}$ là $\mathbf{X}$ và $\mathbf{H}$ sau khi thêm cột 1, $\mathbf{W}_1$ có kích thước $(d+1)\times m$ (mỗi cột là một nơ-ron ẩn), $\mathbf{W}_2$ có kích thước $(m+1)\times K$, và tanh áp dụng lên từng phần tử. Lớp cuối $\mathbf{P} = \text{softmax}\big(\widetilde{\mathbf{H}}\mathbf{W}_2\big)$ **chính là Softmax Regression**, nhưng chạy trên đặc trưng $\mathbf{H}$ do mạng tự học thay vì $\mathbf{X}$. Nếu bỏ lớp ẩn (coi $\mathbf{H} = \mathbf{X}$), ta trở về mô hình ở mục 2. Hàm mất mát vẫn là cross-entropy $J = -\frac{1}{n}\sum_i \ln q_i$ ở mục 3.1. Số tham số là $(d+1)m + (m+1)K$.

Lan truyền ngược: phần lớn công thức đã có

Ta cần gradient của J theo $\mathbf{W}_1$ và $\mathbf{W}_2$. Phép tính bắt đầu từ chính kết quả ở mục 3.3: gradient theo ma trận điểm số của lớp ra là

$$\frac{\partial J}{\partial \mathbf{S}} = \frac{1}{n}(\mathbf{P} - \mathbf{Y}) =: \mathbf{D}_2$$

Từ đó gradient của lớp ra có đúng dạng của Softmax Regression, với $\widetilde{\mathbf{H}}$ thay cho $\mathbf{X}$:

$$\nabla_{\mathbf{W}_2} J = \widetilde{\mathbf{H}}^T \mathbf{D}_2$$

Để lan truyền về lớp ẩn, gọi $\mathbf{V}$ là $\mathbf{W}_2$ bỏ hàng đầu (hàng hệ số chệch). Theo quy tắc đạo hàm hàm hợp và $\tanh'(z) = 1 - \tanh^2(z)$:

$$\mathbf{D}_1 = \frac{\partial J}{\partial\big(\widetilde{\mathbf{X}}\mathbf{W}_1\big)} = (\mathbf{D}_2\mathbf{V}^T) \odot (\mathbf{1} - \mathbf{H} \odot \mathbf{H}), \qquad \nabla_{\mathbf{W}_1} J = \widetilde{\mathbf{X}}^T \mathbf{D}_1$$

trong đó $\odot$ là phép nhân từng phần tử. Kiểm tra kích thước: $\mathbf{D}_2$ là $n \times K$, $\mathbf{V}^T$ là $K \times m$, nên $\mathbf{D}_1$ là $n \times m$ và $\nabla_{\mathbf{W}_1} J$ là $(d+1)\times m$, khớp với $\mathbf{W}_1$. Quy tắc cập nhật vẫn là Gradient Descent: $\mathbf{W}_1 \leftarrow \mathbf{W}_1 - \eta\nabla_{\mathbf{W}_1} J$, $\mathbf{W}_2 \leftarrow \mathbf{W}_2 - \eta\nabla_{\mathbf{W}_2} J$. Như vậy, việc ta đã nhận ra $\mathbf{P} - \mathbf{Y}$ là sai số ở lớp ra (mục 3.3) chính là "bước đầu tiên" của lan truyền ngược trong mọi mạng phân loại nhiều lớp.

Cài đặt và kết quả trên ba vòng tròn đồng tâm

```
def mang_softmax(X, Y, so_an=8, eta=0.5, so_epoch=5000, seed=0):
    """Mạng nơ-ron một lớp ẩn (tanh), đầu ra softmax, huấn luyện bằng lan truyền ngược (batch)."""
    rng = np.random.default_rng(seed)
    n, K = len(X), Y.shape[1]
    Xb = them_bias(X)
    W1 = rng.normal(0, 1, (Xb.shape[1], so_an))          # (d+1) x m: mỗi cột là một nơ-ron ẩn
    W2 = rng.normal(0, 1, (so_an + 1, K))                # (m+1) x K: lớp ra chính là Softmax Regression
    lich_su = []
    for ep in range(so_epoch):
        H = np.tanh(Xb @ W1)                             # lớp ẩn
        Hb = them_bias(H)
        P = softmax(Hb @ W2)                             # xác suất đầu ra
        lich_su.append(-np.mean(np.sum(Y * np.log(np.clip(P, 1e-12, 1)), axis=1)))
        D2 = (P - Y) / n                                 # gradient theo điểm
```

```
số đầu ra: (P - Y)/n
        g_W2 = Hb.T @ D2                                    # lớp ra: dạng giống
hệt Softmax Regression
        dH = (D2 @ W2[1:].T) * (1 - H ** 2)                 # lan truyền ngược v
ề lớp ẩn (tanh' = 1 - tanh^2)
        g_W1 = Xb.T @ dH
        W2 = W2 - eta * g_W2
        W1 = W1 - eta * g_W1
    return W1, W2, lich_su

def du_doan_mang(X, W1, W2):
    return softmax(them_bias(np.tanh(them_bias(X) @ W1)) @ W2)
```

Ta dùng bộ dữ liệu ba vòng tròn ở mục 4.5 (*sinh_vong*, chia 70/30, chuẩn hóa), kiểm tra gradient bằng đạo hàm số rồi huấn luyện mạng với $m = 2,4,8$ nơ-ron ẩn, mỗi cỡ từ 5 khởi tạo ngẫu nhiên khác nhau:

```
Xv, yv = sinh_vong()
Xa, Xb_, ya, yb = chia_train_test(Xv, yv)
Za, Zb_, _, _ = chuan_hoa(Xa, Xb_); Ya = one_hot(ya, 3)

def J_mang(X, Y, W1, W2):
    P = softmax(them_bias(np.tanh(them_bias(X) @ W1)) @ W2); return -np.mean(
np.sum(Y * np.log(P), axis=1))
def gradient_mang(X, Y, W1, W2):
    n = len(X); Xb = them_bias(X); H = np.tanh(Xb @ W1); Hb = them_bias(H)
    D2 = (softmax(Hb @ W2) - Y) / n
    return Xb.T @ ((D2 @ W2[1:].T) * (1 - H ** 2)), Hb.T @ D2

rng = np.random.default_rng(1); Xs, Ys = Za[:20], Ya[:20]
W1 = rng.normal(0, 1, (3, 4)); W2 = rng.normal(0, 1, (5, 3)); e = 1e-6; sai =
0
g1, g2 = gradient_mang(Xs, Ys, W1, W2)
for A, g in ((W1, g1), (W2, g2)):
    for i in range(A.shape[0]):
        for j in range(A.shape[1]):
            A[i, j] += e; jp = J_mang(Xs, Ys, W1, W2); A[i, j] -= 2 * e; jm =
J_mang(Xs, Ys, W1, W2); A[i, j] += e
            sai = max(sai, abs((jp - jm) / (2 * e) - g[i, j]))
print("gradient check max err", sai)
for m in (2, 4, 8):
    accs, Js = [], []
    for seed in range(5):
        W1, W2, ls = mang_softmax(Za, Ya, so_an=m, eta=0.5, so_epoch=5000, se
ed=seed)
        accs.append(np.mean(du_doan_mang(Zb_, W1, W2).argmax(axis=1) == yb));
Js.append(ls[-1])
    print(m, "test acc", np.round(accs, 3), "J train", np.round(Js, 4), "para
ms", 3 * m + (m + 1) * 3)
```

```
gradient check max err 2.5156851601870756e-10
2 test acc [0.628 0.7   0.694 0.694 0.7  ] J train [0.6927 0.652  0.6533 0.65
33 0.6519] params 15
4 test acc [0.983 0.989 0.989 0.989 0.961] J train [0.0702 0.0646 0.0586 0.06
4  0.0639] params 27
8 test acc [0.994 0.978 0.972 0.961 0.972] J train [0.0137 0.0193 0.019  0.02
14 0.0119] params 51
```

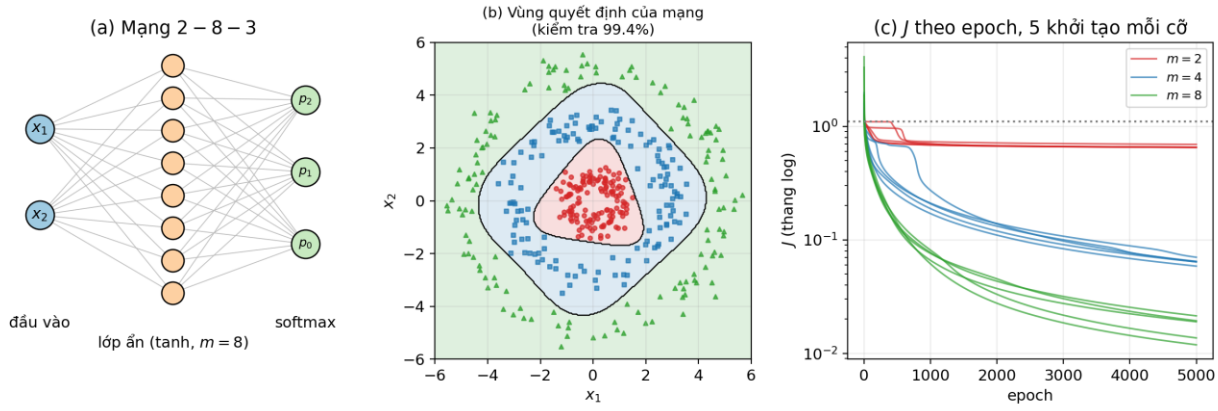
Gradient lan truyền ngược khớp đạo hàm số đến cỡ 10^{-10}. Kết quả trên tập kiểm tra gồm 180 điểm, so với các mô hình ở mục 4.5:

Mô hình	Đặc trưng đưa vào	Số tham số	Accuracy kiểm tra
Softmax Regression	x_1, x_2	9	35,0%
Softmax Regression + đa thức bậc 2 (tự chọn đặc trưng)	5 đặc trưng	18	99,4%
Mạng $2-2-3$	x_1, x_2	15	62,8% đến 70,0%
Mạng $2-4-3$	x_1, x_2	27	96,1% đến 98,9%
Mạng $2-8-3$	x_1, x_2	51	96,1% đến 99,4%

Có thể rút ra:

- Mạng với $m = 8$ chỉ nhận hai đặc trưng gốc x_1, x_2 mà vẫn đạt tới 99,4% (một khởi tạo), tương đương mô hình dùng đặc trưng đa thức do ta tự chọn. Các đường biên cong (Hình 20b) do mạng tự học, không cần biết trước "biên là vòng tròn". Cái giá là số tham số lớn hơn (51 so với 18) và không còn biết trước kết quả.
- Cỡ lớp ẩn quan trọng: $m = 2$ quá ít để tạo ra các vùng lồng nhau cho ba lớp, J dừng ở khoảng 0,65 (Hình 20c), còn $m = 4$ và $m = 8$ giảm J xuống 0,06 và 0,02.
- **Hàm mất mát không còn lồi.** Với $m \geq 4$, các khởi tạo khác nhau cho J cuối khác nhau (0,0586 đến 0,0702 với $m = 4$; 0,0119 đến 0,0214 với $m = 8$) và độ chính xác kiểm tra khác nhau (96,1% đến 99,4% với $m = 8$). Đây là khác biệt cơ bản so với Softmax Regression, nơi mọi khởi tạo cho cùng J^* và cùng $\mathbf{P}$ (mục 4.2 và 4.3).
- **Cần khởi tạo ngẫu nhiên.** Nếu khởi tạo $\mathbf{W}_1 = \mathbf{0}$ và $\mathbf{W}_2 = \mathbf{0}$ như ở Softmax Regression, thì $\mathbf{H} = \tanh(\mathbf{0}) = \mathbf{0}$, và gradient $\nabla_{\mathbf{W}_1} J = \tilde{\mathbf{X}}^T[(\mathbf{D}_2\mathbf{V}^T) \odot (\mathbf{1} - \mathbf{H} \odot \mathbf{H})]$ chứa $\mathbf{V} = \mathbf{0}$ nên bằng $\mathbf{0}$; các nơ-ron ẩn không bao giờ học (kể cả khi chạy 5000 epoch, ta kiểm tra được J đứng ở $\ln 3 = 1{,}0986$ và accuracy 33,3%). Ngay cả khi các nơ-ron ẩn khởi tạo giống hệt nhau, chúng nhận cùng

gradient nên luôn giống nhau. Vì vậy mạng cần khởi tạo ngẫu nhiên để phá đối xứng, trái với Softmax Regression (mục 4.2).



Hình 20. (a) Mạng $2 - 8 - 3$: hai đầu vào, tám nơ-ron ẩn (tanh) và lớp ra softmax chính là Softmax Regression trên các đặc trưng ẩn. (b) Vùng quyết định của mạng trên dữ liệu ba vòng tròn đồng tâm. (c) J theo epoch (thang log) với 5 khởi tạo khác nhau cho mỗi cỡ lớp ẩn m; đường chấm là $\ln 3$.

Học sâu: cùng công thức, nhiều lớp hơn

Mạng nhiều lớp ẩn hơn chỉ là lặp lại ý tưởng trên: $\mathbf{H}_1 = \varphi(\tilde{\mathbf{X}}\mathbf{W}_1)$, $\mathbf{H}_2 = \varphi(\tilde{\mathbf{H}}_1\mathbf{W}_2)$, … rồi $\mathbf{P} = \text{softmax}(\tilde{\mathbf{H}}_L\mathbf{W}_{L+1})$, và gradient lan truyền ngược qua từng lớp theo quy tắc đạo hàm hàm hợp. Một số điểm nối với những gì ta đã học:

- **Softmax là lớp đầu ra chuẩn.** Trong hầu hết mạng phân loại hiện đại (mạng tích chập, Transformer), điểm số $\mathbf{s}$ của lớp cuối thường gọi là *logits*, rồi qua softmax và cross-entropy. Gradient của lớp ra vẫn là $\mathbf{P} - \mathbf{Y}$ (chia cho số mẫu).
- **Điều chuẩn hóa L2 gọi là weight decay**, với cập nhật $\mathbf{W} \leftarrow (1 - \eta\lambda)\mathbf{W} - \eta\nabla J$ như ở mục 4.4. Ngoài ra còn dừng sớm, dropout…
- **Mini-batch SGD và các biến thể** (momentum, Adam) là cách huấn luyện chuẩn (mục 3.5), vì dữ liệu lớn và J không lồi.
- **Hàm kích hoạt.** Thay tanh bằng ReLU, $\varphi(z) = \max(0, z)$, giúp huấn luyện mạng sâu ổn định hơn.
- **Chuẩn hóa đầu vào, đánh giá bằng ma trận nhầm lẫn, log-loss, hiệu chuẩn** (mục 3.4 và 5) giữ nguyên vai trò. Các mạng sâu thường **quá tự tin**, nên hiệu chuẩn lại (ví dụ bằng cách chia logits cho một nhiệt độ, như ở mục 6.2) là bước phổ biến.

Ta có thể gom các mô hình đã học vào một "mạch" chung:

Mô hình	Điểm số	Đầu ra	Mất mát	Gradient theo điểm số
Hồi quy tuyến tính	$\mathbf{Xw}$	chính điểm số	bình phương sai số	$\propto \hat{\mathbf{y}} - \mathbf{y}$
Logistic Regression	$\mathbf{Xw}$	$\sigma(s)$	cross-entropy nhị phân	$\sigma(\mathbf{s}) - \mathbf{y}$
Softmax Regression	$\mathbf{XW}$	$\text{softmax}(\mathbf{s})$	cross-entropy nhiều lớp	$\mathbf{P} - \mathbf{Y}$
Mạng nơ-ron nhiều lớp (đầu ra softmax)	$\tilde{\mathbf{H}}\mathbf{W}_{L+1}$, với $\mathbf{H}$ do mạng học	$\text{softmax}(\mathbf{s})$	cross-entropy nhiều lớp	$\mathbf{P} - \mathbf{Y}$, rồi lan truyền ngược

Cả ba mô hình đầu đều thuộc họ mô hình tuyến tính tổng quát với dạng gradient chung $\mathbf{X}^T(\text{dự đoán} - \text{thực tế})$, còn mạng nơ-ron mở rộng họ này bằng cách để các đặc trưng được học ra.

6.4. Bài tập thực hành

Các bài tập sắp xếp từ cơ bản đến nâng cao. Phần A là tính tay và lý thuyết, phần B là lập trình, phần C là mở rộng.

Phần A. Tính tay và lý thuyết

A1. Cho $K = 3$ và vector điểm số $\mathbf{s} = [2 \;\; 0 \;\; -1]$. Tính $\mathbf{p} = \text{softmax}(\mathbf{s})$. Nếu lớp thật là lớp 2 (phần tử cuối), tính mất mát cross-entropy và gradient $\mathbf{p} - \mathbf{y}$ theo điểm số.

A2. Cho hai mẫu một đặc trưng, ma trận $\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 1 & 2 \end{bmatrix}$ (cột đầu là hệ số chệch), nhãn $\mathbf{y} = [0 \;\; 2]$ ($K = 3$) và $\mathbf{W}^{(0)} = \mathbf{0}$. Tính $\mathbf{Y}$, $J(\mathbf{W}^{(0)})$, $\nabla_{\mathbf{W}} J$, $\mathbf{W}^{(1)}$ với $\eta = 1$, và $J(\mathbf{W}^{(1)})$.

A3. Chứng minh Softmax Regression với $K = 2$ tương đương Logistic Regression với $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_0$. Từ quy tắc cập nhật của hai cột, chứng minh quỹ đạo của $\mathbf{w}_1 - \mathbf{w}_0$ trùng với quỹ đạo của Logistic Regression có tốc độ học 2η (khởi tạo $\mathbf{0}$).

A4. Một nghiệm tối ưu có $\mathbf{W}^* = \begin{bmatrix} 1 & -1 & 0 \\ 2 & 0 & -2 \end{bmatrix}$ (hàng đầu là hệ số chệch). Cộng $\mathbf{v}\mathbf{1}_K^T$ với $\mathbf{v} = [1 \;\; 3]^T$ vào $\mathbf{W}^*$ được $\mathbf{W}'$. Với mẫu $\mathbf{x} = [1 \;\; 1]$, tính điểm số và xác suất của hai ma trận và so sánh. Điều này cho biết gì về tính duy nhất của nghiệm?

A5. Dùng $\mathbf{W}_{\text{gốc}}$ ở mục 6.1, tính hệ số nhân của odds "giỏi : đạt" khi thêm một buổi vắng, và hệ số nhân của odds "đạt : rớt" khi thêm 2 giờ học.

A6. Cho $\mathbf{s} = [1 \;\; 2 \;\; 3]$. Tính $\mathbf{p}$ và tỉ lệ p_1/p_0. Bỏ lớp thứ ba, tính lại $\mathbf{p}$ cho hai lớp và tỉ lệ p_1/p_0. Rút ra gì về tính độc lập với phương án không liên quan? Với ví dụ ô tô và hai xe buýt (ô tô và xe buýt đỏ ban đầu có xác suất $\frac{1}{2}$ mỗi loại), mô hình softmax dự đoán gì sau khi thêm xe buýt xanh có cùng điểm số với xe buýt đỏ?

A7. Tính $\text{softmax}(c\,\mathbf{s})$ với $\mathbf{s} = [1 \;\; 2 \;\; 3]$ và $c = 1$, $c = 2$. Điều gì xảy ra khi $c \to 0$ và $c \to \infty$?

A8. Một mạng có $d = 4$ đặc trưng, $m = 10$ nơ-ron ẩn và $K = 3$ lớp. Tính số tham số và kích thước của $\nabla_{\mathbf{W}_1} J$, $\nabla_{\mathbf{W}_2} J$. So với Softmax Regression trên 4 đặc trưng. Chuyện gì xảy ra nếu khởi tạo $\mathbf{W}_1 = \mathbf{0}$ và $\mathbf{W}_2 = \mathbf{0}$?

Phần B. Lập trình

B1. Cài đặt Softmax Regression từ đầu bằng NumPy (dùng các hàm ở mục 3.7). Huấn luyện trên bộ dữ liệu hoa Iris (4 đặc trưng, 3 lớp; chia $70/30$ có phân tầng, `random_state=0`, chuẩn hóa theo tập huấn luyện), với $\eta = 0{,}5$ và 2000 epoch. Kiểm tra gradient bằng đạo hàm số và in J ban đầu (kỳ vọng $\ln 3$). Ghi lại J cuối cùng, accuracy huấn luyện và kiểm tra, rồi so sánh với `LogisticRegression` của scikit-learn.

B2. Với mô hình ở B1, tính Hessian $\mathbf{H} = \frac{1}{n}\sum_i \mathbf{A}_i \otimes \mathbf{x}_i^T \mathbf{x}_i$ tại $\mathbf{W}^*$ (hàm `hessian` ở mục 4.3), tính các trị riêng, và kiểm tra số trị riêng bằng 0 cùng hạng của $\mathbf{H}$. So sánh với dự đoán $d + 1$ và $(K - 1)(d + 1)$.

B3. Trên dữ liệu 9 sinh viên (đặc trưng chuẩn hóa), cài đặt điều chuẩn hóa L2 như ở mục 4.4 và huấn luyện với $\lambda \in \{0;\ 0{,}001;\ 0{,}01;\ 0{,}1\}$ ($\eta = 0{,}5$, 10^5 bước). Vẽ hoặc in $\| \widetilde{\mathbf{W}} \|_F$ và xác suất của sinh viên 6 theo λ.

B4. Tái hiện kết quả mục 5 trên bộ dữ liệu 600 sinh viên: ma trận nhầm lẫn, precision/recall/F1 theo từng lớp và macro-F1, log-loss, ECE và AUC one-vs-rest. Tự cài đặt các chỉ số (không dùng thư viện) rồi đối chiếu với `classification_report` và `roc_auc_score`.

B5. Dùng bộ dữ liệu chữ số viết tay `load_digits` của scikit-learn (64 đặc trưng, 10 lớp). Chia $70/30$ có phân tầng (`random_state=0`), dùng pipeline `StandardScaler` và `LogisticRegression(C=1)`. Báo cáo accuracy, macro-F1, log-loss, top-3 accuracy và AUC one-vs-rest, rồi tìm cặp lớp bị nhầm lẫn nhiều nhất trong ma trận nhầm lẫn 10×10.

B6. Trên cùng dữ liệu ở B5, so sánh Softmax Regression với one-vs-rest (`OneVsRestClassifier(LogisticRegression(C=1))`) theo accuracy, macro-F1 và log-loss. Nhận xét.

B7. Trên dữ liệu 600 sinh viên, huấn luyện Softmax Regression ở các giá trị $\eta \in \{0{,}01;\ 0{,}1;\ 0{,}5;\ 2;\ 10\}$ trên đặc trưng chuẩn hóa và trên đặc trưng gốc, ghi lại số epoch để J cách J^* không quá $0{,}001$ (hoặc "không hội tụ"). Đối chiếu với bảng ở mục 4.2 và ngưỡng $2/\lambda_{\max}(\mathbf{H})$ ở mục 4.3.

Phần C. Mở rộng

C1. Cài đặt mạng một lớp ẩn với đầu ra softmax (hàm `mang_softmax` ở mục 6.3) và kiểm tra gradient bằng đạo hàm số. Huấn luyện trên ba vòng tròn đồng tâm với $m = 2{,}4{,}8$ nơ-ron ẩn, mỗi cỡ 5 khởi tạo khác nhau. Báo cáo J cuối cùng và accuracy kiểm tra. Có thể giải thích vì sao $m = 2$ không đủ?

C2. Kiểm chứng rằng mạng ở C1 không học được khi khởi tạo $\mathbf{W}_1 = \mathbf{W}_2 = \mathbf{0}$. Thử khởi tạo $\mathbf{W}_1$ bằng một hằng số khác 0 (mọi phần tử giống nhau) và quan sát các nơ-ron ẩn.

C3. Tạo bộ dữ liệu mất cân bằng từ 600 sinh viên: giữ 140 mẫu huấn luyện của lớp rớt và đạt nhưng chỉ 14 mẫu của lớp giỏi (tập kiểm tra giữ nguyên). Huấn luyện `LogisticRegression(C=10)` với `class_weight=None` và `class_weight="balanced"`, so sánh accuracy, macro-F1 và recall của lớp giỏi. Giải thích vai trò của trọng số lớp trong hàm mất mát $J_w(\mathbf{W}) = -\frac{1}{n}\sum_i w_{y_i} \ln q_i$.

C4. Một hệ thống gắn thẻ cho bài báo: mỗi bài có thể có nhiều thẻ cùng lúc (ví dụ "kinh tế" và "chính trị"). Nên dùng softmax hay K sigmoid độc lập? Viết hàm mất mát tương ứng và công thức gradient theo điểm số.

Gợi ý đáp án

A1. $e^2 = 7{,}389$, $e^0 = 1$, $e^{-1} = 0{,}368$, tổng $= 8{,}757$, nên $\mathbf{p} \approx [0{,}844 \;\; 0{,}114 \;\; 0{,}042]$. Lớp thật là lớp 2 nên mất mát $-\ln 0{,}042 \approx 3{,}170$, và gradient theo điểm số là $\mathbf{p} - \mathbf{y} \approx [0{,}844 \;\; 0{,}114 \;\; -0{,}958]$ (các phần tử cộng lại bằng 0).

A2. $\mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$. Với $\mathbf{W}^{(0)} = \mathbf{0}$ thì $\mathbf{P}$ toàn $\frac{1}{3}$ và $J = \ln 3 \approx 1{,}0986$. Ta có $\mathbf{P} - \mathbf{Y} = \begin{bmatrix} -\frac{2}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & -\frac{2}{3} \end{bmatrix}$ và

$$\nabla_{\mathbf{W}} J = \frac{1}{2}\mathbf{X}^T(\mathbf{P} - \mathbf{Y}) = \begin{bmatrix} -\frac{1}{6} & \frac{1}{3} & -\frac{1}{6} \\ \frac{1}{3} & \frac{1}{3} & -\frac{2}{3} \end{bmatrix}$$

(hàng đầu là hệ số chệch, hàng hai là đặc trưng). Với $\eta = 1$, $\mathbf{W}^{(1)} = -\nabla_{\mathbf{W}} J = \begin{bmatrix} \frac{1}{6} & -\frac{1}{3} & \frac{1}{6} \\ -\frac{1}{3} & -\frac{1}{3} & \frac{2}{3} \end{bmatrix}$.

Điểm số mới: $\mathbf{s}_1 = \begin{bmatrix} \frac{1}{6} & -\frac{1}{3} & \frac{1}{6} \end{bmatrix}$ và $\mathbf{s}_2 = \begin{bmatrix} -\frac{1}{2} & -1 & \frac{3}{2} \end{bmatrix}$; do đó $\mathbf{p}_1 \approx [0{,}384 \;\; 0{,}233 \;\; 0{,}384]$, $\mathbf{p}_2 \approx [0{,}111 \;\; 0{,}067 \;\; 0{,}821]$ và $J\left(\mathbf{W}^{(1)}\right) = \frac{1}{2}(-\ln 0{,}384 - \ln 0{,}821) \approx 0{,}577$, giảm so với 1,0986.

A3. $p_1 = \frac{e^{s_1}}{e^{s_0} + e^{s_1}} = \frac{1}{1 + e^{-(s_1 - s_0)}} = \sigma\big(\mathbf{x}(\mathbf{w}_1 - \mathbf{w}_0)\big)$ (mục 2.3), và cross-entropy hai lớp là cross-entropy nhị phân của p_1 (mục 3.1). Vì $\mathbf{p}_{:,0} - \mathbf{y}_{:,0} = -\big(\mathbf{p}_{:,1} - \mathbf{y}_{:,1}\big)$, ta có $\nabla_{\mathbf{w}_0} J = -\nabla_{\mathbf{w}_1} J$. Cập nhật cả hai cột rồi trừ nhau: $(\mathbf{w}_1 - \mathbf{w}_0)' = (\mathbf{w}_1 - \mathbf{w}_0) - 2\eta \nabla_{\mathbf{w}_1} J$, mà $\nabla_{\mathbf{w}_1} J = \frac{1}{n}\mathbf{X}^T(\sigma(\mathbf{X}\mathbf{w}) - \mathbf{y})$ chính là gradient của Logistic Regression, nên quỹ đạo của $\mathbf{w}$ trùng với Logistic Regression có tốc độ học 2η.

A4. $\mathbf{W}' = \mathbf{W}^* + \mathbf{v}\mathbf{1}_K^T = \begin{bmatrix} 2 & 0 & 1 \\ 5 & 3 & 1 \end{bmatrix}$. Với $\mathbf{x} = [1 \;\; 1]$: $\mathbf{s}^* = \mathbf{x}\mathbf{W}^* = [3 \;\; -1 \;\; -2]$ và $\mathbf{s}' = [7 \;\; 3 \;\; 2] = \mathbf{s}^* + 4$ (vì $\mathbf{x}\mathbf{v} = 4$). Cộng cùng một hằng số vào mọi điểm số không đổi softmax, nên $\mathbf{p}$ giống hệt nhau ($\approx [0{,}976 \;\; 0{,}018 \;\; 0{,}007]$). Vậy $\mathbf{W}'$ cũng tối ưu: nghiệm không duy nhất, chỉ $\mathbf{P}$ là duy nhất (mục 4.3).

A5. Odds giỏi:đạt khi thêm một buổi vắng nhân với $e^{w_{2,\text{giỏi}}-w_{2,\text{đạt}}} = e^{-0{,}569-(-0{,}244)} = e^{-0{,}325} \approx 0{,}722$. Odds đạt:rớt khi thêm một giờ nhân với $e^{0{,}081-(-1{,}408)} = e^{1{,}489} \approx 4{,}43$, nên thêm 2 giờ nhân với $4{,}43^2 = e^{2{,}978} \approx 19{,}6$.

A6. Với $\mathbf{s} = [1\ \ 2\ \ 3]$: $\mathbf{p} \approx [0{,}090\ \ 0{,}245\ \ 0{,}665]$ và $p_1/p_0 = e \approx 2{,}718$. Bỏ lớp thứ ba: $\mathbf{p} \approx [0{,}269\ \ 0{,}731]$ và $p_1/p_0 = e$. Tỉ lệ giữa hai lớp không phụ thuộc lớp thứ ba (tính chất IIA). Với ví dụ xe buýt, ô tô và xe buýt đỏ ban đầu có tỉ lệ 1; sau khi thêm xe buýt xanh có cùng điểm số với xe buýt đỏ, tỉ lệ ô tô : xe buýt đỏ vẫn bằng 1, và hai xe buýt có xác suất bằng nhau nên mô hình dự đoán $\left[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right]$, trong khi thực tế người dùng sẽ chia đôi phần xe buýt: $\left[\frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right]$.

A7. $c = 1$: $\mathbf{p} \approx [0{,}090\ \ 0{,}245\ \ 0{,}665]$. $c = 2$: $\text{softmax}([2\ \ 4\ \ 6]) \approx [0{,}016\ \ 0{,}117\ \ 0{,}867]$. Khi $c \to 0$ mọi điểm số $c\,s_k \to 0$ nên phân phối tiến tới đều $\left[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right]$. Khi $c \to \infty$ xác suất dồn vào lớp có điểm số lớn nhất (one-hot), và $\frac{1}{c}\ell_i \to \max_j s_{ij} - s_{i,y_i}$ (mục 6.2).

A8. Số tham số: $(d+1)m + (m+1)K = 5 \cdot 10 + 11 \cdot 3 = 83$, so với $(d+1)K = 15$ của Softmax Regression. $\nabla_{\mathbf{W}_1} J$ có kích thước 5×10 và $\nabla_{\mathbf{W}_2} J$ có kích thước 11×3. Nếu $\mathbf{W}_1 = \mathbf{0}$ và $\mathbf{W}_2 = \mathbf{0}$ thì $\mathbf{H} = \tanh(\mathbf{0}) = \mathbf{0}$ nên chỉ hàng chệch của $\mathbf{W}_2$ nhận gradient khác 0; vì $\mathbf{V} = \mathbf{0}$, $\nabla_{\mathbf{W}_1} J = \mathbf{0}$ và $\mathbf{W}_1$ không bao giờ cập nhật. Mạng suy biến thành mô hình chỉ có hệ số chệch (dự đoán theo tỉ lệ lớp). Vì vậy phải khởi tạo ngẫu nhiên.

B1. Kết quả tham khảo (khi chạy đúng như mô tả): J ban đầu $= \ln 3 = 1{,}0986$, J cuối $\approx 0{,}0634$, accuracy huấn luyện $\approx 97{,}1\%$ (105 mẫu) và accuracy kiểm tra $= 100\%$ (45 mẫu); gradient giải tích khớp đạo hàm số đến cỡ 10^{-10} hoặc nhỏ hơn. `LogisticRegression` cho kết quả tương tự (cùng accuracy kiểm tra).

B2. Với Iris, $d = 4$ và $K = 3$ nên $\mathbf{H}$ có kích thước 15×15, kỳ vọng có đúng $d + 1 = 5$ trị riêng bằng 0 (cỡ 10^{-17}) và hạng $(K-1)(d+1) = 10$. Các trị riêng dương còn lại có thể rất nhỏ (cỡ 10^{-4}) vì Iris gần như phân tách tuyến tính, nên cần phân biệt "bằng 0" (hướng $\mathbf{v}\mathbf{1}_K^T$) với "rất nhỏ nhưng dương".

B3. Kết quả tham khảo (mục 4.4): $\|\,\widetilde{\mathbf{W}}\,\|_F \approx 23{,}0$ (và còn tăng nếu chạy thêm), 9,08, 4,68 và 1,81 ứng với $\lambda = 0;\ 0{,}001;\ 0{,}01;\ 0{,}1$; xác suất của sinh viên 6 đi từ $[0{,}000\ \ 1{,}000\ \ 0{,}000]$ đến $[0{,}334\ \ 0{,}589\ \ 0{,}077]$.

B4. Kết quả tham khảo: ma trận nhầm lẫn $\begin{bmatrix} 49 & 11 & 0 \\ 9 & 48 & 3 \\ 0 & 4 & 56 \end{bmatrix}$, F1 theo lớp $0{,}831;\ 0{,}780;\ 0{,}941$, macro-F1 $0{,}851$, log-loss $0{,}3536$, ECE $0{,}050$, AUC theo lớp $0{,}969;\ 0{,}920;\ 0{,}993$ (macro $0{,}961$).

B5. Kết quả tham khảo: accuracy kiểm tra $\approx 97{,}2\%$ (trên 540 mẫu), macro-F1 $\approx 0{,}972$, log-loss $\approx 0{,}102$, top-3 accuracy $= 100\%$, AUC one-vs-rest $\approx 0{,}999$. Trong ma trận nhầm lẫn 10×10, nhầm lẫn nhiều nhất là chữ số 8 bị đoán thành 1 (khoảng 3 mẫu). Số liệu có thể khác chút ít tùy phiên bản thư viện.

B6. Kết quả tham khảo: one-vs-rest đạt accuracy $\approx 96{,}9\%$, macro-F1 $\approx 0{,}969$ và log-loss $\approx 0{,}138$ (sau khi chuẩn hóa xác suất), kém Softmax ở cả ba chỉ số (97,2%; 0,972; 0,102). Khoảng

cách nhỏ về accuracy vì $K = 10$ lớp ở đây gần như tách được tuyến tính từng lớp khỏi phần còn lại, nhưng log-loss khác biệt rõ hơn vì xác suất của Softmax được hiệu chuẩn tốt hơn (tổng bằng 1).

B7. Kết quả tham khảo (đặc trưng chuẩn hóa): $\eta = 0{,}01$ cần 24569 epoch, $\eta = 0{,}1$ cần 2455, $\eta = 0{,}5$ cần 490, $\eta = 2$ cần 121, $\eta = 10$ dao động quanh $J \approx 0{,}405$ (không hội tụ). Trên đặc trưng gốc: $\eta = 0{,}05$ cần 16341, $\eta = 0{,}1$ cần 8170 epoch, còn $\eta \geq 0{,}2$ dao động. Các ngưỡng này khớp với $2/\lambda_{\max}(\mathbf{H}) \approx 8{,}75$ (chuẩn hóa) và $\approx 0{,}199$ (gốc).

C1. Kết quả tham khảo (đã chạy trong mục 6.3): $m = 2$ cho accuracy kiểm tra $62{,}8\%$ đến $70{,}0\%$ và $J \approx 0{,}65$; $m = 4$ cho $96{,}1\%$ đến $98{,}9\%$; $m = 8$ cho $96{,}1\%$ đến $99{,}4\%$. Với $m = 2$, hai nơ-ron ẩn chỉ tạo được hai đường cong chia mặt phẳng, không đủ để tạo cả vùng giữa (hình tròn) lẫn vùng vành khuyên cho ba lớp.

C2. Với $\mathbf{W}_1 = \mathbf{W}_2 = \mathbf{0}$, $\mathbf{W}_1$ không bao giờ cập nhật; J đứng ở $\ln 3 \approx 1{,}0986$ và accuracy $\frac{1}{3}$ (ta đã kiểm tra: sau 5000 epoch $\mathbf{W}_1$ và hàng ẩn của $\mathbf{W}_2$ vẫn bằng 0). Với khởi tạo hằng số khác 0, các nơ-ron ẩn giống hệt nhau nên nhận cùng gradient và luôn giống nhau (đối xứng không bao giờ bị phá), mạng chỉ hoạt động như có một nơ-ron ẩn. Vì vậy cần khởi tạo ngẫu nhiên.

C3. Kết quả tham khảo: không dùng trọng số lớp, accuracy kiểm tra $75{,}0\%$, macro-F1 $0{,}753$ và recall lớp giỏi chỉ $56{,}7\%$ (mô hình hay đoán sinh viên giỏi thành đạt: 26 mẫu); với `class_weight="balanced"`, accuracy $82{,}2\%$, macro-F1 $0{,}825$ và recall lớp giỏi $83{,}3\%$. Trọng số w_k tỉ lệ nghịch với số mẫu của lớp k làm mẫu lớp hiếm đóng góp nhiều hơn vào J_w, nên mô hình không bỏ qua chúng (đổi lại precision lớp giỏi giảm từ $1{,}00$ xuống $0{,}94$).

C4. Dùng K sigmoid độc lập, vì các thẻ không loại trừ nhau (softmax ép $\sum_k p_k = 1$). Hàm mất mát là tổng cross-entropy nhị phân cho từng thẻ: $J = -\frac{1}{n}\sum_i \sum_{k=0}^{K-1}\big[y_{ik}\ln\sigma(s_{ik}) + (1 - y_{ik})\ln\big(1 - \sigma(s_{ik})\big)\big]$ với $y_{ik} \in \{0{,}1\}$ độc lập giữa các k. Gradient theo điểm số là $\sigma(s_{ik}) - y_{ik}$ (chia cho n), tức $\frac{1}{n}\mathbf{X}^T(\sigma(\mathbf{S}) - \mathbf{Y})$ với $\mathbf{Y} \in \{0{,}1\}^{n \times K}$ có thể nhiều giá trị 1 trên mỗi hàng.